

UNIVERSIDAD DEL VALLE

Programación Funcional y Concurrente

El problema de reconstrucción de cadenas - Proyecto

Libardo Alejandro Quintero Gómez - 2342181
Jerson Alexis Ortiz Velazco - 2410014
Juan Carlos Cruz Muñoz - 1824389
Jairo Hernan Gonzalez Barreto - 2324314

24 de julio del 2025

1. Introducción

El Problema de Reconstrucción de Cadenas (PRC) consiste en reconstruir una secuencia oculta S de longitud conocida N a partir de un oráculo o que, dado cualquier candidato w, devuelve verdadero si w aparece en S como subcadena consecutiva y falso en caso contrario. Este modelo abstrae situaciones prácticas, como la secuenciación de ADN, donde cada consulta al laboratorio es costosa. El reto principal radica en minimizar el número de consultas a o para hallar la cadena completa de manera eficiente.

La primera aproximación, denominada *solución ingenua*, explora exhaustivamente todas las posibles secuencias de longitud N hasta identificar S. Aunque garantiza el éxito, su coste crece exponencialmente con N. Para mejorar, la *solución mejorada* construye S carácter a carácter: en cada paso extiende las cadenas parciales solo con aquellos caracteres cuya concatenación es aceptada por o, reduciendo así drásticamente el espacio de búsqueda.

Partiendo de estos dos métodos, la estrategia *turbo* acelera el proceso al combinar progresivamente subcadenas ya validadas, de modo que en cada fase se trabaja con bloques de longitud creciente y se consultan únicamente las uniones que pueden conducir a la cadena final. La *turbo mejorada* añade un filtro interno que verifica todas las subcadenas intermedias de cada candidato antes de llamar a o, evitando consultas redundantes y optimizando aún más el uso del oráculo. Finalmente, la *turbo acelerada* incorpora una estructura de sufijos (trie) para almacenar en memoria los fragmentos validados y realizar búsquedas de subcadenas de forma directa y rápida, lo que reduce tanto el número de operaciones lógicas como las llamadas a o.

Cada una de estas cinco soluciones se implementa inicialmente en su versión secuencial utilizando Scala, aprovechando expresiones funcionales y LazyLists para gestionar generación y filtrado de candidatos. A continuación, se adapta cada algoritmo a un entorno concurrente, distribuyendo las tareas de generación y verificación de subcadenas entre múltiples hilos. Finalmente, mediante pruebas de rendimiento sobre distintas longitudes de S, se comparan los tiempos de ejecución y las aceleraciones obtenidas, con el fin de identificar las ventajas y limitaciones de cada enfoque tanto a nivel algorítmico como de paralelismo.

2. Informe de uso de técnicas y estructura de datos

Esta sección del informe presenta un análisis detallado de las técnicas de programación funcional aplicadas en el desarrollo del proyecto, junto con la implementación de estructuras de datos diseñadas para resolver el problema de forma eficiente. Se explicará el uso adecuado de conceptos fundamentales como la recursión, el reconocimiento de patrones, la encapsulación, las funciones de alto orden y los iteradores, mostrando cómo estos contribuyen a la claridad, modularidad y corrección del código. Asimismo, se discutirá la selección de estructuras de datos adecuadas para optimizar el manejo de colecciones de secuencias.

2.1. Técnicas usadas en cada función

Se trabajará el análisis de las técnicas funcionales utilizadas en las diferentes funciones implementadas. El objetivo es identificar y documentar cuáles herramientas del paradigma funcional se aplicaron en cada caso, y cómo estas apoyaron la solución del problema.

A continuación se muestra una tabla general en la que se especifican las técnicas de programación funcional empleadas en cada función. Estas se explicarán con detalle más adelante:

Función	Recursión	Colecciones	Recon. Patrones	Alto Orden	Encapsulación	for
`pertenece`	Sí	Sí	Sí	Sí	No	No
`adicionar`	Sí	Sí	Sí	Sí	Sí	No
`arbolDeSufijos`	No	Sí	No	Sí	Sí	Sí
`reconstruirCadenaIngenuo`	Sí	Sí	No	Sí	Sí	Sí
`reconstruirCadenaMejorado`	Sí	Sí	No	Sí	Sí	Sí
`reconstruirCadenaTurbo`	Sí	Sí	No	Sí	Sí	Sí
`reconstruirCadenaTurboMejorada`	Sí	Sí	No	Sí	Sí	Sí
`reconstruirCadenaTurboAcelerada`	Sí	Sí	No	Sí	Sí	Sí

Cuadro 1: *
Presencia de técnicas de programación funcional en cada función.

2.1.1. Solución ingenua: `reconstruirCadenaIngenuo`

Para abordar el problema de la reconstrucción de la cadena oculta S mediante un enfoque exhaustivo, se ha diseñado una variante de la solución por fuerza bruta que explora todo el espacio de posibles cadenas de longitud n sobre el alfabeto $\Sigma = a, c, g, t$. El objetivo es identificar la secuencia correcta únicamente mediante consultas a un oráculo Ψ_S, el cual verifica si una cadena candidata coincide con la buscada.

Formalmente, el espacio de búsqueda se corresponde con el conjunto cartesiano

$$\Sigma^n = \{(s_1, s_2, \ldots, s_n) \mid s_i \in \Sigma\}$$

por lo que el número total de cadenas posibles es $|\Sigma|^n = 4^n$.

Para explorar exhaustivamente este espacio, se implementa un generador perezoso de todas las combinaciones posibles de longitud n. Es decir, las cadenas candidatas no se construyen ni almacenan en memoria de manera anticipada, sino que se generan únicamente cuando es necesario evaluarlas. Esto se logra mediante el uso de listas perezosas (`LazyList`), que permiten recorrer el conjunto Σ^n de forma eficiente, evitando un consumo excesivo de recursos. En términos funcionales, la función generadora construye recursivamente todas las secuencias posibles concatenando, en cada paso, cada símbolo del alfabeto a todas las combinaciones de longitud $n-1$, logrando así una cobertura exhaustiva del espacio de búsqueda.

Con el objetivo de organizar y facilitar la exploración del espacio de soluciones, el conjunto de todas las combinaciones generadas se divide en ocho segmentos de igual tamaño. Matemáticamente, esto equivale a particionar el conjunto Σ^n en 8 subconjuntos disjuntos:

$$\Sigma^n = A_1 \cup A_2 \cup \cdots \cup A_8$$

Donde cada segmento A_i contiene aproximadamente $\frac{|\Sigma|^n}{8}$ cadenas, y cada subconjunto A_i está determinado por un rango específico de índices dentro del orden de generación lexicográfica. Es decir,

$$A_i = \left\{ s \in \Sigma^n : \text{ind}(s) \in \left[(i-1) \cdot \frac{|\Sigma|^n}{8},\ i \cdot \frac{|\Sigma|^n}{8} \right) \right\}$$

Donde $\text{ind}(s)$ representa la posición de la cadena s en la enumeración lexicográfica de Σ^n.

Esta segmentación no altera la exhaustividad del procedimiento, ya que cada posible cadena candidata se encuentra exactamente en uno de los subconjuntos A_i. Así, al explorar todos los segmentos, se garantiza que todas las cadenas del espacio Σ^n son evaluadas, asegurando la correctitud del método: si existe una solución, será encontrada.

Esta organización permite, además, que cada segmento sea procesado de manera independiente, consultando para cada cadena candidata el oráculo Ψ_S hasta hallar la solución. Adicionalmente, esta segmentación sienta las bases para optimizaciones futuras, como la evaluación paralela, preservando el estilo declarativo y funcional de la implementación.

A continuación se muestra la implementación en Scala de la función `reconstruirCadenaIngenuo`, que materializa este enfoque segmentado sobre listas perezosas:

```scala
def reconstruirCadenaIngenuo(n: Int, o: Oraculo): Seq[Char] = {
  def generarCombinaciones(n: Int): LazyList[Seq[Char]] = {
    if (n == 0) LazyList(Seq.empty)
    else for {
      suf <- generarCombinaciones(n - 1)
      c <- LazyList.from(alfabeto)
    } yield c +: suf
  }

  val combinaciones = generarCombinaciones(n)

  val total = math.pow(alfabeto.length, n).toInt
  val octavo = total / 8

  val ((res1, res2, res3, res4), (res5, res6, res7, res8)) = (
    (
      combinaciones.slice(0 * octavo, 1 * octavo).find(o),
      combinaciones.slice(1 * octavo, 2 * octavo).find(o),
      combinaciones.slice(2 * octavo, 3 * octavo).find(o),
      combinaciones.slice(3 * octavo, 4 * octavo).find(o)
    ),
    (
      combinaciones.slice(4 * octavo, 5 * octavo).find(o),
      combinaciones.slice(5 * octavo, 6 * octavo).find(o),
      combinaciones.slice(6 * octavo, 7 * octavo).find(o),
      combinaciones.slice(7 * octavo, total).find(o)
    )
  )

  res1.orElse(res2).orElse(res3).orElse(res4)
    .orElse(res5).orElse(res6).orElse(res7).orElse(res8)
    .getOrElse(Seq.empty)
}
```

Listing 1: Implementación de `reconstruirCadenaIngenuo`

Como puede observarse, la función genera perezosamente todas las combinaciones posibles de longitud n, las divide en ocho segmentos, y busca en cada uno de ellos de manera secuencial la cadena que cumple el criterio establecido por el oráculo.

Uso de función auxiliar: `generarCombinaciones` Para construir el conjunto Σ^n de manera eficiente y perezosa, se utiliza una función auxiliar denominada `generarCombinaciones`, definida de forma recursiva dentro de la propia función principal `reconstruirCadenaIngenuo`. Esta función se encarga de generar todas las combinaciones posibles de longitud n a partir del alfabeto, produciendo cada cadena sólo cuando es requerida.

```scala
def generarCombinaciones(n: Int): LazyList[Seq[Char]] = {
    if (n == 0) LazyList(Seq.empty)
    else for {
        suf <- generarCombinaciones(n - 1)
        c <- LazyList.from(alfabeto)
    } yield c +: suf
}
```

Listing 2: Función auxiliar generarCombinaciones

Esta función explota las ventajas de las listas perezosas (`LazyList`), permitiendo representar potencialmente grandes espacios de búsqueda sin sobrecargar la memoria, ya que las secuencias se calculan bajo demanda. La definición es puramente funcional: utiliza recursión y expresiones `for` para construir, en cada paso, todas las extensiones posibles de las combinaciones previas, comenzando con la cadena vacía y extendiéndola con cada símbolo del alfabeto.

- **Recursión de árbol**: La función `generarCombinaciones` utiliza recursión en forma de árbol porque, para cada llamada con parámetro n, invoca recursivamente `generarCombinaciones`$(n-1)$ y, a partir de cada secuencia retornada, genera tantas ramas como caracteres haya en el alfabeto. De este modo, en el nivel n se crean múltiples subproblemas de tamaño $n-1$, cada uno de los cuales se expande con todos los símbolos posibles, conformando una estructura de llamadas que crece exponencialmente y se ramifica en cada nivel del árbol recursivo.

- `LazyList` (listas perezosas): La utilización de `LazyList` permite que la generación de combinaciones sea *perezosa*: las cadenas no se construyen ni almacenan en memoria a menos que sean requeridas por la evaluación posterior. Esto es fundamental cuando el espacio de búsqueda es exponencial, ya que permite manejar espacios potencialmente enormes sin agotar la memoria del sistema. Así, se garantiza eficiencia espacial y se facilita la exploración secuencial o parcial del espacio de soluciones.

- Expresiones `for`: Proporciona una forma declarativa, legible y concisa de expresar combinaciones anidadas. En este caso, se utiliza para combinar todas las cadenas previamente generadas (de longitud $n-1$) con cada uno de los símbolos del alfabeto, generando así todas las cadenas posibles de longitud n. Esto elimina la necesidad de bucles imperativos y refuerza el estilo funcional e inmutable del código.

Análisis de la implementación en Scala

El comportamiento global de la función `reconstruirCadenaIngenuo` puede analizarse en dos etapas diferenciadas. En la primera etapa, se utiliza la función auxiliar `generarCombinaciones` para construir recursivamente todas las posibles cadenas de longitud n sobre el alfabeto, representadas como una secuencia que se explora según se necesite.

En la segunda etapa, esta secuencia se divide equitativamente en ocho partes utilizando la operación `slice`, lo que permite organizar el espacio de búsqueda en segmentos independientes. Para cada segmento, se aplica la función `find`, que examina las cadenas del segmento una a una hasta encontrar la primera que cumple el criterio especificado por el oráculo `o`. Posteriormente, los resultados de los distintos segmentos se combinan mediante sucesivas aplicaciones de `orElse`, devolviendo así la primera coincidencia encontrada en cualquiera de los segmentos.

Por último, se utiliza la función `getOrElse(Seq.empty)` sobre el resultado, que asegura que la función siempre retorne una secuencia válida. Esto es relevante porque `find` devuelve un valor de tipo `Option`, lo que obliga a considerar la posibilidad de no encontrar ninguna coincidencia; sin embargo, bajo el supuesto de que el oráculo está correctamente implementado y existe una única cadena válida de longitud n, este caso no debería presentarse en la práctica.

Técnicas de programación funcional utilizadas:

- **Recursión de árbol**: Empleada en la función auxiliar `generarCombinaciones` para construir, de manera natural y compacta, todas las posibles secuencias de longitud n sobre el alfabeto.
- **Evaluación perezosa con `LazyList`**: Permite representar el espacio Σ^n sin calcular ni almacenar todas las combinaciones a priori, procesando únicamente aquellas que sean necesarias durante la búsqueda.
- **Partición funcional del espacio de búsqueda**: Mediante el uso de la función `slice`, el conjunto de candidatos se divide en subconjuntos A_i, facilitando la organización y la posible exploración paralela.
- **Funciones de alto orden**: Uso de funciones como `find` para buscar el primer elemento que satisface el predicado del oráculo, y `orElse` para combinar resultados de segmentos de manera elegante y concisa.
- **Expresiones `for`**: Utilizadas en la generación de combinaciones, permitiendo expresar la construcción de secuencias de forma declarativa y sin bucles imperativos.
- **Inmutabilidad**: Todas las colecciones y estructuras utilizadas son inmutables, lo que facilita el razonamiento y la composición del código.
- **Manejo seguro de resultados**: Uso del tipo `Option` junto con `getOrElse` para garantizar que siempre se retorna un valor adecuado, gestionando correctamente la posibilidad de ausencia de resultados.

2.1.2. Solución ingenua paralela: `reconstruirCadenaIngenuoPar`

Con el objetivo de acelerar el proceso exhaustivo de reconstrucción de la cadena oculta S, se ha diseñado una versión paralela de la solución ingenua, denominada `reconstruirCadenaIngenuoPar`. Esta variante explota la independencia de los subconjuntos A_i en la segmentación del espacio Σ^n, permitiendo que la búsqueda de la solución se realice en paralelo sobre cada uno de los ocho segmentos definidos previamente.

El espacio de búsqueda Σ^n sigue dividiéndose en ocho partes disjuntas, pero, a diferencia de la versión secuencial, cada segmento es explorado de forma simultánea. Esto aprovecha los recursos multinúcleo disponibles en los sistemas modernos, reduciendo el tiempo total de exploración casi proporcionalmente al número de núcleos físicos del procesador (hasta cierto límite impuesto por el tamaño de los segmentos y la sobrecarga de gestión de tareas).

La implementación utiliza la abstracción de paralelismo provista en Scala, haciendo uso de las funciones parallel y task definidas en la infraestructura común del proyecto. Gracias a esto, la estructura del código se mantiene declarativa y funcional, pero permite que la evaluación de los segmentos se realice concurrentemente en distintos hilos de ejecución.

La función tiene la siguiente forma:

```scala
def reconstruirCadenaIngenuoPar(umbral: Int)(n: Int, o: Oraculo): Seq[Char] = {
  def generarCombinaciones(n: Int): LazyList[Seq[Char]] = {
    if (n == 0) LazyList(Seq.empty)
    else for {
      suf <- generarCombinaciones(n - 1)
      c <- LazyList.from(alfabeto)
    } yield c +: suf
  }

  val combinaciones = generarCombinaciones(n)
  val total = math.pow(alfabeto.length, n).toInt
  val octavo = total / 8

  val ((res1, res2, res3, res4), (res5, res6, res7, res8)) = parallel(
    parallel(
      combinaciones.slice(0 * octavo, 1 * octavo).find(o),
      combinaciones.slice(1 * octavo, 2 * octavo).find(o),
      combinaciones.slice(2 * octavo, 3 * octavo).find(o),
      combinaciones.slice(3 * octavo, 4 * octavo).find(o)
    ),
    parallel(
      combinaciones.slice(4 * octavo, 5 * octavo).find(o),
      combinaciones.slice(5 * octavo, 6 * octavo).find(o),
      combinaciones.slice(6 * octavo, 7 * octavo).find(o),
      combinaciones.slice(7 * octavo, total).find(o)
    )
  )

  res1.orElse(res2).orElse(res3).orElse(res4)
    .orElse(res5).orElse(res6).orElse(res7).orElse(res8)
    .getOrElse(Seq.empty)
}
```

Listing 3: Implementación paralela de `reconstruirCadenaIngenuoPar`

Principales diferencias respecto a la versión secuencial

- **Evaluación concurrente:** Los ocho segmentos en los que se divide el espacio de búsqueda se exploran en paralelo utilizando la función `parallel`. Esto permite aprovechar múltiples núcleos del procesador, reduciendo significativamente el tiempo total de búsqueda en comparación con la versión secuencial.
- **Infraestructura de paralelismo:** La versión paralela emplea primitivas de tareas (`task`, `parallel`) proporcionadas por el módulo `common`, que internamente gestionan un *pool* de hilos de tipo `Fork/Join`. Esta infraestructura resulta especialmente eficiente para tareas computacionalmente intensivas y para la división del trabajo en partes independientes.
- **Inmutabilidad y transparencia:** Al igual que la versión secuencial, todas las estructuras de datos y funciones mantienen el paradigma funcional e inmutable. Esto garantiza la ausencia de efectos colaterales y asegura la seguridad en contextos concurrentes, facilitando el razonamiento y la depuración del código.

La paralelización es natural y eficiente, dado que cada segmento es independiente y la consulta al oráculo no produce efectos secundarios. La función sigue utilizando listas perezosas (`LazyList`), por lo que no hay penalización significativa en el uso de memoria, y el coste principal proviene de la evaluación del oráculo en cada rama.

2.1.3. Solución mejorada: reconstruirCadenaMejorada

En busca de una mayor eficiencia en la reconstrucción de la cadena oculta S, se propone una mejora respecto a la solución exhaustiva ingenua. El objetivo principal es reducir el espacio de búsqueda evitando la generación y evaluación de combinaciones que pueden descartarse de antemano, incrementando así la eficiencia de la exploración.

La función `reconstruirCadenaMejorado` adopta una estrategia de filtrado progresivo: en lugar de generar todas las secuencias posibles de longitud n para luego evaluarlas, construye recursivamente las cadenas candidatas, pero en cada paso sólo extiende aquellas que ya han sido verificadas positivamente por el oráculo en sus prefijos. Formalmente, en cada nivel k de la recursión, la generación de combinaciones se restringe al conjunto:

$$\mathcal{C}_k = \left\{ (c_1, \ldots, c_k) \in \Sigma^k \mid \Psi_S\big((c_1, \ldots, c_k)\big) = \text{True} \right\}$$

Donde, en cada paso de la construcción recursiva, sólo se consideran las cadenas que son subcadenas válidas de la cadena objetivo S según el oráculo.

Esto reduce significativamente el número de combinaciones a evaluar en comparación con la solución ingenua, pues se descartan tempranamente aquellas extensiones que no podrían formar parte de la cadena buscada.

A continuación se muestra la implementación en Scala de la función `reconstruirCadenaMejorado`:

```
def reconstruirCadenaMejorado(n: Int, o: Oraculo): Seq[Char] = {
    def generarCombinaciones(n: Int): Seq[Seq[Char]] = {
        if (n == 0) LazyList(Seq.empty)
        else for {
            suf <- generarCombinaciones(n - 1).filter(o)
            c <- alfabeto
        } yield c +: suf
    }
    generarCombinaciones(n).headOption.getOrElse(Seq.empty)
}
```

Listing 4: Implementación de `reconstruirCadenaMejorado`

Como puede observarse, la función `reconstruirCadenaMejorado` introduce una estrategia de filtrado progresivo: en lugar de construir y evaluar todas las combinaciones posibles, descarta tempranamente aquellas secuencias que ya no cumplen el criterio del oráculo en etapas intermedias, optimizando así la búsqueda.

Uso de función auxiliar: `generarCombinaciones` se declara de manera interna, y su uso consiste en generar todas las posibles cadenas de longitud n aplicando un filtrado en cada paso. Se invoca con el parámetro n para producir únicamente aquellas secuencias que, en cada etapa de construcción, cumplen el criterio del oráculo. El resultado es una colección de secuencias candidatas ya verificadas, de la cual se selecciona la primera que satisface el problema utilizando `headOption.getOrElse(Seq.empty)`.

```
def generarCombinaciones(n: Int): Seq[Seq[Char]] = {
    if (n == 0) LazyList(Seq.empty)
    else for {
    suf <- generarCombinaciones(n - 1).filter(o)
    c <- alfabeto
    } yield c +: suf
}
```

Listing 5: Función auxiliar para el caso mejorado

En cada llamada recursiva, la utilización de `filter(o)` permite restringir el crecimiento del árbol de búsqueda únicamente a las combinaciones que el oráculo considera posibles, lo que evita el procesamiento de caminos que se descartan tempranamente. Así, el algoritmo explora de manera selectiva solo aquellas extensiones que pueden derivar en la solución correcta.

Esta definición se mantiene en el paradigma funcional: emplea recursión, comprensiones `for` y colecciones inmutables, facilitando un código claro, expresivo y seguro ante errores. El resultado es un recorrido más eficiente y dirigido del espacio de combinaciones, adaptándose dinámicamente a la información parcial proporcionada por el oráculo.

Análisis de la implementación en Scala

La función `reconstruirCadenaMejorado` implementa una estrategia de poda progresiva y eficiente del espacio de búsqueda, permitiendo descartar tempranamente aquellas combinaciones de símbolos que no pueden formar parte de la cadena objetivo, según las respuestas parciales del oráculo. El flujo funcional y la estructura del código pueden desglosarse en las siguientes etapas principales:

- **Inicialización:** Se inicia el proceso con una colección que contiene únicamente la cadena vacía. Este será el punto de partida sobre el cual se irán extendiendo las combinaciones candidatas de forma sucesiva.
- **Función auxiliar `generarCombinaciones`:** Esta función recursiva es responsable de construir todas las posibles combinaciones de longitud k a partir de los candidatos de longitud $k-1$. Para cada secuencia parcial válida en la etapa anterior, se generan nuevas combinaciones agregando a la derecha cada símbolo posible del alfabeto (`for (c <- alfabeto; suf <- sc) yield suf :+ c`). Tras la extensión, la función filtra inmediatamente las nuevas combinaciones, conservando únicamente aquellas subcadenas intermedias que son reconocidas como válidas por el oráculo (`filter(o)`). De este modo, se evita propagar ramas que no pueden conducir a la solución, reduciendo significativamente el espacio de búsqueda respecto al método exhaustivo.
- **Iteración y avance recursivo:** El proceso se repite recursivamente, incrementando la longitud de las combinaciones en cada paso, hasta alcanzar la longitud n deseada. En cada etapa, solo sobreviven las combinaciones que pasan todos los filtros, lo que produce un crecimiento mucho más controlado del número de candidatos.
- **Selección de solución:** Una vez generadas todas las combinaciones válidas de longitud n, la función selecciona la primera secuencia que cumple el criterio del oráculo, mediante métodos como `headOption` o `find`, garantizando la eficiencia y seguridad en la extracción del resultado.
- **Eficiencia y claridad funcional:** Todo el proceso se desarrolla sin variables mutables ni bucles imperativos. Se emplean colecciones y funciones puras, manteniendo la inmutabilidad de los datos y la transparencia referencial. La estrategia de filtrado en cada etapa permite reducir el espacio de búsqueda y el número de consultas necesarias al oráculo, haciendo que la solución sea mucho más eficiente y elegante que la versión ingenua.

La estructura recursiva, el filtrado progresivo y la ausencia de efectos secundarios aseguran que la solución sea tanto eficiente como mantenible, alineándose con los principios fundamentales del paradigma funcional en Scala.

Técnicas de programación funcional utilizadas:

- **Recursión de árbol.** La función auxiliar `generarCombinaciones` implementa la generación de secuencias de longitud n mediante un esquema recursivo puro, donde en cada llamada se construyen nuevas combinaciones a partir de las anteriores, evitando cualquier tipo de bucle imperativo.
- **Expresiones `for`.** Se utiliza la comprensión `for` para combinar de forma declarativa todas las subcadenas válidas generadas en el paso previo con cada símbolo del alfabeto, facilitando la construcción del producto cartesiano entre secuencias y símbolos.

- **Funciones de alto orden.** La función `filter` se aplica en cada nivel recursivo para seleccionar únicamente las subcadenas aceptadas por el oráculo, podando ramas inválidas de manera eficiente. Al final del proceso, `headOption` y `getOrElse` se emplean para recuperar una solución válida o, en su defecto, una secuencia vacía.
- **Evaluación segura con `Option`.** El uso de `headOption` permite manejar de forma segura el caso en que no se encuentre ninguna secuencia válida, evitando errores en tiempo de ejecución y garantizando que siempre se devuelve un resultado definido gracias a `getOrElse`.
- **Inmutabilidad de datos.** Todas las secuencias (`Seq[Char]`) manipuladas a lo largo de la construcción y filtrado son inmutables. Cada paso de la recursión produce nuevas colecciones sin modificar las existentes, lo que asegura transparencia referencial y ausencia de efectos laterales.
- **Composición funcional.** El flujo principal de la función es una composición de funciones puras (`generarCombinaciones`, `filter`, `headOption`), prescindiendo de instrucciones imperativas, variables mutables o asignaciones de estado.

2.1.4. Solución mejorada paralela: `reconstruirCadenaMejoradoPar`

Para incrementar aún más la eficiencia de la reconstrucción y aprovechar los recursos computacionales actuales, se implementa una versión paralela de la solución mejorada. La función `reconstruirCadenaMejoradoPar` aplica la misma estrategia de filtrado progresivo que su contraparte secuencial, pero introduce paralelismo en la generación de combinaciones candidatas, permitiendo así una mayor aceleración, especialmente en sistemas con múltiples núcleos de procesamiento.

El principio fundamental de esta versión es explotar el hecho de que la construcción de nuevas combinaciones en cada etapa de la recursión es independiente para cada subcadena parcial válida, lo que habilita la aplicación segura y eficiente de paralelismo sobre los datos.

A continuación se muestra la implementación en Scala de la función `reconstruirCadenaMejoradoPar`:

```
def reconstruirCadenaMejoradoPar(umbral: Int)(n: Int, o: Oraculo): Seq[Char] = {
def generarCombinaciones(n: Int): Seq[Seq[Char]] = {
    if (n == 0) LazyList(Seq.empty)
    else for {
      suf <- generarCombinaciones(n - 1).filter(o)
      c <- alfabeto.par
    } yield c +: suf
}
generarCombinaciones(n).headOption.getOrElse(Seq.empty)
}
```

Listing 6: Implementación de `reconstruirCadenaMejoradoPar`

Como puede observarse, la única modificación necesaria respecto a la versión secuencial consiste en el uso de `alfabeto.par` en la comprensión `for`, lo que convierte la colección en una colección paralela y distribuye automáticamente la generación de nuevas combinaciones entre los distintos núcleos del procesador.

Uso de función auxiliar: `generarCombinaciones` La función auxiliar se define de forma análoga al caso secuencial, pero al emplear una colección paralela para el alfabeto, la extensión de cada subcadena parcial se realiza en paralelo. Esto resulta especialmente útil en instancias donde el número de subcadenas candidatas es elevado, permitiendo reducir de manera significativa el tiempo total de computación.

```
def generarCombinaciones(n: Int): Seq[Seq[Char]] = {
    if (n == 0) LazyList(Seq.empty)
    else for {
        suf <- generarCombinaciones(n - 1).filter(o)
        c <- alfabeto.par
    } yield c +: suf
}
```

Listing 7: Función auxiliar paralela para el caso mejorado

Principales diferencias respecto a la versión secuencial

- **Paralelización de la generación de combinaciones:** Al utilizar `alfabeto.par`, la generación de nuevas combinaciones a partir de cada subcadena parcial válida se realiza en paralelo, distribuyendo el trabajo entre los diferentes núcleos del sistema y acelerando la exploración del espacio de búsqueda.
- **Eficiencia mejorada:** El paralelismo sobre los datos permite reducir el tiempo total requerido para encontrar la solución, especialmente en casos donde el oráculo es costoso o el espacio de combinaciones sigue siendo grande tras la poda.
- **Mínimo cambio estructural:** La transición al paradigma paralelo se realiza mediante una simple conversión de la colección, manteniendo intacto el estilo funcional y la claridad del código. El resto de la estructura, incluidas las propiedades de inmutabilidad y transparencia, se preservan.

2.1.5. Solución turbo: reconstruirCadenaTurbo

Con el fin de optimizar aún más la reconstrucción de la cadena oculta S, se introduce un enfoque conocido como solución turbo, que reduce drásticamente el número de combinaciones a explorar en comparación con el método exhaustivo tradicional. La idea central es aprovechar la estructura de superposición de subcadenas en el espacio Σ^n y construir secuencias candidatas duplicando su longitud en cada etapa, en vez de incrementarla paso a paso.

En este enfoque, la reconstrucción de S se modela mediante la construcción iterativa de subconjuntos $\mathcal{C}_k \subseteq \Sigma^k$ que contienen todas las cadenas de longitud k que podrían ser prefijos válidos de la solución. En cada iteración, se genera el conjunto de combinaciones candidatas concatenando pares de cadenas válidas de menor tamaño, y se filtran aquellas que no cumplen con las condiciones impuestas por el oráculo Ψ_S.

Formalmente, el procedimiento se basa en la generación y refinamiento sucesivo del conjunto:

$$\mathcal{C}_k = \left\{ w \| v \mid w, v \in \mathcal{C}_{k/2},\ \Psi_S(w\|v) = \text{True} \right\}$$

donde $\mathcal{C}_{k/2}$ es el conjunto de cadenas válidas de longitud $k/2$, y $w\|v$ denota la concatenación de w y v.

El algoritmo comienza con el conjunto de cadenas más cortas —es decir, las letras individuales del alfabeto que ya cumplen el criterio del oráculo— y, en cada iteración, duplica la longitud de las combinaciones al concatenar pares de secuencias válidas de la etapa anterior. Este procedimiento se repite de manera recursiva hasta alcanzar la longitud n deseada.

De esta forma, el tamaño del espacio de búsqueda se mantiene bajo control en cada etapa y se evita la generación de combinaciones innecesarias, ya que en todo momento sólo se consideran aquellas secuencias que continúan siendo válidas según las restricciones del oráculo.

Este enfoque permite reducir de manera significativa la cantidad total de evaluaciones, logrando una búsqueda mucho más eficiente en comparación con el método exhaustivo tradicional.

```scala
def reconstruirCadenaTurbo(n: Int, o: Oraculo): Seq[Char] = {
  // Genera el conjunto Sigma^k a partir de un conjunto inicial
  def generarSigmaK(sc: Seq[Seq[Char]]): Seq[Seq[Char]] = {
    for {
      w <- sc
      v <- sc
    } yield w ++ v
  }
  // Funcion recursiva que duplica la longitud k en cada paso
  def aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char] = {
    val sigmaK = generarSigmaK(scInicial).filter(o)
    val cadenaW = sigmaK.head
    if (k == n) cadenaW else aux(sigmaK, k * 2)
  }
  // Punto de partida: las letras individuales que cumplen el oraculo
  val initialSc = alfabeto.map(c => Seq(c)).filter(o)
  aux(initialSc, 2)
}
```

Listing 8: Implementación de `reconstruirCadenaTurbo`

Uso de función auxiliar: `generarSigmaK` Para construir las combinaciones candidatas en cada etapa del algoritmo turbo, se define una función auxiliar denominada `generarSigmaK`. Esta función toma como entrada el conjunto de secuencias válidas generadas en la iteración anterior—es decir, las cadenas que hasta ese momento cumplen con el criterio del oráculo para una longitud $k/2$—y produce todas las posibles secuencias de longitud k mediante la concatenación de pares de dichas cadenas. De esta manera, en cada paso del algoritmo, el conjunto de entrada representa las soluciones válidas acumuladas hasta la etapa previa, y la función genera a partir de ellas el nuevo conjunto de candidatos para la siguiente longitud.

```scala
def generarSigmaK(sc: Seq[Seq[Char]]): Seq[Seq[Char]] = {
    for {
        w <- sc
        v <- sc
    } yield w ++ v
}
```

Listing 9: Función auxiliar generarSigmaK

- **Recursión de cola:** La función auxiliar `aux` es responsable de la lógica principal del algoritmo. Mediante recursión, toma un conjunto de cadenas válidas (`scInicial`) y, en cada llamada, construye todas las posibles combinaciones de longitud k duplicando la longitud anterior, hasta llegar a la longitud n. En cada etapa, la recursión controla la progresión del algoritmo de forma pura, sin bucles ni cambios de estado.
- **Expresiones `for`:** En `generarSigmaK`, una expresión `for` doble implementa el producto cartesiano entre los elementos de la colección `sc`, generando todas las posibles concatenaciones de pares de cadenas válidas. Esto permite, de forma declarativa y concisa, crear el conjunto de candidatos para la siguiente etapa.
- **Funciones de alto orden:** En cada nivel de la recursión, tras generar las nuevas combinaciones, se aplica `filter(o)` para descartar aquellas cadenas que no cumplen el criterio impuesto por el oráculo. Finalmente, el método `head` se utiliza para seleccionar una secuencia válida del conjunto resultante, suponiendo que siempre existirá al menos una solución (por la definición del problema y el oráculo).
- **Inmutabilidad de datos:** Todas las colecciones (`Seq[Seq[Char]]`, `Seq[Char]`) utilizadas en el proceso se crean y manipulan de forma inmutable. En cada etapa de la recursión se producen

nuevas secuencias sin alterar las existentes, lo que asegura transparencia referencial y elimina efectos colaterales.

- **Composición funcional:** El flujo completo del algoritmo se estructura mediante la composición directa de funciones puras: `aux` invoca a `generarSigmaK` y a `filter`, componiendo los resultados para la siguiente llamada recursiva. Así, la lógica de construcción, filtrado y avance está integrada sin necesidad de instrucciones imperativas ni variables intermedias mutables.
- **Encapsulamiento mediante funciones auxiliares anidadas:** Tanto `generarSigmaK` como `aux` se definen como funciones locales dentro de `reconstruirCadenaTurbo`, lo que mejora la organización, restringe su visibilidad al contexto donde son necesarias y refuerza la modularidad del diseño funcional.

Análisis de la implementación en Scala

La implementación de la función `reconstruirCadenaTurbo` se organiza en torno a una estrategia de construcción progresiva y filtrado de secuencias. En la primera etapa, se parte de las cadenas más simples posibles: las letras individuales del alfabeto que cumplen el criterio del oráculo. Estas secuencias iniciales representan el conjunto de candidatos válidos de longitud 1.

A continuación, en cada iteración, se utiliza la función auxiliar `generarSigmaK` para construir todas las posibles concatenaciones de pares de secuencias válidas de la etapa previa, duplicando así la longitud de las cadenas en cada paso. Tras cada expansión, se aplica un filtro con el oráculo para conservar únicamente aquellas combinaciones que sigan siendo potencialmente correctas.

Este proceso se repite de manera recursiva dentro de la función `aux`, incrementando la longitud de las cadenas candidatas en potencias de dos, hasta alcanzar la longitud objetivo n. Finalmente, se selecciona la primera secuencia válida encontrada mediante `head`, que, bajo las condiciones del problema, debe corresponderse con la solución buscada.

El uso de funciones auxiliares anidadas, expresiones `for`, filtrado funcional y colecciones inmutables refleja plenamente los principios del paradigma funcional: la lógica es declarativa, no se utilizan variables mutables ni estructuras de control imperativas, y el flujo de datos es transparente y seguro.

Esta solución turbo no sólo reduce significativamente la cantidad de combinaciones evaluadas, sino que establece una base clara y funcional sobre la que se pueden implementar versiones aún más eficientes o con técnicas avanzadas de filtrado y paralelización.

Técnicas de programación funcional utilizadas:

- **Recursión de cola:** La función auxiliar `aux` emplea recursión para duplicar el tamaño de las cadenas en cada paso, controlando la generación y filtrado de secuencias hasta alcanzar la longitud objetivo.
- **Expresiones `for`:** La comprensión `for` se usa para construir todas las combinaciones posibles concatenando pares de secuencias válidas, representando el producto cartesiano de los conjuntos actuales.
- **Funciones de alto orden:** La función `filter` se emplea para seleccionar únicamente aquellas cadenas que cumplen el criterio del oráculo en cada iteración. El método `head` se utiliza para obtener una solución válida tras el filtrado.
- **Inmutabilidad de datos:** Todas las secuencias generadas y manipuladas son inmutables, garantizando transparencia referencial y ausencia de efectos secundarios.
- **Composición funcional:** El flujo de la función es una composición directa de funciones puras (`generarSigmaK`, `filter`, `aux`), evitando el uso de variables mutables y estructuras imperativas.

- **Encapsulamiento mediante funciones auxiliares anidadas:** Las funciones auxiliares `generarSigmaK` y `aux` están definidas dentro del alcance de la función principal, facilitando la modularidad y el control de estado local.

2.1.6. Solución turbo paralela: `reconstruirCadenaTurboPar`

Para aprovechar la concurrencia y acelerar aún más el proceso de reconstrucción, se implementa una versión paralela del algoritmo turbo, denominada `reconstruirCadenaTurboPar`. Esta versión traslada el paradigma funcional y las optimizaciones de la solución turbo al contexto de cómputo paralelo, distribuyendo la carga de trabajo entre múltiples núcleos de procesamiento.

La clave de la paralelización en este enfoque radica en que la generación de nuevas combinaciones candidatas en cada etapa (producto cartesiano de pares de secuencias válidas) es completamente independiente para cada par de cadenas. Esto permite utilizar colecciones paralelas y expresiones `for` sobre datos paralelos para repartir eficientemente las operaciones de concatenación y filtrado.

A continuación se muestra la implementación en Scala de la función `reconstruirCadenaTurboPar`:

```
def reconstruirCadenaTurboPar(umbral: Int)(n: Int, o: Oraculo): Seq[Char] = {
// Genera el conjunto Sigma^k en paralelo a partir de un conjunto inicial
def generarSigmaK(sc: Seq[Seq[Char]]): Seq[Seq[Char]] = {
    (for {
        w <- sc.par
        v <- sc
    } yield w ++ v).seq
}
// Funcion recursiva paralela que duplica la longitud k en cada paso
def aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char] = {
    val sigmaK = generarSigmaK(scInicial).par.filter(o).seq
    val cadenaW = sigmaK.head
    if (k == n) cadenaW else aux(sigmaK, k * 2)
    }
    // Punto de partida: letras individuales aceptadas por el oraculo
    val initialSc = alfabeto.par.map(c => Seq(c)).filter(o).seq
    aux(initialSc, 2)
}
```

Listing 10: Implementación de `reconstruirCadenaTurboPar`

La diferencia fundamental respecto a la versión secuencial es el uso de colecciones paralelas (`.par`), tanto para el alfabeto inicial como para la generación y filtrado de combinaciones en cada etapa. Esto permite explotar la concurrencia de datos y reducir significativamente el tiempo de ejecución en sistemas multinúcleo.

Uso de función auxiliar: `generarSigmaK` En la versión paralela, la función auxiliar `generarSigmaK` utiliza la colección paralela (`sc.par`) para recorrer los candidatos válidos y generar, en paralelo, todas las concatenaciones posibles con los elementos del conjunto anterior. Esto distribuye la operación de construcción de pares de secuencias sobre los núcleos disponibles, acelerando la generación del conjunto de candidatos en cada paso.

```
def generarSigmaK(sc: Seq[Seq[Char]]): Seq[Seq[Char]] = {
    (for {
        w <- sc.par
        v <- sc
    } yield w ++ v).seq
}
```

Listing 11: Función auxiliar paralela para Turbo

Principales diferencias respecto a la versión secuencial

- **Paralelización del producto cartesiano y filtrado:** La generación de nuevas secuencias por medio del producto cartesiano entre los candidatos (`w <- sc.par; v <- sc`) y el filtrado por el oráculo (`.par.filter(o)`) se ejecutan en paralelo, repartiendo el trabajo entre los núcleos del sistema.
- **Inicialización y avance paralelo:** Tanto la construcción inicial del conjunto de candidatos (`alfabeto.par`) como las sucesivas etapas de duplicación de longitud (`aux`) aprovechan la paralelización de datos, acelerando la exploración del espacio de búsqueda.
- **Conservación de la inmutabilidad y transparencia:** El paradigma funcional se mantiene intacto: todas las colecciones y funciones son inmutables y puras, asegurando la ausencia de efectos colaterales y la seguridad en contextos concurrentes.

2.1.7. Solución turbo mejorada: reconstruirCadenaTurboMejorada

se propone la solución turbo mejorada, que extiende la estrategia de duplicación de longitud del método turbo clásico incorporando controles adicionales de validez en cada paso.

La idea central consiste no solo en aprovechar la estructura de superposición de subcadenas, sino también en asegurar que toda subcadena relevante dentro de cada combinación generada sea válida respecto al conjunto de candidatos de la etapa anterior, según la información proporcionada por el oráculo Ψ_S.

En este enfoque, la reconstrucción de S se modela mediante la construcción iterativa de subconjuntos $\mathcal{C}_k \subseteq \Sigma^k$ que contienen todas las cadenas de longitud k que pueden ser candidatas válidas. En cada iteración, el conjunto de combinaciones candidatas se genera concatenando pares de cadenas válidas de longitud $k/2$.

Sin embargo, a diferencia de la versión turbo básica, aquí se exige además que todas las subcadenas de longitud $k/2$ contenidas en la nueva combinación también pertenezcan al conjunto de candidatos previos. Solo las cadenas que cumplen tanto la validez global como la validez de todas sus subcadenas intermedias son retenidas.

Formalmente, el procedimiento se basa en la generación y refinamiento sucesivo del conjunto:

$$\mathcal{C}_k = \left\{ w\|v \;\middle|\; w, v \in \mathcal{C}_{k/2},\ \forall\, 0 \le i < k,\ s_i = \text{subcadena de longitud } \frac{k}{2} \text{ de } (w\|v),\ s_i \in \mathcal{C}_{k/2},\ \Psi_S(w\|v) = \text{True} \right\}$$

Donde $\mathcal{C}_{k/2}$ es el conjunto de cadenas válidas de longitud $k/2$ y $w\|v$ representa la concatenación de w y v.

El algoritmo comienza con el conjunto de cadenas más cortas —es decir, las letras individuales del alfabeto validadas por el oráculo— y, en cada iteración, duplica la longitud de las combinaciones al concatenar pares de secuencias válidas de la etapa anterior, filtrando cuidadosamente aquellas cuya estructura interna no sea consistente con el conjunto de candidatos.Este proceso recursivo se repite hasta alcanzar la longitud n buscada.

De esta forma, el espacio de búsqueda queda restringido en cada etapa, ya que en todo momento sólo se consideran aquellas secuencias que no sólo cumplen la restricción global del oráculo, sino que también mantienen la validez de todas sus subestructuras relevantes.

Este enfoque permite reducir de manera significativa el número total de evaluaciones, logrando una búsqueda mucho más eficiente y precisa que la del método turbo tradicional y, por supuesto, que el método exhaustivo.

```scala
def reconstruirCadenaTurboMejorada(n: Int, o: Oraculo): Seq[Char] = {
  // Funcion para filtrar solo las combinaciones validas segun el oraculo y las
      subcadenas
  def filtrar(sc: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
    for {
      w <- sc
      v <- sc
      s = w ++ v
      if (1 until k).forall(i => sc.contains(s.slice(i, i + k)))
    } yield s
  }

  // Funcion recursiva principal, duplicando longitud en cada paso
  def aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char] = {
    val sigmaK = filtrar(scInicial, k / 2).filter(o)
    val cadenaW = sigmaK.head
    if (k == n) cadenaW else aux(sigmaK, k * 2)
  }

  // Inicializacion: letras individuales filtradas por el oraculo
  val initialSc = alfabeto.map(c => Seq(c)).filter(o)
  aux(initialSc, 2)
}
```

Listing 12: Implementación de `reconstruirCadenaTurboMejorada`

Uso de función auxiliar: `filtrar` Para construir las combinaciones candidatas en cada etapa del algoritmo turbo mejorado, se define una función auxiliar denominada `filtrar`. Esta función recibe como entrada el conjunto de secuencias válidas generadas en la iteración anterior—es decir, aquellas cadenas de longitud $k/2$ que cumplen con el criterio del oráculo—y produce todas las posibles secuencias de longitud k mediante la concatenación de pares de dichas cadenas. A diferencia de la versión turbo básica, `filtrar` incorpora una validación adicional: asegura que todas las subcadenas de longitud $k/2$ de la nueva secuencia generada también pertenezcan al conjunto de candidatos previos. Este control extra garantiza que cada nueva combinación generada mantiene la consistencia con el conjunto de soluciones válidas en todas sus subestructuras, evitando así la propagación de combinaciones no válidas en las siguientes etapas.

```scala
def filtrar(sc: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
    for {
        w <- sc
        v <- sc
        s = w ++ v
        if (1 until k).forall(i => sc.contains(s.slice(i, i + k)))
    } yield s
}
```

Listing 13: Función auxiliar filtrar para Turbo Mejorada

La función `filtrar` itera sobre todos los pares posibles de secuencias del conjunto actual `sc`, genera sus concatenaciones y verifica, utilizando la función de orden superior `forall`, que cada subcadena de longitud k dentro de la nueva secuencia también está presente en `sc`. Solo si todas las subcadenas son válidas, la secuencia concatenada se incluye en el nuevo conjunto de candidatos para la siguiente etapa del algoritmo. Esta validación incremental refuerza la solidez del método y reduce aún más el espacio de búsqueda, asegurando que únicamente se exploren caminos compatibles con la estructura de soluciones del problema.

Uso de función auxiliar: `aux`

Para controlar el avance recursivo y la construcción de soluciones en el algoritmo turbo mejorado, se define una función auxiliar denominada `aux`. Esta función toma como entrada el conjunto actual de candidatos (`scInicial`) y la longitud actual k, y se encarga de generar las combinaciones de la siguiente etapa aplicando la función `filtrar` y validando las nuevas secuencias con el oráculo. A continuación, si la longitud objetivo n no ha sido alcanzada, la función invoca recursivamente a sí misma con el nuevo conjunto filtrado y la longitud duplicada, hasta completar el proceso. De este modo, `aux` estructura el flujo principal del algoritmo, asegurando que en cada paso sólo se mantengan y evalúen los candidatos válidos.

```
def aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char] = {
    val sigmaK = filtrar(scInicial, k / 2).filter(o)
    val cadenaW = sigmaK.head
    if (k == n) cadenaW else aux(sigmaK, k * 2)
}
```

Listing 14: Función auxiliar aux para Turbo Mejorada

La función `aux` aplica primero la función `filtrar` para generar y depurar el nuevo conjunto de secuencias de longitud k, y luego filtra nuevamente las secuencias con el oráculo para garantizar su validez global. Utiliza el método `head` para obtener una secuencia candidata, bajo el supuesto de que siempre existirá al menos una solución válida. En caso contrario, si la longitud objetivo no ha sido alcanzada, realiza una llamada recursiva sobre el nuevo conjunto de candidatos y con la longitud duplicada. Esta organización recursiva permite gestionar de manera clara y eficiente el proceso iterativo de construcción de la solución, integrando tanto la generación como la validación en cada etapa.

Análisis de la implementación en Scala

La implementación de la función `reconstruirCadenaTurboMejorada` se fundamenta en una estrategia de construcción iterativa y validación reforzada de secuencias. En la etapa inicial, se parte de las cadenas más simples: las letras individuales del alfabeto que son aceptadas por el oráculo. Estas secuencias conforman el conjunto inicial de candidatos válidos de longitud 1.

En cada iteración posterior, el proceso de generación y filtrado se realiza mediante dos funciones auxiliares anidadas: `filtrar` y `aux`. La función `filtrar` toma el conjunto de secuencias válidas de longitud $k/2$ y produce todas las combinaciones posibles de longitud k concatenando pares de dichas cadenas. Sin embargo, a diferencia del enfoque turbo estándar, aquí se incorpora un control adicional: se verifica que ciertas subcadenas de longitud $k/2$ de la nueva combinación también pertenezcan al conjunto de candidatos previos.

Esto se logra utilizando la función de orden superior `forall`, que comprueba, para cada concatenación generada, que todas sus subcadenas relevantes figuren entre los candidatos válidos de la etapa anterior, asegurando así la coherencia y la consistencia estructural de las soluciones parciales.

La función `aux`, por su parte, controla el avance recursivo del algoritmo: en cada llamada, toma el conjunto actual de candidatos y la longitud correspondiente, aplica `filtrar` para generar el siguiente conjunto de secuencias y luego filtra por el oráculo para validar su corrección global. Si aún no se ha alcanzado la longitud objetivo n, realiza una llamada recursiva sobre el conjunto resultante, duplicando la longitud en cada paso. Cuando se alcanza la longitud deseada, se selecciona la primera secuencia válida mediante `head`.

Esta organización funcional, basada en el uso de funciones auxiliares anidadas, expresiones `for`, funciones de alto orden como `filter` y `forall`, y colecciones inmutables, refleja una aplicación rigurosa del paradigma funcional. No se utilizan variables mutables ni estructuras de control imperativas, y la lógica declarativa facilita la composición y el razonamiento sobre el código.

Gracias a la validación incremental implementada por `filtrar`, la versión turbo mejorada logra restringir el espacio de búsqueda mucho más agresivamente que la versión turbo estándar, evaluando

únicamente aquellas combinaciones que mantienen la consistencia con todas las subestructuras relevantes.

Técnicas de programación funcional utilizadas:

- **Recursión de cola.** La función auxiliar `aux` emplea recursión para duplicar la longitud de las cadenas en cada paso, generando y refinando el conjunto de secuencias candidatas hasta alcanzar la solución final.
- **Expresiones `for`.** La comprensión `for` facilita la construcción del producto cartesiano entre las secuencias candidatas (`sc`), generando todas las posibles concatenaciones y permitiendo aplicar condiciones adicionales de validez sobre las subcadenas.
- **Funciones de alto orden.** Se utiliza `filter` para conservar únicamente las secuencias que cumplen con el criterio del oráculo y con la propiedad de aparición de subcadenas. También se usa `forall` para verificar que todas las subcadenas intermedias se encuentren presentes en el conjunto actual.
- **Inmutabilidad de datos.** Todas las colecciones (`Seq[Char]`, `Seq[Seq[Char]]`) manipuladas durante el proceso son inmutables, evitando efectos colaterales y facilitando la transparencia referencial.
- **Composición funcional.** La función principal se construye mediante la composición de funciones puras (`filtrar`, `aux`, `filter`, `forall`), sin uso de estructuras de control imperativas ni variables mutables.
- **Encapsulamiento mediante funciones auxiliares anidadas.** Las funciones auxiliares `filtrar` y `aux` están definidas dentro de la función principal, favoreciendo la modularidad y el control del estado interno.

2.1.8. Solución turbo mejorada paralela: `reconstruirCadenaTurboMejoradaPar`

Para maximizar la eficiencia y aprovechar plenamente los recursos de cómputo disponibles, se propone la versión paralela del método turbo mejorado, denominada `reconstruirCadenaTurboMejoradaPar`. Esta implementación traslada las ventajas del filtrado estructural del algoritmo turbo mejorado al contexto de cómputo concurrente, permitiendo una reducción sustancial en los tiempos de ejecución sobre instancias de mayor tamaño.

La clave de esta paralelización reside en la independencia de los procesos de generación y validación de cada combinación candidata: tanto la construcción del producto cartesiano como la comprobación de las subcadenas pueden ejecutarse en paralelo, ya que no existen dependencias entre los elementos individuales de cada etapa.

```
def reconstruirCadenaTurboMejoradaPar(umbral: Int)(n: Int, o: Oraculo): Seq[Char
    ] = {
    def filtrar(sc: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
        (for {
            w <- sc.par
            v <- sc
            s = w ++ v
            if (1 until k).forall(i => sc.contains(s.slice(i, i + k)))
        } yield s).seq
    }

    def aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char] = {
        val sigmaK = filtrar(scInicial, k / 2).par.filter(o).seq
        val cadenaW = sigmaK.head
        if (k == n) cadenaW else aux(sigmaK, k * 2)
        }

    val initialSc = alfabeto.map(c => Seq(c)).filter(o)
    aux(initialSc, 2)
}
```

Listing 15: Implementación de `reconstruirCadenaTurboMejoradaPar`

Uso de función auxiliar: `filtrar` paralela En la función auxiliar `filtrar`, el uso de `sc.par` permite que la generación y comprobación de las combinaciones $w \| v$ se realice en paralelo, distribuyendo el trabajo entre los diferentes núcleos del procesador. La validación de subcadenas para cada combinación también se ejecuta de forma concurrente, lo que acelera significativamente la etapa de filtrado.

```
def filtrar(sc: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
    (for {
        w <- sc.par
        v <- sc
        s = w ++ v
        if (1 until k).forall(i => sc.contains(s.slice(i, i + k)))
    } yield s).seq
}
```

Listing 16: Función auxiliar filtrar paralela para Turbo Mejorada

Principales diferencias respecto a la versión secuencial

- **Paralelización del producto cartesiano y comprobación de subcadenas:** El trabajo computacional más costoso (combinación de pares y validación de subcadenas) se distribuye automáticamente gracias al uso de colecciones paralelas (`.par`), permitiendo aprovechar todos los núcleos de la máquina.
- **Reducción efectiva del tiempo de ejecución:** El procesamiento paralelo de las combinaciones acelera tanto la generación como el filtrado, especialmente para valores altos de n, donde el espacio de búsqueda es más grande.
- **Conservación del paradigma funcional:** Todas las operaciones se realizan sobre estructuras inmutables y con funciones puras, lo que garantiza ausencia de efectos colaterales y facilita la depuración y el razonamiento sobre el código en contextos concurrentes.

2.1.9. Árbol de sufijos: pertenece

Para continuar con el desarrollo del paquete `ArbolSufijos`, se implementó una función que permite verificar si una secuencia de caracteres pertenece al conjunto de cadenas representadas por un Trie. El

propósito de esta función es determinar, de forma estructurada, si una cadena dada puede ser reconocida recorriendo los nodos del Trie, desde la raíz hasta una hoja o nodo marcado. Esta operación es fundamental para muchas otras construcciones del proyecto, como la inserción de cadenas o la validación previa antes de realizar consultas al oráculo.

La lógica del algoritmo sigue el principio clásico de búsqueda en un árbol de prefijos. Se parte de la raíz del Trie y se examina, en cada paso, si alguno de los nodos hijos contiene el símbolo actual que se desea reconocer. Si existe un nodo con ese símbolo, la función continúa de manera recursiva con el resto de la cadena, usando como nuevo Trie el subárbol que comienza en ese nodo. En caso de llegar a una hoja sin haber consumido toda la cadena, se concluye que la cadena no pertenece al Trie. Finalmente, si la cadena es consumida completamente, se verifica si el nodo donde terminó el recorrido está marcado como fin de palabra; si no lo está, se considera que la cadena no pertenece al conjunto representado.

```
def pertenece(s: Seq[Char], t: Trie): Boolean = {
  s match {
    case s1 +: ss => t match {
      case Nodo(_, _, hijos) =>
        hijos.exists(node => raiz(node) == s1 && pertenece(ss, node))
      case Hoja(_, _) => false
    }

    case Seq() => t match {
      case Nodo(_, marcada, _) => marcada
      case Hoja(_, marcada)    => marcada
    }
  }
}
```

Listing 17: Función pertenece

La función utiliza el patrón 'match' sobre la cadena de entrada `s` y sobre el Trie `t`. Si la cadena es no vacía, se descompone en su primer símbolo `s1` y el resto `ss`. En este caso, si el Trie actual es un nodo con hijos, se verifica si alguno de los hijos tiene como raíz el símbolo `s1`; si lo tiene, se continúa recursivamente con el resto de la cadena `ss` desde ese subárbol. Este proceso modela exactamente el recorrido secuencial por los nodos del Trie siguiendo el orden de los caracteres en la cadena de entrada.

Por otro lado, si la cadena es vacía, se ha terminado el recorrido, y lo que determina si la palabra pertenece al Trie o no, es si el nodo actual está marcado. Esto es coherente con la semántica del Trie, donde solo los nodos marcados (o las hojas) representan cadenas completas reconocidas por el conjunto. Una cadena que sea prefijo de otra pero cuyo nodo final no esté marcado, no debe ser reconocida como perteneciente al conjunto.

Técnicas de programación funcional utilizadas:

- Reconocimiento de patrones (`match`). Para descomponer tanto la cadena `s` como la estructura `Trie`, lo que permite distinguir los distintos casos de recorrido o terminación.
- Recursión lineal. La función `pertenece` emplea recursión lineal porque, en cada llamada, procesa un único elemento de la secuencia de entrada (`s1`) y delega la comprobación del resto (`ss`) a una única invocación recursiva. Este patrón continúa hasta alcanzar el caso base (`Seq()`), garantizando que solo se crea una rama de llamadas cuya profundidad es proporcional a la longitud de la secuencia.
- Funciones de alto orden (`exists`). Se emplea para verificar si existe un nodo hijo cuya raíz coincida con el carácter actual y que permita continuar el recorrido exitosamente.
- Inmutabilidad en las estructuras. Tanto la cadena como el Trie son estructuras inmutables, lo que garantiza que la función no altera ningún estado y se comporta de forma puramente funcional.
- Composición funcional. La lógica condicional está estructurada sin variables intermedias ni mutaciones, utilizando directamente combinaciones de expresiones y llamadas funcionales.

El resultado es una función compacta, declarativa y correcta, que refleja fielmente la semántica de un verificador de pertenencia sobre árboles de sufijos y preserva las propiedades del paradigma funcional.

2.1.10. Árbol de sufijos: adicionar

Para completar las operaciones fundamentales del paquete `ArbolSufijos`, se implementó una función que permite adicionar una nueva cadena al Trie. Esta operación es esencial para construir progresivamente el árbol de sufijos y enriquecer la estructura de manera eficiente, asegurando que cada nueva cadena se integre correctamente sin duplicar caminos ya existentes.

El objetivo de esta función es extender el Trie dado con una nueva cadena, pero respetando las ramas previamente construidas. Para esto, se parte de una verificación inicial: si la cadena ya pertenece al Trie, no se realiza ninguna operación y simplemente se retorna el árbol original. En caso contrario, se hace un análisis estructural del Trie y se avanza recursivamente agregando nodos o ramas donde sea necesario, reutilizando partes existentes del árbol.

```
def adicionar(s: Seq[Char], t: Trie): Trie = {

  def construirRama(s: Seq[Char]): Trie = {
    s match {
      case s1 +: Seq() => Hoja(s1, true)
      case s1 +: ss    => Nodo(s1, false, List(construirRama(ss)))
    }
  }

  if (pertenece(s, t)) { t } else {
    (s, t) match {

      case (s1 +: ss, Hoja(c, m)) =>
        Nodo(c, m, List(construirRama(s1 +: ss)))

      case (s1 +: ss, Nodo(c, m, hijos)) =>
        if (cabezas(t).contains(s1)) {
          val nuevosHijos = hijos.map(hijo =>
            if (raiz(hijo) == s1) adicionar(ss, hijo) else hijo)
          Nodo(c, m, nuevosHijos)
        } else {
          Nodo(c, m, hijos ++ List(construirRama(s1 +: ss)))
        }

      case (Seq(), Nodo(c, m, hijos)) =>
        Nodo(c, true, hijos)

      case (Seq(), t) =>
        t
    }
  }
}
```

Listing 18: Función adicionar

Uso de función auxiliar: `construirRama`

La función auxiliar `construirRama` se encarga de generar una nueva rama del Trie a partir de una cadena que no puede ser representada por ninguna rama existente del árbol. Esta función es recursiva y actúa sobre la estructura de la cadena: crea un nodo por cada símbolo, estableciendo como hijos los nodos correspondientes al resto de la cadena, hasta llegar a la hoja. El nodo hoja se marca con `true` para indicar que representa el final de una palabra válida, mientras que todos los nodos intermedios se marcan como `false`.

```scala
def construirRama(s: Seq[Char]): Trie = {
  s match {
    case s1 +: Seq() => Hoja(s1, true)
    case s1 +: ss    => Nodo(s1, false, List(construirRama(ss)))
  }
}
```

Listing 19: Función auxiliar construirRama

Esta función está anidada dentro de `adicionar`, ya que solo es relevante dentro de su contexto, lo que refleja un principio de encapsulación funcional. La recursión sobre la estructura de la cadena permite construir el Trie asociado sin necesidad de variables mutables ni bucles imperativos.

Técnicas de programación funcional utilizadas en `construirRama`:

- Reconocimiento de patrones (`match`) para descomponer la cadena y distinguir el caso base (un solo símbolo) del caso inductivo (más de un símbolo).
- La función `adicionar` emplea recursión lineal, ya que en cada llamada consume un único carácter de la secuencia de entrada (`s1`) y realiza una única invocación recursiva con el resto (`ss`). De este modo, la profundidad de las llamadas crece de forma proporcional a la longitud de la secuencia que se inserta en el trie. Asimismo, la función auxiliar `construirRama` recorre linealmente la secuencia para crear nuevos nodos, manteniendo el mismo patrón de recursión sobre la lista de caracteres.
- Encapsulación mediante función anidada.
- Construcción declarativa e inmutable del Trie.

Análisis de la implementación en Scala

La función `adicionar` puede analizarse en varias etapas funcionales. En primer lugar, se verifica si la cadena ya pertenece al Trie, utilizando la función previamente definida `pertenece`. Esta verificación evita modificar el árbol cuando no es necesario, lo que mejora la eficiencia.

Luego, se analiza la forma de la cadena y la estructura del Trie para decidir cómo integrarla:

- Si la estructura es una `Hoja` y la cadena tiene símbolos por procesar, se reemplaza la hoja por un `Nodo` y se le agrega la nueva rama construida.
- Si se está en un `Nodo` y el símbolo inicial de la cadena coincide con alguno de sus hijos, se aplica recursión sobre ese hijo y el resto de la cadena. El resto de los hijos se conserva igual.
- Si el símbolo inicial no está entre los hijos, se conserva el nodo actual y se le añade una nueva rama.
- Si la cadena es vacía y se llega a un nodo, significa que se ha completado el recorrido, y basta con marcar el nodo como `true`.
- Si la cadena es vacía y el nodo no es un `Nodo` (por ejemplo, una hoja), simplemente se retorna sin cambios, pues las hojas ya están marcadas por definición.

Este flujo funcional permite modificar de manera localizada solo las ramas necesarias del Trie, respetando la inmutabilidad de las estructuras y garantizando que no se pierda información previa al adicionar nuevas cadenas.

Técnicas de programación funcional utilizadas:

- Reconocimiento de patrones sobre tuplas `(s, t)` y descomposición estructural de listas y nodos.
- Uso de `map` para transformar selectivamente los hijos del nodo actual.
- Uso de funciones de alto orden como `map` y `++` para crear nuevas listas de hijos sin mutar las existentes.

- Encapsulación de lógica auxiliar dentro de funciones anidadas.
- Composición funcional para construir nuevas estructuras a partir de las anteriores, de forma inmutable.

En resumen, la función `adicionar` demuestra un uso completo y bien integrado del paradigma funcional: todas las decisiones estructurales del algoritmo están expresadas mediante patrones, llamadas recursivas y operaciones sobre estructuras inmutables. Esta implementación respeta la semántica de los árboles de sufijos, garantiza la corrección del Trie resultante y mantiene un estilo declarativo y expresivo en todo momento.

2.1.11. Árbol de sufijos: arbolDeSufijos

La operación de construcción del árbol de sufijos corresponde al proceso de generar una estructura Trie que contenga todos los sufijos posibles de una colección de cadenas. Esta función es esencial para la solución turbo acelerada del problema de reconstrucción de cadenas, ya que permite almacenar de forma eficiente un conjunto extenso de subcadenas y verificar su pertenencia sin necesidad de realizar búsquedas secuenciales costosas.

El algoritmo general consiste en obtener todos los sufijos de todas las cadenas de entrada, y luego adicionar uno por uno cada sufijo al Trie. Para esto se parte de un Trie inicial vacío, representado por un nodo raíz con carácter neutro (‘’ ’‘), no marcado y sin hijos. A partir de este Trie inicial, se recorre la lista de sufijos y se aplica la operación ‘adicionar‘ para ir insertándolos progresivamente en la estructura.

```
def arbolDeSufijos(ss: Seq[Seq[Char]]): Trie = {
  def sufijosDeSec(ss: Seq[Seq[Char]]): Seq[Seq[Char]] = {
    ss.foldLeft(Seq.empty[Seq[Char]])((suf, cad) =>
      suf ++ (for (n <- 0 until cad.length) yield cad.drop(n)))
  }

  val trie: Trie = Nodo(car = ' ', marcada = false, List())
  val sufijos = sufijosDeSec(ss)

  sufijos.foldLeft(trie)((t, s) => adicionar(s, t))
}
```

Listing 20: Función arbolDeSufijos

Uso de función auxiliar: `sufijosDeSec` La función auxiliar `sufijosDeSec` se encarga de transformar una secuencia de cadenas en una única secuencia que contiene todos los sufijos de todas las cadenas originales. Esta transformación es necesaria para generar el conjunto de entradas que serán insertadas en el Trie. Internamente, se utiliza un `foldLeft` que actúa como acumulador funcional, y una expresión `for` que itera sobre los índices posibles de cada cadena, aplicando `drop(n)` para generar los sufijos a partir de cada posición.

```
def sufijosDeSec(ss: Seq[Seq[Char]]): Seq[Seq[Char]] = {
  ss.foldLeft(Seq.empty[Seq[Char]])((suf, cad) =>
    suf ++ (for (n <- 0 until cad.length) yield cad.drop(n)))
}
```

Listing 21: Función auxiliar sufijosDeSec

Esta función representa de forma elegante el principio de recursión sobre una colección, junto con la composición funcional que caracteriza al paradigma. Al ser inmutable y pura, asegura la consistencia de los datos generados para la siguiente etapa del algoritmo.

Técnicas de programación funcional utilizadas en `sufijosDeSec`:

- Expresiones `for` con generadores para producir, a partir de cada índice, una subcadena correspondiente a un sufijo.
- Composición funcional mediante la concatenación de secuencias (`++`) como forma de acumulación funcional.
- Inmutabilidad en las colecciones, ya que no se modifican estructuras previas, sino que se construyen nuevas en cada paso.
- Encapsulación lógica a través de una función auxiliar anidada dentro de la función principal.

Análisis de la implementación en Scala

Una vez construida la secuencia de todos los sufijos de entrada, la función `arbolDeSufijos` define un Trie inicial vacío:

```
1 val trie: Trie = Nodo(car = ' ', marcada = false, List())
```

Listing 22: Inicialización del Trie vacío

Este nodo representa la raíz del árbol, no está marcado y no contiene hijos. A continuación, se genera la secuencia de sufijos aplicando la función auxiliar `sufijosDeSec` sobre la secuencia de entrada:

```
1 val sufijos = sufijosDeSec(ss)
```

Listing 23: Obtención de todos los sufijos

Esta línea produce una secuencia plana que contiene todos los sufijos posibles de todas las cadenas originales. Es importante destacar que cada elemento de esta secuencia será tratado como una cadena independiente que debe añadirse al árbol.

Sobre esta secuencia, se aplica un `foldLeft` que itera por cada sufijo y lo inserta en la estructura del Trie, utilizando la función previamente desarrollada `adicionar`. Dado que `adicionar` respeta las ramas ya existentes y solo modifica lo necesario, este proceso genera un árbol de sufijos estructuralmente correcto, sin caminos redundantes.

```
1 sufijos.foldLeft(trie)((t, s) => adicionar(s, t))
```

Listing 24: Inserción de todos los sufijos

El uso de `foldLeft` en esta etapa permite construir el árbol de forma acumulativa, pasando el Trie resultante de cada adición como nuevo acumulador al siguiente paso. El resultado final es un Trie que contiene todos los caminos posibles que corresponden a los sufijos de todas las cadenas de entrada.

Técnicas de programación funcional utilizadas:

- Encapsulación de lógica auxiliar mediante función anidada.
- Composición funcional para la integración del Trie usando `foldLeft` y `adicionar`.
- Funciones de alto orden: `foldLeft` y `map`/`yield` usadas como mecanismo de recorrido y generación estructurada.
- Inmutabilidad en las transformaciones intermedias y finales.
- Declaratividad y transparencia en la construcción del resultado, sin estados intermedios mutables ni estructuras de control imperativas.

El resultado es una función eficiente, expresiva y alineada completamente con los principios de la programación funcional. Permite generar de forma sistemática un Trie que agrupa una gran cantidad de información de manera estructurada y óptima para consultas posteriores.

2.1.12. Solución turbo acelerada: reconstruirCadenaTurboAcelerada

la solución turbo acelerada, que incorpora técnicas adicionales de filtrado y verificación optimizada de subcadenas, como el uso de árboles de sufijos u otras estructuras especializadas.

La idea fundamental de este enfoque es mantener la estrategia de duplicación progresiva de longitud de las combinaciones, pero introduciendo mecanismos eficientes que permiten verificar de manera rápida si las subcadenas relevantes de cada candidata pertenecen al conjunto de soluciones válidas generadas en etapas anteriores.

En este método, la reconstrucción de S se modela como la generación iterativa de subconjuntos $\mathcal{C}_k \subseteq \Sigma^k$, pero ahora, en cada paso, no solo se concatenan pares de cadenas válidas, sino que también se comprueba, utilizando estructuras como árboles de sufijos, que todas las subcadenas significativas de longitud $k/2$ de la nueva combinación están presentes entre los candidatos previos. Esta validación avanzada permite descartar de manera mucho más eficiente aquellas combinaciones que no pueden conducir a la solución, reduciendo aún más el espacio de búsqueda.

Formalmente, el procedimiento sigue la generación y refinamiento del conjunto:

$$\mathcal{C}_k = \left\{ w \| v \;\middle|\; w, v \in \mathcal{C}_{k/2},\ \forall\, 0 \leq i < k,\ s_i = \text{subcadena de longitud } \frac{k}{2} \text{ de } (w\|v),\ s_i \in \mathcal{C}_{k/2},\ \Psi_S(w\|v) = \text{True} \right\}$$

donde la verificación de la pertenencia de cada subcadena s_i al conjunto $\mathcal{C}_{k/2}$ se realiza de manera eficiente mediante una estructura especializada, y $w\|v$ denota la concatenación de w y v.

El algoritmo se inicia con el conjunto de cadenas más cortas —las letras individuales del alfabeto validadas por el oráculo— y en cada etapa duplica la longitud de las combinaciones, verificando rápida y eficazmente la validez de todas las subcadenas intermedias gracias a la estructura acelerada empleada. Este proceso se repite recursivamente hasta alcanzar la longitud n.

Gracias a este filtrado acelerado, el tamaño del espacio de búsqueda se reduce drásticamente en cada etapa, evitando la generación y evaluación de combinaciones redundantes o imposibles.

El resultado es una búsqueda que combina la eficiencia estructural del método turbo con la potencia de estructuras de datos avanzadas, logrando así un rendimiento notablemente superior incluso en instancias de gran tamaño o complejidad.

```
def reconstruirCadenaTurboAcelerada(n: Int, o: Oraculo): Seq[Char] = {
  // Usa arboles de sufijos para verificar si una subcadena pertenece al
     conjunto de candidatos
  def filtrar(sc1: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
    val scTrie = arbolDeSufijos(sc1)
    for {
      w <- sc1
      v <- sc1
      s = w ++ v
      if (1 until k).forall(c => pertenece(s.slice(c, c + k), scTrie))
    } yield s
  }

  // Recursion que va duplicando la longitud hasta alcanzar n
  def aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char] = {
    val sigmaK = filtrar(scInicial, k / 2).filter(o)
    val cadenaW = sigmaK.head
    if (k == n) cadenaW else aux(sigmaK, k * 2)
  }

  // Inicializa con las letras individuales filtradas por el oraculo
  val initialSc = alfabeto.map(c => Seq(c)).filter(o)
  aux(initialSc, 2)
}
```

Listing 25: Implementación de `reconstruirCadenaTurboAcelerada`

Uso de función auxiliar: `filtrar`

Para construir las combinaciones candidatas en cada etapa del algoritmo turbo acelerado, se define una función auxiliar denominada `filtrar`. Esta función toma como entrada el conjunto de secuencias válidas generadas en la iteración anterior—es decir, aquellas cadenas de longitud $k/2$ que cumplen con el criterio del oráculo—y genera todas las posibles secuencias de longitud k mediante la concatenación de pares de dichas cadenas. A diferencia de la versión turbo mejorada, la función `filtrar` de la versión acelerada incorpora el uso de una estructura eficiente (como un árbol de sufijos) para comprobar rápidamente que cada subcadena relevante de longitud $k/2$ en la nueva combinación también pertenece al conjunto de candidatos previos. Este procedimiento permite que la validación de subcadenas se realice de manera mucho más eficiente, lo que se traduce en una reducción considerable del tiempo de ejecución y en una mayor escalabilidad para cadenas largas.

```
def filtrar(sc: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
    val scTrie = arbolDeSufijos(sc)
    for {
        w <- sc
        v <- sc
        s = w ++ v
        if (1 until k).forall(i => pertenece(s.slice(i, i + k), scTrie))
    } yield s
}
```

Listing 26: Función auxiliar filtrar para Turbo Acelerada

La función `filtrar` recorre todos los pares posibles de secuencias del conjunto actual `sc`, genera sus concatenaciones y utiliza la estructura eficiente `scTrie` para comprobar, mediante la función de orden superior `forall`, que cada subcadena relevante de la nueva secuencia está presente en el conjunto de candidatos previos. Solo si todas estas subcadenas son válidas según la estructura de datos, la secuencia concatenada se incluye en el nuevo conjunto de candidatos para la siguiente etapa del algoritmo. Este proceso incrementa de manera significativa la eficiencia del filtrado, permitiendo abordar instancias de

mayor tamaño y complejidad sin perder solidez ni consistencia en la generación de soluciones.

Uso de función auxiliar: `aux` Para coordinar el avance recursivo y la generación progresiva de soluciones en el algoritmo turbo acelerado, se define la función auxiliar `aux`. Esta función gestiona, en cada invocación, el conjunto actual de candidatos (`scInicial`) y la longitud k correspondiente. En cada llamada, `aux` utiliza la función `filtrar` para construir el conjunto de combinaciones válidas de longitud k a partir de los candidatos anteriores. Tras el filtrado, se aplica el oráculo a las nuevas secuencias para asegurar su validez global. Si la longitud objetivo n aún no ha sido alcanzada, la función realiza una llamada recursiva, duplicando la longitud y usando el nuevo conjunto de candidatos filtrados como entrada. Cuando finalmente se logra la longitud requerida, la función retorna la primera secuencia válida encontrada, garantizando la corrección de la solución bajo las hipótesis del problema.

```
def aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char] = {
val sigmaK = filtrar(scInicial, k / 2).filter(o)
val cadenaW = sigmaK.head
if (k == n) cadenaW else aux(sigmaK, k * 2)
}
```

Listing 27: Función auxiliar aux para Turbo Acelerada

La función `aux` articula el flujo principal del algoritmo, alternando en cada etapa la generación eficiente de nuevos candidatos (gracias a la función `filtrar` y al uso de estructuras especializadas) con la validación final mediante el oráculo. Este esquema recursivo no solo mantiene la claridad y modularidad del diseño funcional, sino que también permite escalar el algoritmo para abordar cadenas de gran longitud y complejidad, aprovechando al máximo las ventajas de la aceleración estructural implementada en el filtrado.

Análisis de la implementación en Scala

La implementación de la función `reconstruirCadenaTurboAcelerada` sigue una estrategia iterativa y funcional basada en la construcción progresiva y filtrado eficiente de secuencias, usando auxiliares y estructuras avanzadas. El proceso puede detallarse en los siguientes pasos:

- **Inicialización:** Se comienza generando el conjunto inicial de candidatos, correspondiente a las letras individuales del alfabeto (`alfabeto.map(c =>Seq(c))`), que luego son filtradas mediante el oráculo (`filter(o)`) para obtener solo aquellas que pueden ser el inicio de la cadena buscada.
- **Función auxiliar `filtrar`:** En cada etapa, la función `filtrar` toma como entrada el conjunto de secuencias válidas actuales (`sc`) y la longitud k. Se construye una estructura eficiente, como un árbol de sufijos (`arbolDeSufijos(sc)`), que permite comprobar de manera óptima la pertenencia de subcadenas. Para cada par de candidatos `w` y `v` en `sc`, se concatena (`s = w ++ v`), generando así todas las posibles secuencias de longitud k. Luego, usando la función de orden superior `forall`, se verifica que cada subcadena relevante de longitud $k/2$ en la nueva secuencia también está presente en la estructura eficiente (`pertenece(..., scTrie)`). Solo las combinaciones que cumplen este filtro avanzado se mantienen como candidatas.
- **Función auxiliar `aux`:** Esta función orquesta el avance recursivo del algoritmo. Recibe como argumentos el conjunto actual de candidatos y la longitud actual k. Llama a `filtrar` para generar las nuevas combinaciones, filtra nuevamente por el oráculo para asegurar validez global, y selecciona una secuencia candidata mediante `head`. Si la longitud objetivo n no ha sido alcanzada, se invoca recursivamente a sí misma con el nuevo conjunto filtrado y longitud duplicada (`k * 2`); de lo contrario, retorna la primera secuencia válida hallada.
- **Iteración y selección de solución:** El proceso recursivo continúa hasta que se alcanza la longitud n, momento en el cual se retorna la cadena buscada, garantizando que cumple tanto los criterios locales de subcadenas como la validación global del oráculo.

- **Eficiencia y escalabilidad:** La utilización de una estructura especializada para la verificación de subcadenas (por ejemplo, un trie o árbol de sufijos) transforma el filtrado de costoso y lineal a un proceso prácticamente constante o logarítmico en el tamaño del conjunto. Esto permite que la solución escale a cadenas de mayor longitud y conjuntos de candidatos más extensos, sin sacrificar la corrección ni la transparencia funcional.

En conjunto, la implementación de `reconstruirCadenaTurboAcelerada` articula recursión, filtrado funcional y validación eficiente, manteniendo en todo momento la inmutabilidad de los datos y la claridad declarativa.

Técnicas de programación funcional utilizadas:

- **Recursión de cola.** La función auxiliar `aux` aplica recursión para construir secuencias cada vez más largas, duplicando su tamaño en cada paso hasta alcanzar la longitud deseada.
- **Expresiones `for`.** Se utilizan comprensiones `for` para construir todas las posibles concatenaciones de candidatos válidos, lo que representa el producto cartesiano de secuencias, de manera declarativa y legible.
- **Funciones de alto orden.** Empleo de funciones como `filter`, `forall` y `map` para filtrar y verificar propiedades sobre colecciones. En particular, `forall` comprueba que todas las subcadenas relevantes estén presentes en el árbol de sufijos.
- **Inmutabilidad de datos.** Todas las estructuras (`Seq[Char]`, `Seq[Seq[Char]]`) utilizadas en el proceso son inmutables, garantizando que no haya efectos secundarios y facilitando la transparencia referencial.
- **Composición funcional.** El flujo del algoritmo es la composición directa de funciones puras, evitando el uso de variables mutables y control imperativo, y favoreciendo la claridad y modularidad del código.
- **Encapsulamiento mediante funciones auxiliares.** Tanto la función de filtrado (`filtrar`) como la función recursiva (`aux`) están anidadas en la función principal, permitiendo una gestión localizada del estado y mejor organización del código.
- **Uso de estructuras especializadas funcionales.** Se utiliza un árbol de sufijos (`arbolDeSufijos`) como estructura funcional para mejorar la eficiencia en las búsquedas de subcadenas, integrando técnicas avanzadas dentro del paradigma funcional.

2.1.13. Solución turbo acelerada paralela: `reconstruirCadenaTurboAceleradaPar`

En línea con los avances previos, la solución turbo acelerada también admite una versión paralela, denominada `reconstruirCadenaTurboAceleradaPar`. Esta variante aprovecha tanto el filtrado eficiente de subcadenas mediante árboles de sufijos como la capacidad de computar en paralelo las combinaciones y validaciones en cada etapa, lo que permite abordar instancias de mayor tamaño en tiempos significativamente reducidos.

La naturaleza independiente de las operaciones de generación y verificación de pares de secuencias en cada etapa habilita una paralelización segura y efectiva, sin interferencias ni condiciones de carrera.

```
def reconstruirCadenaTurboAceleradaPar(umbral: Int)(n: Int, o: Oraculo): Seq[
    Char] = {
// Usa arboles de sufijos y paraleliza la generacion y comprobacion de
    combinaciones
def filtrar(sc1: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
    val scTrie = arbolDeSufijos(sc1)
    (for {
        w <- sc1.par
        v <- sc1
        s = w ++ v
        if (1 until k).forall(c => pertenece(s.slice(c, c + k), scTrie))
    } yield s).seq
}

    def aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char] = {
        val sigmaK = filtrar(scInicial, k / 2).par.filter(o).seq
        val cadenaW = sigmaK.head
        if (k == n) cadenaW else aux(sigmaK, k * 2)
    }

    val initialSc = alfabeto.map(c => Seq(c)).filter(o)
    aux(initialSc, 2)
}
```

Listing 28: Implementación de `reconstruirCadenaTurboAceleradaPar`

Uso de función auxiliar: `filtrar` paralela La función auxiliar `filtrar` implementa la generación y validación de combinaciones en paralelo. Utilizando `sc1.par`, la iteración externa del producto cartesiano se distribuye entre los distintos núcleos del sistema, permitiendo procesar muchas combinaciones simultáneamente. Además, la comprobación de pertenencia de subcadenas se realiza mediante un árbol de sufijos, maximizando la eficiencia de la verificación.

```
def filtrar(sc: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
    val scTrie = arbolDeSufijos(sc)
    (for {
        w <- sc.par
        v <- sc
        s = w ++ v
        if (1 until k).forall(i => pertenece(s.slice(i, i + k), scTrie))
    } yield s).seq
}
```

Listing 29: Función auxiliar filtrar paralela para Turbo Acelerada

Principales diferencias respecto a la versión secuencial

- **Paralelización integral de la generación y verificación:** La iteración principal del producto cartesiano y la comprobación de subcadenas relevantes se distribuyen entre los hilos del sistema gracias al uso de colecciones paralelas (`.par`). Esto permite una aceleración lineal (idealmente) respecto al número de núcleos disponibles, especialmente en casos de alta carga computacional.
- **Uso combinado de estructuras avanzadas y concurrencia:** Se mantiene la ventaja estructural del árbol de sufijos para las verificaciones rápidas de subcadenas, ahora aplicada de forma concurrente sobre múltiples combinaciones candidatas.
- **Eficiencia y escalabilidad superiores:** El enfoque paralelo, junto con la validación acelerada, permite abordar valores de n mucho más grandes y conjuntos de candidatos mucho más extensos que en cualquier versión previa, manteniendo la corrección y transparencia del paradigma funcional.

2.2. Estructuras de datos usadas en cada función

Función	LazyList	Seq	Option	Trie	ParSeq	List
`reconstruirCadenaIngenuo`	Sí	Sí	Sí	No	No	Sí
`reconstruirCadenaMejorado`	Sí	Sí	Sí	No	No	Sí
`reconstruirCadenaTurbo`	No	Sí	No	No	No	Sí
`reconstruirCadenaTurboMejorada`	No	Sí	No	No	No	Sí
`reconstruirCadenaTurboAcelerada`	No	Sí	Sí	Sí	No	Sí
`reconstruirCadenaIngenuoPar`	Sí	Sí	Sí	No	No	Sí
`reconstruirCadenaMejoradoPar`	Sí	Sí	Sí	No	Sí	Sí
`reconstruirCadenaTurboPar`	No	Sí	No	No	Sí	Sí
`reconstruirCadenaTurboMejoradaPar`	No	Sí	No	No	Sí	Sí
`reconstruirCadenaTurboAceleradaPar`	No	Sí	No	Sí	Sí	Sí
`adicionar`	No	Sí	No	Sí	No	Sí
`pertenece`	No	Sí	No	Sí	No	Sí
`arbolDeSufijos`	No	Sí	No	Sí	No	Sí

Cuadro 2: *
Tabla de estructuras de datos utilizadas por cada función del proyecto

En este análisis , examinaremos algunas una de las funciones implementadas desde la perspectiva de las estructuras de datos que emplean: Cabe recalcar que se marca el uso de List, consecutivamente al uso de Seq, dado que al usar el trait Seq; Scala usará por defecto la colección List, a pesar de no definirse en el código.

Función: `reconstruirCadenaIngenuo`

Seq[Char]: Se usa porque permite una construcción eficiente y flexible de cadenas de caracteres. Es inmutable, lo que garantiza seguridad al componer secuencias en distintas ramas de la recursión.
Su uso es fundamental para:

- Dividir y combinar cadenas (`s1 ++ s2`)
- Construir intentos parciales de reconstrucción

Función: `reconstruirCadenaMejorado`

LazyList: Se utiliza para generar todas las combinaciones posibles de secuencias de longitud n mediante evaluación perezosa. En el cuerpo de la función:

```
LazyList[Seq[Char]]
```

esto permite construir un espacio de búsqueda enorme sin generar todas las combinaciones de inmediato. En particular, se define recursivamente:

```
def generarCombinaciones(n: Int): LazyList[Seq[Char]]
```

Cada llamada va generando combinaciones de longitud n filtradas por el oráculo, y el uso de LazyList asegura que se detendrá tan pronto encuentre una combinación válida (gracias a `headOption`).

Seq[Char]: Se usa como representación inmutable de cada posible secuencia de caracteres candidata a ser la solución. Esta estructura permite construir nuevas combinaciones de forma segura, por ejemplo, al anteponer un carácter:

```
c +: suf
```

donde `suf` es una secuencia ya validada parcialmente. También se usa como retorno final de la función, por lo que es el tipo de la secuencia reconstruida.

Option[Seq[Char]]: Aparece implícitamente a través de la expresión:

```
generarCombinaciones(n).headOption.getOrElse(Seq.empty)
```

La llamada a `headOption` devuelve un `Option`, lo que permite manejar de forma segura el caso en que no se encuentre ninguna combinación válida. En tal caso, se retorna `Seq.empty` mediante `getOrElse`, garantizando que la función no falle ante una LazyList vacía. Esta estructura es crucial para el control de flujo sin errores en la etapa final del algoritmo.

Función: `reconstruirCadenaTurbo`

Seq[Char] y **Seq[Seq[Char]]:** La función `reconstruirCadenaTurbo` se basa exclusivamente en el uso de secuencias inmutables para representar tanto cadenas individuales como conjuntos de combinaciones. Las estructuras clave utilizadas son:

- `Seq[Char]`: representa una posible secuencia candidata a ser reconstruida.
- `Seq[Seq[Char]]`: representa un conjunto de múltiples secuencias que serán filtradas o combinadas en cada etapa.

El uso de `Seq` permite trabajar con una colección inmutable de combinaciones, garantizando que cada paso produzca una nueva colección sin modificar la anterior. Esta inmutabilidad es esencial para preservar la claridad funcional del código y evitar efectos colaterales entre iteraciones.

Función: `reconstruirCadenaTurboMejorada`

Seq[Char] y **Seq[Seq[Char]]:** Esta función mejora la eficiencia de `reconstruirCadenaTurbo` al agregar un filtro más restrictivo, pero sigue utilizando exclusivamente secuencias inmutables como estructuras de datos clave.

- `Seq[Char]` representa una posible secuencia candidata de longitud creciente que el algoritmo intenta validar.
- `Seq[Seq[Char]]` representa un conjunto de candidatos que se construyen y filtran a lo largo del proceso iterativo.

La función interna más relevante es:

```
def filtrar(sc: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
  for {
    w <- sc
    v <- sc
    s = w ++ v
    if (1 until k).forall(i => sc.contains(s.slice(i, i + k)))
  } yield s
}
```

Aquí se genera una nueva secuencia `s = w ++ v`, resultado de combinar dos secuencias válidas anteriores. Sin embargo, en lugar de simplemente verificar `o(s)`, esta versión refuerza la validez exigiendo que *todos los subsegmentos contiguos de tamaño k* de la nueva secuencia también estén presentes en la colección anterior `sc`. Esto permite descartar muchas combinaciones inválidas anticipadamente.

El uso de `Seq` es clave por varias razones:

- Es una estructura inmutable, lo que evita efectos colaterales durante la generación y validación de candidatos.

- Permite fácilmente operaciones como `++`, `slice`, `forall`, `contains` y comprensión por `for`, todas fundamentales en la función.
- Su naturaleza ordenada permite controlar con precisión la posición de subcadenas, algo necesario en la validación de segmentos con `s.slice(i, i + k)`.

La estructura `Seq` proporciona la base sobre la cual se construye y filtra el árbol de combinaciones crecientes de forma eficiente, sin comprometer la seguridad ni la claridad del algoritmo.

Función: `reconstruirCadenaTurboAcelerada`

Trie: Es la estructura central que diferencia esta versión "turbo acelerada" de las anteriores. Se construye previamente a partir del alfabeto y el oráculo mediante la función:

```
val t = arbolDeSufijos(alfabeto, o)
```

Este `Trie` contiene todas las subcadenas válidas que pueden aparecer durante el proceso de reconstrucción. Su uso permite reemplazar múltiples llamadas al oráculo por simples consultas al árbol, acelerando así el filtrado de combinaciones de longitud creciente.

Durante el filtrado de candidatos, se evalúa que todas las subcadenas de longitud k estén en el `Trie`:

```
if (1 until k).forall(i => pertenece(t, s.slice(i, i + k)))
```

Esta técnica permite una validación más eficiente, ya que el acceso al `Trie` es rápido y estructurado, a diferencia de la evaluación directa del oráculo que puede implicar cómputos costosos.

Seq[Char] y **Seq[Seq[Char]]:** Como en las versiones anteriores, `Seq` se utiliza para representar secuencias individuales y colecciones de candidatos. La operación central sigue siendo la combinación:

```
s = w ++ v
```

donde `s` es una nueva secuencia formada por la concatenación de dos anteriores. Esta estructura inmutable permite componer candidatos sin modificar las secuencias originales, respetando el paradigma funcional.

List[Char]: Aunque el procesamiento principal se hace con `Seq`, cada subcadena evaluada contra el `Trie` debe convertirse en `List`, ya que la función `pertenece` fue diseñada para operar con listas:

```
pertenece(t, s.slice(i, i + k).toList)
```

Esto refleja una decisión de diseño en la definición del `Trie`, centrada en facilitar el recorrido recursivo usando listas.

Option[Seq[Char]]: La función retorna la primera combinación válida encontrada (si existe) usando:

```
headOption.getOrElse(Seq.empty)
```

Esto genera un `Option[Seq[Char]]`, que permite devolver de forma segura una solución o una secuencia vacía si ninguna fue válida. De esta manera, se evita el uso de excepciones o valores nulos.

Función: `reconstruirCadenaIngenuoPar`

LazyList[Seq[Char]]: La función se apoya en la evaluación perezosa para generar combinaciones de secuencias sin construirlas todas de forma anticipada. El fragmento clave:

```
val intentos = LazyList.from(alfabeto).map(c => intento ++ Seq(c))
```

crea un flujo infinito (o potencialmente grande) de secuencias parciales, extendiendo una combinación previa con cada carácter del alfabeto. Gracias a que `LazyList` solo evalúa lo necesario, esto permite eficiencia en memoria y en tiempo cuando se encuentra una solución válida tempranamente.

Option[Seq[Char]]: El resultado de buscar una solución válida entre múltiples candidatos se representa mediante `Option`, retornando `Some(solución)` o `None`. Esto se observa en la expresión:

```
    parallel(
      combinaciones.slice(...).find(o),
      ...
    ).orElse(...)
```

Cada intento devuelve una `Option`, y las combinaciones de resultados se encadenan con `orElse` para quedarse con la primera solución exitosa. Esta técnica permite implementar una lógica de búsqueda robusta y limpia sin lanzar excepciones ni usar valores nulos.

parallel(...): No se utiliza `ParSeq`, sino una función auxiliar `parallel` que permite evaluar en paralelo dos búsquedas sobre subconjuntos de combinaciones. Esto busca acelerar la evaluación de oráculos sobre LazyLists grandes, dividiendo el trabajo entre múltiples hilos:

```
    val medio = combinaciones.length / 2
    val (c1, c2) = combinaciones.splitAt(medio)
    parallel(c1.find(o), c2.find(o))
```

Este patrón divide el espacio de búsqueda y combina los resultados usando `Option.orElse`, seleccionando el primero que funcione.

Seq[Char]: Como en otras funciones del proyecto, se utiliza para representar cada secuencia candidata. La inmutabilidad de `Seq` garantiza que cada intento parcial pueda manipularse de forma segura sin afectar el resto de combinaciones.

Función: `reconstruirCadenaMejoradoPar`

ParSeq[Seq[Char]] (Estructura principal): La función construye combinaciones mediante recursión paralelizada:

```
def generarCombinaciones(n: Int): ParSeq[Seq[Char]] = {
  if (n == 0) LazyList(Seq.empty).par // Conversion inicial
  else (for {
    suf <- generarCombinaciones(n - 1).filter(o) // Filtrado temprano
    c <- alfabeto
  } yield c +: suf).par // Paralelizacion explicita
}
```

Características clave:

- Conversión perezosa inicial (`LazyList.toParSeq`)
- Filtrado paralelo de subsecuencias válidas
- Generación recursiva de combinaciones

Option[Seq[Char]] (Manejo de resultados): La búsqueda paralela devuelve un resultado envuelto:

```
generarCombinaciones(n).headOption.getOrElse(Seq.empty)
```

- `headOption`: Obtiene el primer resultado paralelo como `Option`
- `getOrElse`: Proporciona fallback seguro

Seq[Char] (Estructura base):

- Tipo de todas las subsecuencias generadas
- Inmutable y segura para concurrencia
- Soporta operaciones funcionales (`+:`, `filter`, etc.)

Función: `reconstruirCadenaTurboPar`

ParSeq[Seq[Char]] (Paralelismo de datos): El núcleo del algoritmo paraleliza la generación de combinaciones:

```
    (for {
      w <- sc.par  // Paraleliza el primer generador
      v <- sc      // Mantiene secuencial el segundo
    } yield w ++ v).seq  // Vuelve a secuencial para siguiente paso
```

- `.par`: Distribuye el trabajo entre hilos
- `.seq`: Garantiza orden para pasos posteriores
- Estrategia híbrida: Solo paraleliza un generador para evitar sobrecarga

Seq[Char] (Estructura base):

- Representa tanto subsecuencias parciales como resultado final
- Inmutabilidad clave para seguridad en concurrencia
- Operaciones principales: `++` (concat) y `filter(o)`

Parámetro `umbral`: Controla cuándo activar el paralelismo:

```
if (sc.length > umbral) generarSigmaK(sc.par) else generarSigmaK(sc)
```

Función: `reconstruirCadenaTurboMejoradaPar`

Estrategia Turbo Mejorada (Divide y Vencerás con Recursión): La función implementa una versión optimizada del enfoque turbo. Se basa en dividir el problema en mitades sucesivas y combinar subsecuencias válidas mediante concatenación. A diferencia de versiones anteriores, introduce un filtrado adicional que evita combinaciones irrelevantes.

ParSeq[Seq[Char]] (Paralelismo de datos): El algoritmo utiliza colecciones paralelas para generar combinaciones de forma eficiente. Las combinaciones `w ++ v` se generan a partir de un producto cartesiano paralelo:

```
(for {
  w <- sc.par
  v <- sc
  s = w ++ v
  if (1 until k).forall(i => sc.contains(s.slice(i, i + k)))
} yield s)
```

Este patrón paraleliza el primer generador para distribuir el trabajo entre hilos, mientras mantiene el segundo secuencial para evitar sobrecarga. La evaluación resultante es un `ParSeq[Seq[Char]]`, que mejora el rendimiento al aprovechar varios núcleos del procesador.

Seq[Char] y Seq[Seq[Char]] (Estructuras base):

- `Seq[Char]` representa cada subsecuencia candidata, construida como `w ++ v`.
- `Seq[Seq[Char]]` contiene todas las combinaciones aceptadas tras el filtro inteligente.
- Son estructuras inmutables, ideales para ejecución concurrente segura y eficientes para operaciones funcionales como `slice`, `contains`, y `forall`.

Resumen: Esta función no utiliza `Option` ni estructuras auxiliares como `Trie`. La mejora clave radica en su filtro inteligente aplicado sobre combinaciones generadas en paralelo, junto con una recursión que duplica progresivamente el tamaño de las secuencias hasta alcanzar la longitud deseada.

Función: `reconstruirCadenaTurboAceleradaPar`

Estrategia turbo paralela con filtrado inteligente y Trie auxiliar: Esta versión es la más optimizada del enfoque turbo. Divide el problema en pasos sucesivos de longitud k, generando y filtrando combinaciones paralelamente. En cada etapa, las combinaciones de longitud k se filtran utilizando un `Trie` como estructura auxiliar para verificar si todas sus subcadenas internas ya han sido generadas.

El flujo es recursivo, duplicando k en cada paso hasta alcanzar n:

```
aux(combinacionesIniciales, k = 2)
  ↓
aux(combinacionesFiltradas, k = 4)
  ↓
...
  ↓
k == n => devolver resultado
```

Filtrado inteligente con Trie (verificador transitorio):

A diferencia de otras versiones, aquí el `Trie` no representa todas las soluciones posibles, sino que se construye en cada llamada a `filtrar` para verificar la validez de nuevas combinaciones:

```
val scTrie = arbolDeSufijos(sc1)
if (1 until k).forall(c => pertenece(s.slice(c, c + k), scTrie))
```

Este filtro evita generar combinaciones inconsistentes, mejorando drásticamente la eficiencia. El `Trie` se usa como una estructura local de apoyo y se destruye tras cada paso.

Doble paralelización: El algoritmo paraleliza tanto la generación como el filtrado de combinaciones. Tras generar las posibles combinaciones con el filtro del Trie, se aplica el oráculo en paralelo:

`filtrar(...).par.filter(o).seq`

- `.par`: Distribuye la verificación entre hilos
- `.filter(o)`: Elimina combinaciones inválidas según el oráculo
- `.seq`: Convierte de nuevo a secuencial para el siguiente paso recursivo

ParSeq[Seq[Char]] (Generación paralela): Las combinaciones se generan usando un producto cartesiano parcialmente paralelizado:

```
    (for {
      w <- sc.par
      v <- sc
      s = w ++ v
      if (condicionTrie)
    } yield s)
```

Esta estrategia balancea rendimiento y sobrecarga paralela.

Option[Seq[Char]] (Resultado final): El resultado final se maneja de forma segura usando `Option`:

`resultado.headOption.getOrElse(Seq.empty)`

Permite retornar una secuencia válida o vacía en caso de que no se encuentre una solución.
Seq[Char], Seq[Seq[Char]], List[Seq[Char]] (Estructuras base):

- `Seq[Char]`: Representa cada subsecuencia candidata
- `Seq[Seq[Char]]`: Contiene las combinaciones en cada paso recursivo
- `List[Seq[Char]]`: Resultado de la búsqueda, sobre el cual se aplica `headOption`

Cada estructura de datos se emplea en un rol específico que maximiza la eficiencia sin sacrificar claridad ni seguridad.

Función: `adicionar`

Trie (Estructura central): La función `adicionar` inserta una nueva secuencia de caracteres en un `Trie` (árbol de sufijos). Este Trie está representado por dos tipos de nodos:

```
case class Nodo(car: Char, marcada: Boolean, hijos: List[Trie])
case class Hoja(car: Char, marcada: Boolean)
```

La estructura es completamente recursiva, donde cada nodo contiene un carácter, una marca booleana que indica si el camino forma una secuencia válida, y una lista de subtries como hijos.

- Si el Trie ya contiene la secuencia (verificada con `pertenece`), se retorna sin cambios.
- Si es una hoja y se requiere agregar más caracteres, se convierte en un nodo con nuevos hijos construidos recursivamente.

- Si es un nodo, se recorre su lista de hijos para localizar el subárbol correspondiente y se actualiza recursivamente.

Este enfoque evita mutaciones, garantizando seguridad estructural y consistencia funcional.

Seq[Char] (Secuencia a insertar): La secuencia de entrada se trata como una `Seq[Char]` y se descompone con pattern matching:

```
case s1 +: ss =>
```

Esto permite trabajar con la cabeza de la secuencia (`s1`) y el resto del sufijo (`ss`) de manera clara y funcional. Si se llega al final de la secuencia, se marca el nodo como completo.

Construcción recursiva: Cuando no se encuentra un hijo con el carácter deseado, se invoca la función auxiliar `construirRama` para crear una nueva rama entera del Trie, compuesta por hojas sucesivas, una por cada carácter restante.

```
def construirRama(s: Seq[Char]): Trie
```

Esta estrategia garantiza que todas las secuencias insertadas queden representadas como caminos completos en el Trie.

Resumen: `adicionar` es una función puramente funcional que extiende un Trie inmutable para incluir nuevas secuencias. Utiliza pattern matching sobre secuencias, construcción recursiva de subtries y mantiene la estructura jerárquica del árbol mediante listas de hijos. Es clave para formar el árbol de sufijos utilizado por los algoritmos de reconstrucción turbo.

Función: `pertenece`

Trie (Estructura central): La función `pertenece` determina si una secuencia de caracteres está contenida dentro de un `Trie`, representado mediante los siguientes tipos:

```
1 sealed trait Trie
2 case class Nodo(car: Char, marcada: Boolean, hijos: List[Trie])
3 case class Hoja(car: Char, marcada: Boolean)
```

Cada nodo contiene un carácter, una marca booleana que indica si el camino hasta ese nodo forma una secuencia válida, y una lista de hijos que representan las posibles continuaciones.

List[Trie] (Subtries de los nodos): Los hijos de cada nodo se almacenan como una `List[Trie]`. Para determinar si existe una rama que continúa con el siguiente carácter de la secuencia, se utiliza la función `exists`:

```
hijos.exists(node => raiz(node) == s1 && pertenece(ss, node))
```

Esta estrategia permite realizar una búsqueda eficiente sin necesidad de estructuras de mapeo adicionales. El uso de `exists` garantiza un recorrido corto, ya que se detiene al encontrar la primera coincidencia.

Seq[Char] (Secuencia a verificar): La secuencia de entrada se descompone mediante `pattern matching` como `s1 +: ss`, lo que permite evaluar el carácter actual y procesar recursivamente el resto.

Resumen: `pertenece` es una función recursiva que verifica si una secuencia de caracteres se encuentra registrada en un Trie. Utiliza listas para representar los hijos, `exists` para realizar búsquedas eficientes, y `pattern matching` para descomponer y recorrer la secuencia. Esta función es fundamental para validar combinaciones en estructuras de sufijos.

Función: `arbolDeSufijos`

Trie (Estructura central): La función `arbolDeSufijos` construye un `Trie` a partir de todos los sufijos de una secuencia de caracteres, almacenando cada uno mediante inserciones repetidas. Esta estructura permite realizar posteriormente búsquedas eficientes de subcadenas. El `Trie` se define mediante:

```
sealed trait Trie
case class Nodo(car: Char, marcada: Boolean, hijos: List[Trie])
case class Hoja(car: Char, marcada: Boolean)
```

La función reutiliza la función auxiliar `adicionar`, que incorpora secuencias de caracteres en el Trie de forma recursiva.

LazyList[Seq[Char]] (Generación de sufijos): Los sufijos se generan usando una estructura perezosa (`LazyList`) que construye todos los segmentos finales de la secuencia:

```
val sufijos = LazyList.tabulate(s.length)(i => s.drop(i))
```

Esto permite que los sufijos se calculen bajo demanda, lo cual es eficiente cuando no se requiere procesarlos todos inmediatamente.

List[Trie] (Hijos de los nodos): Cada nodo del `Trie` contiene una lista de subtries como hijos. Durante la inserción, se actualizan estas listas para incorporar las nuevas ramas derivadas de los sufijos.

Proceso completo:

1. Se generan todos los sufijos posibles de la secuencia de entrada mediante un `LazyList`.
2. Se inicializa un `Trie` vacío con un nodo raíz arbitrario.
3. Cada sufijo se inserta sucesivamente en el `Trie` usando la función `adicionar`.

Resumen: La función `arbolDeSufijos` crea un Trie a partir de todos los sufijos de una secuencia, utilizando una construcción perezosa para generarlos y una inserción estructurada para almacenarlos. Esta función es esencial para algoritmos que requieren búsquedas eficientes de subcadenas.

Conclusión general

A lo largo de este informe se ha realizado un análisis detallado de las estructuras de datos utilizadas en las distintas funciones del proyecto, clasificadas según su tipo y propósito. Se abordaron las funciones por paquetes, identificando estructuras clave como `Seq`, `LazyList`, `Option`, `Trie`, `ParSeq` y `List`, y se examinó el papel específico que cumple cada una en la lógica y eficiencia de los algoritmos implementados.

El análisis no solo consideró la presencia de estas estructuras, sino también su comportamiento (por ejemplo, inmutabilidad, evaluación perezosa, concurrencia), así como su interacción con funciones de orden superior y patrones de diseño funcional. Para cada función se describió cómo las estructuras se integran con la estrategia algorítmica empleada, ya sea por recursión, paralelismo o composición secuencial.

Asimismo, se elaboró una tabla comparativa que resume qué estructuras de datos son utilizadas por cada función del proyecto, lo cual facilita una visión general y permite identificar patrones de uso, modularidad y evolución de las implementaciones.

Este informe proporciona, en conjunto, una visión estructurada y completa del diseño funcional del proyecto, destacando cómo el uso adecuado de las estructuras de datos ha sido esencial para lograr soluciones expresivas, eficientes y escalables en el contexto de la programación funcional en Scala.

3. Informe de correcciones

Conceptualización previa

Antes de iniciar las demostraciones de corrección, definimos de manera formal los espacios, funciones y firmas que utilizaremos en todas las funciones de reconstrucción de cadenas.

Alfabeto y secuencias Sea el alfabeto que representa las bases nitrogenadas que componen una cadena de ADN

$$\Sigma = \{\mathtt{a}, \mathtt{c}, \mathtt{g}, \mathtt{t}\}.$$

De este modo,

$$\Sigma^n = \{\, s = (s_1, s_2, \ldots, s_n) \mid s_i \in \Sigma,\ 1 \leq i \leq n\}$$

es el conjunto de todas las cadenas (secuencias) de longitud exactamente n. De forma más general,

$$\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n$$

es el conjunto de todas las secuencias finitas sobre Σ, con $\Sigma^0 = \{\varepsilon\}$ (la cadena vacía).

Oráculo Un *oráculo* es una función de decisión sobre cadenas:

$$o : \Sigma^* \longrightarrow \{\text{true}, \text{false}\}.$$

El oráculo está definido a partir de una cadena solución fija $S \in \Sigma^n$ que se desea reconstruir. Evaluar una cadena $s \in \Sigma^*$ mediante el oráculo corresponde a verificar si s es una subcadena de S.

Esto se expresa formalmente como:

$$o(s) = \text{true} \iff s \subseteq_{\text{sub}} S,$$

donde $s \subseteq_{\text{sub}} S$ indica que s es una subcadena (consecutiva) de S.

La implementación del oráculo en Scala es la siguiente:

```
(s: Seq[Char]) => S.containsSlice(s)
```

Además, el oráculo introduce una latencia artificial mediante `Thread.sleep(delay)`, con el objetivo de simular que cada consulta tiene un costo computacional. Esto impacta directamente en la eficiencia de las funciones reconstrucción, incentivando la reducción de llamadas al oráculo.

Modelo de sustitución Para razonar sobre la evaluación de expresiones en Scala, emplearemos el *modelo de sustitución*, expandiendo paso a paso las definiciones de funciones auxiliares (p. ej. `generarCombinaciones`) y aplicando reglas de evaluación (aplicación de funciones, concatenación + :, selección `find`, etc.).

Firmas generales de las funciones de reconstrucción Todas las funciones que implementan reconstrucción ingenua o paralela comparten la misma firma matemática:

$$\texttt{reconstruir}(n, o) : \mathbb{N} \times \left(\Sigma^* \to \{\text{true}, \text{false}\}\right) \longrightarrow \Sigma^*.$$

Aquí:

- $n \in \mathbb{N}$ es la longitud deseada de la cadena de ADN a reconstruir.
- o es el oráculo que determina la validez de cualquier candidato $s \in \Sigma^*$.
- El resultado $s' \in \Sigma^*$ cumple
$$|s'| = n, \quad o(s') = \text{true}$$
si existe alguna solución de longitud n, y $s' = \varepsilon$ (cadena vacía) en caso contrario.

Este planteamiento es independiente de la estrategia interna (secuencial, paralela, uso de `LazyList`, `Iterator`, etc.), pues la corrección se basa únicamente en que la función devuelva la primera cadena válida (o vacía) según el oráculo.

Propiedad que deben cumplir las funciones de reconstrucción Aunque la firma de tipo indica que el resultado está en Σ^*, el comportamiento correcto de la función debe cumplir la siguiente especificación lógica:

$$\texttt{reconstruir}(n, o) : \mathbb{N} \times (\Sigma^* \to \{\text{true}, \text{false}\}) \to \Sigma^*$$

tal que:

si $\exists s \in \Sigma^n : o(s) = \text{true}$, entonces $\texttt{reconstruir}(n, o) \in \Sigma^n$ y $o(\texttt{reconstruir}(n, o)) = \text{true}$,

si no, entonces $\texttt{reconstruir}(n, o) = \varepsilon$.

De este modo, aunque la respuesta en notación pertenezca a Σ^*, las funciones están diseñadas para devolver una cadena de longitud n válida si existe, o la vacía en su defecto.

Corrección de la función `reconstruirCadenaIngenuo`

Enunciado formal

Queremos demostrar formalmente que el programa Scala `reconstruirCadenaIngenuo` implementa correctamente la especificación de reconstrucción de cadenas. Denotando por

- $Pf : \mathbb{N} \times (\Sigma^* \to \{\text{true}, \text{false}\}) \to \Sigma^*$ el comportamiento del programa Scala,
- $f : \mathbb{N} \times (\Sigma^* \to \{\text{true}, \text{false}\}) \to \Sigma^*$ la especificación deseada,

se busca demostrar:

$$\forall (n, o) \in \mathbb{N} \times (\Sigma^* \to \{\text{true}, \text{false}\}) : Pf(n, o) = f(n, o).$$

Es decir, el programa devuelve exactamente la primera cadena válida de longitud n que satisface el oráculo o, o ε si ninguna lo hace.

Descripción general del funcionamiento

La función `reconstruirCadenaIngenuo` recibe como entrada un número natural n y un oráculo $o : \Sigma^* \to \{\text{true}, \text{false}\}$, y retorna la primera secuencia de longitud n que satisface el oráculo, o la cadena vacía si no existe ninguna. Para lograr esto, genera todas las combinaciones posibles de longitud n sobre el alfabeto Σ, representadas como una `LazyList[Seq[Char]]`, y aplica una búsqueda filtrada utilizando la operación `find(o)` sobre subconjuntos de combinaciones, los cuales se combinan con `orElse` hasta encontrar la primera secuencia válida.

La generación de combinaciones se realiza recursivamente mediante una función auxiliar `generarCombinaciones`, definida estructuralmente como:

$$\texttt{generarCombinaciones}(n) = \begin{cases} \texttt{LazyList}(\varepsilon) & \text{si } n = 0, \\ \{\, a+ : s \mid a \in \Sigma,\ s \in \texttt{generarCombinaciones}(n-1) \,\} & \text{si } n > 0. \end{cases}$$

El resultado de `reconstruirCadenaIngenuo(n, o)` se construye a partir de esta lista con una búsqueda perezosa mediante llamadas a `find(o)`, asegurando que la evaluación se detiene al hallar la primera cadena válida. Es por esta razón, que hay que demostrar que `generarCombinaciones` devuelve efectivamente el total de las cadenas correspondientes para cualquier longitud de una cadena.

Definición y propósito de la función `generarCombinaciones`

La función auxiliar `generarCombinaciones` es una construcción recursiva que tiene como objetivo producir todas las posibles cadenas de longitud exacta n sobre el alfabeto Σ. Se implementa como una función de tipo

$$\texttt{generarCombinaciones} : \mathbb{N} \longrightarrow \texttt{LazyList}(\Sigma^*),$$

y su propósito es construir el conjunto Σ^n de todas las secuencias de longitud n, de forma perezosa. Esta generación se realiza en orden lexicográfico implícito, garantizando que la evaluación posterior con el oráculo se realice de forma determinista y eficiente en términos de memoria.

Su definición estructural es la siguiente:

$$\texttt{generarCombinaciones}(n) = \begin{cases} \texttt{LazyList}(\varepsilon) & \text{si } n = 0, \\ \{\, a+ : s \mid a \in \Sigma,\ s \in \texttt{generarCombinaciones}(n-1) \,\} & \text{si } n > 0. \end{cases}$$

Esta expresión significa lo siguiente:

- Cuando $n = 0$, la única cadena posible es la vacía ε, y por tanto el resultado es un `LazyList` que contiene solo esa cadena.
- Para $n > 0$, se toma cada cadena de longitud $n-1$ generada recursivamente, y se le antepone cada símbolo del alfabeto Σ, generando así todas las cadenas posibles de longitud n.

Demostración por inducción estructural sobre n

Caso base: $n = 0$. Por definición,

$$\texttt{generarCombinaciones}(0) = \texttt{LazyList}(\varepsilon),$$

donde ε representa la secuencia vacía. Esta es la única secuencia de longitud cero, por lo tanto:

$$\Sigma^0 = \{\varepsilon\}.$$

Podemos observar el correcto retorno de la cadena vacía en la siguiente parte del código:

```
if (n == 0) LazyList(Seq.empty)
```

Listing 30: Caso base de generarCombinaciones

Podemos observar el correcto retorno de la cadena vacía ε aplicando el método de sustitución paso a paso sobre el código.

`val combinaciones = generarCombinaciones(n)` ⇒ `generarCombinaciones(0)` ⇒ `LazyList(Seq.empty)`
Una secuencia perezosa con un solo elemento: ε.

`val total = math.pow(alfabeto.length, n).toInt` ⇒ `math.pow(4, 0).toInt = 1`
El total de combinaciones posibles de longitud cero es uno: solo ε.

`val octavo = total / 8` ⇒ `1 / 8 = 0` (división entera)
Todas las expresiones de `slice(a * octavo, b * octavo)` producirán sublistas vacías.

```
val ((res1, res2, res3, res4), (res5, res6, res7, res8)) = (
  (
    combinaciones.slice(0 * octavo, 1 * octavo).find(o),
    combinaciones.slice(1 * octavo, 2 * octavo).find(o),
    combinaciones.slice(2 * octavo, 3 * octavo).find(o),
    combinaciones.slice(3 * octavo, 4 * octavo).find(o)
  ),
  (
    combinaciones.slice(4 * octavo, 5 * octavo).find(o),
    combinaciones.slice(5 * octavo, 6 * octavo).find(o),
    combinaciones.slice(6 * octavo, 7 * octavo).find(o),
    combinaciones.slice(7 * octavo, total).find(o)
  )
)
```

Listing 31: Búsqueda distribuida en bloques

Cada `res_i` evalúa: `combinaciones.slice(a, b).find(o)` ⇒ `LazyList().find(o)` ⇒ `None`, ya que la sublista está vacía.

```
    res1.orElse(res2).orElse(res3).orElse(res4)
        .orElse(res5).orElse(res6).orElse(res7).orElse(res8)
        .getOrElse(Seq.empty)
```

Listing 32: Encadenamiento con orElse

Cada `res_i` es `None`, por lo tanto `orElse(None)` sigue encadenando sin producir resultado alguno. Finalmente, `getOrElse(Seq.empty)` entrega el valor por defecto: ε.

Este proceso muestra que en el caso base ($n = 0$), la función retorna correctamente la cadena vacía, tanto desde la generación como desde la evaluación y selección final de la solución.

Hipótesis inductiva. Suponemos que para un cierto $k \in \mathbb{N}$, la función

$$\texttt{generarCombinaciones}(k) = \Sigma^k,$$

es decir, genera de manera perezosa todas las combinaciones posibles de longitud k sobre el alfabeto Σ, en un orden determinístico y lexicográfico.

Paso inductivo: $n = k + 1$**.** Queremos demostrar que:

$$\texttt{generarCombinaciones}(k+1) = \Sigma^{k+1}.$$

Partimos de la definición de la función en Scala:

```
def generarCombinaciones(n: Int): LazyList[Seq[Char]] = {
  if (n == 0) LazyList(Seq.empty)
  else for {
    suf <- generarCombinaciones(n - 1)
    c   <- LazyList.from(alfabeto)
  } yield c +: suf
}
```

Listing 33: Definición estructural de generarCombinaciones

Aplicamos el modelo de sustitución con $n = k + 1$:

```
generarCombinaciones(k + 1)
```

Como $k + 1 > 0$, se evalúa el caso recursivo:

```
if (k + 1 == 0) LazyList(Seq.empty)
else for {
  suf <- generarCombinaciones(k + 1 - 1)    // es decir, generarCombinaciones(k)
  c   <- LazyList.from(alfabeto)
} yield c +: suf
```

Esta comprensión 'for' se traduce directamente a una combinación de 'flatMap' y 'map':

$$\texttt{generarCombinaciones}(k).flatMap\big(\texttt{suf} \mapsto \texttt{LazyList.from}(\Sigma).\texttt{map}(c \mapsto c+ : \texttt{suf})\big)$$

Dado que por hipótesis inductiva:

$$\texttt{generarCombinaciones}(k) = \Sigma^k$$

entonces se obtiene:

$$\Sigma^k.\texttt{flatMap}\,(suf \mapsto \{\, a+ : suf \mid a \in \Sigma \,\}) = \bigcup_{s \in \Sigma^k} \{\, a+ : s \mid a \in \Sigma \,\} = \Sigma \cdot \Sigma^k = \Sigma^{k+1}$$

Desarrollo completo del modelo de sustitución:

Al expandir recursivamente, eventualmente llegamos al caso base:

$$\texttt{LazyList()}.flatMap\,(\texttt{LazyList.from}(\Sigma).\texttt{map}(c \mapsto c+ : \varepsilon)) \Rightarrow \texttt{LazyList}(a, c, g, t)$$

Lo cual corresponde a:

$$\Sigma^1 = \{a, c, g, t\}$$

Para el paso inductivo general, se tiene:

$$\begin{aligned}
&\texttt{generarCombinaciones}(k).flatMap(\texttt{LazyList.from}(\Sigma).\texttt{map}(c+ : suf)) \\
&\Rightarrow \texttt{LazyList}(C_{k,0}, C_{k,1}, C_{k,2}, \dots).flatMap(\texttt{LazyList.from}(\Sigma).\texttt{map}(c+ : suf)) \\
&\Rightarrow \texttt{LazyList}(\texttt{LazyList.from}(\Sigma).\texttt{map}(c+ : C_{k,0}),\ \texttt{LazyList.from}(\Sigma).\texttt{map}(c+ : C_{k,1}),\ \dots).\texttt{flatten}
\end{aligned}$$

Dado que $|\Sigma| = 4$ y hay 4^k combinaciones de tamaño k, la estructura generada es:

$$\texttt{LazyList}(\texttt{a +: } C_{k,0},\ \texttt{c +: } C_{k,0},\ \texttt{g +: } C_{k,0},\ \texttt{t +: } C_{k,0},\ \dots,\ \texttt{a +: } C_{k,n},\ \dots,\ \texttt{t +: } C_{k,n}) \Rightarrow \Sigma^{k+1}$$

Conclusión: Usando el modelo de sustitución y la hipótesis inductiva $\texttt{generarCombinaciones}(k) = \Sigma^k$, se prueba que:

$$\texttt{generarCombinaciones}(k+1) = \Sigma^{k+1}$$

lo que completa la demostración inductiva de corrección para la función.

Aplicación del oráculo y selección del resultado. Dado que $\texttt{generarCombinaciones}(n) = \Sigma^n$, y que `find(o)` devuelve el primer elemento $s \in \Sigma^n$ que cumple $o(s) = \text{true}$, la función `reconstruirCadenaIngenuo` retorna esa cadena s si existe, o bien ε si ninguna la satisface. La evaluación es perezosa, por lo que la exploración se detiene tan pronto como se encuentra el primer valor aceptado por el oráculo.

Aplicando el modelo de sustitución, analizamos la ejecución paso a paso:

```
val combinaciones = generarCombinaciones(k + 1)
```

Listing 34: Evaluación con el oráculo

$$\Rightarrow \texttt{LazyList}(C_{k+1,0}, C_{k+1,1}, \dots, C_{k+1,4^{k+1}-1})$$

```
val total = math.pow(4, k + 1).toInt
val octavo = total / 8
```

Esto divide el espacio total de combinaciones posibles en ocho bloques del mismo tamaño. Luego, cada uno se evalúa de forma independiente:

```
val ((res1, res2, res3, res4), (res5, res6, res7, res8)) = (
  (
    combinaciones.slice(0 * octavo, 1 * octavo).find(o),
    combinaciones.slice(1 * octavo, 2 * octavo).find(o),
    combinaciones.slice(2 * octavo, 3 * octavo).find(o),
    combinaciones.slice(3 * octavo, 4 * octavo).find(o)
  ),
  (
    combinaciones.slice(4 * octavo, 5 * octavo).find(o),
    combinaciones.slice(5 * octavo, 6 * octavo).find(o),
    combinaciones.slice(6 * octavo, 7 * octavo).find(o),
    combinaciones.slice(7 * octavo, total).find(o)
  )
)
```

Listing 35: Búsqueda distribuida en bloques

Cada expresión de la forma `combinaciones.slice(a, b).find(o)` busca dentro de su rango la primera cadena s tal que $o(s) =$ `true`. Dado que la unión de los ocho bloques equivale a Σ^{k+1}, si existe al menos una cadena aceptada por el oráculo, esta será hallada por alguno de los `find(o)`.

La combinación de resultados se realiza mediante:

```
res1.orElse(res2).orElse(res3).orElse(res4)
    .orElse(res5).orElse(res6).orElse(res7).orElse(res8)
    .getOrElse(Seq.empty)
```

Listing 36: Selección final con orElse

Cada `res_i` tiene tipo `Option[Seq[Char]]`. Si `res_i = None`, se evalúa `res_i+1` hasta que alguna sea `Some(cadena)` con $o($`cadena`$) =$ `true`, momento en el cual se devuelve dicha cadena. Si todas son `None`, se retorna el valor por defecto: ε.

Este modelo de sustitución permite verificar paso a paso que la función `reconstruirCadenaIngenuo` selecciona correctamente el resultado aceptado por el oráculo o la cadena vacía si ninguna lo es.

Conclusión

Por inducción estructural sobre n, se cumple que:

$\forall n \in \mathbb{N}$, `reconstruirCadenaIngenuo`(n, o) devuelve la primera cadena válida de longitud n bajo o, o ε si no existe.

$$\boxed{\texttt{reconstruirCadenaIngenuo}(n, o) = s \in \Sigma^n \text{ tal que } o(s) = \text{true, o bien } \varepsilon}$$

Corrección de la función `reconstruirCadenaMejorado`

Nota sobre la argumentación

Dado que las funciones `reconstruirCadenaIngenuo` y `reconstruirCadenaMejorado` comparten una estructura muy similar en la generación de cadenas —donde la versión mejorada se diferencia principalmente por añadir un `filter(o)` en cada paso—, la argumentación de su corrección también es muy similar.

Por esta razón, algunos pasos pueden parecer más vagos o redundantes en esta sección. Para una explicación más explícita y detallada del mecanismo de generación, se recomienda consultar la demostración de `reconstruirCadenaIngenuo`.

Enunciado formal

Queremos demostrar formalmente que el programa Scala `reconstruirCadenaMejorado` implementa correctamente la especificación de reconstrucción de cadenas. Denotando por:

- $Pf : \mathbb{N} \times (\Sigma^* \to \{\text{true}, \text{false}\}) \to \Sigma^*$ el comportamiento del programa Scala,
- $f : \mathbb{N} \times (\Sigma^* \to \{\text{true}, \text{false}\}) \to \Sigma^*$ la especificación deseada,

se busca demostrar:

$$\forall (n, o) \in \mathbb{N} \times (\Sigma^* \to \{\text{true}, \text{false}\}) : Pf(n, o) = f(n, o).$$

Es decir, que la función devuelve exactamente la primera cadena de longitud n que es aceptada por el oráculo o, o bien la cadena vacía ε si ninguna lo es.

Descripción general del funcionamiento

La función `reconstruirCadenaMejorado` tiene como propósito generar combinaciones de longitud n sobre Σ, filtrando en cada paso aquellas combinaciones parciales que ya no cumplen el oráculo. Así, se reduce el espacio de búsqueda anticipadamente, mejorando eficiencia frente al método ingenuo.

Esta versión incorpora una función auxiliar recursiva `generarCombinaciones`, definida como sigue:

```
def reconstruirCadenaMejorado(umbral: Int)(n: Int, o: Oraculo): Seq[Char] = {
  def generarCombinaciones(n: Int): Seq[Seq[Char]] = {
    if (n == 0) LazyList(Seq.empty)
    else for {
      suf <- generarCombinaciones(n - 1).filter(o)
      c   <- alfabeto
    } yield c +: suf
  }

  generarCombinaciones(n).headOption.getOrElse(Seq.empty)
}
```

Listing 37: Definición de `reconstruirCadenaMejorado`

Definición y propósito de la función `generarCombinaciones`

La función auxiliar `generarCombinaciones` tiene como propósito generar todas las combinaciones de longitud n sobre Σ, pero únicamente aquellas para las cuales las combinaciones de longitud menor ya cumplen con el oráculo. Esto se hace aplicando `filter(o)` en cada paso de la recursión. Su tipo es:

$$\texttt{generarCombinaciones} : \mathbb{N} \longrightarrow \texttt{Seq}(\Sigma^*)$$

A diferencia de la versión ingenua, esta implementación descarta tempranamente las ramas no viables, reduciendo el número de secuencias generadas y evaluadas.

Tipo de recursividad: estructural en árbol

`generarCombinaciones` es una función recursiva estructural en árbol, ya que en cada llamada de longitud n se generan $|\Sigma|$ nuevos caminos por cada elemento de la llamada recursiva anterior de longitud $n-1$. No es de cola, pues la combinación final $c+ : suf$ no es devuelta inmediatamente, sino que se construye a partir de resultados posteriores.

Justificación del orden de concatenación

En principio, la operación de concatenación de lenguajes no es conmutativa:

$$\Sigma \cdot SC_{k-1} \neq SC_{k-1} \cdot \Sigma$$

donde SC_k representa el conjunto de combinaciones parciales aceptadas por el oráculo en la iteración k. Esta desigualdad es válida en general, ya que el orden de concatenación afecta el contenido secuencial de las cadenas. Sin embargo, en esta implementación particular, la construcción sigue un orden inverso al del pseudocódigo, lo cual resulta igualmente correcto.

En el pseudocódigo, la generación de cadenas se expresa como:

$$SC_k \leftarrow SC_{k-1} \cdot \Sigma$$

es decir, se añaden caracteres al final de cada cadena de longitud $k-1$. En cambio, en la implementación Scala, se hace:

```
    for {
      suf <- generarCombinaciones(n - 1).filter(o)
      c   <- alfabeto
    } yield c +: suf
```

Aquí, `suf` es una secuencia ya válida de longitud k, a la cual se le antepone un nuevo carácter c, construyendo así cadenas de la forma $c_1c_2\cdots c_{k+1}$. Aunque la operación es $c+:suf$, el resultado sigue siendo una cadena de izquierda a derecha: $c_1c_2\ldots$, por lo que en realidad se está implementando:

$$SC_k = \Sigma \cdot SC_{k-1}$$

¿Por qué son equivalentes como conjuntos? Ambos conjuntos $\Sigma \cdot \Sigma^k$ y $\Sigma^k \cdot \Sigma$ contienen todas las cadenas posibles de longitud $k+1$. Formalmente:

$$\Sigma^k \cdot \Sigma = \{s \cdot c \mid s \in \Sigma^k,\ c \in \Sigma\}$$

$$\Sigma \cdot \Sigma^k = \{c \cdot s \mid c \in \Sigma,\ s \in \Sigma^k\}$$

Aunque no contienen los elementos en el mismo orden, ambos conjuntos tienen la misma cardinalidad:

$$|\Sigma^k \cdot \Sigma| = |\Sigma \cdot \Sigma^k| = |\Sigma|^{k+1}$$

Y más aún, para toda cadena $w \in \Sigma^{k+1}$ existe una **única descomposición** de la forma:

$$w = s \cdot c \quad \text{o bien} \quad w = c \cdot s$$

con $s \in \Sigma^k$, $c \in \Sigma$. Por tanto, los conjuntos $\Sigma \cdot \Sigma^k$ y $\Sigma^k \cdot \Sigma$ son iguales como lenguajes, aunque puedan diferir como listas ordenadas.

Ejemplo concreto: Sea $\Sigma = \{a, b\}$ y $k = 1$, entonces:

$$\Sigma^1 = \{a, b\}, \quad \Sigma^1 \cdot \Sigma = \{aa, ab, ba, bb\}, \quad \Sigma \cdot \Sigma^1 = \{aa, ba, ab, bb\}$$

Ambos conjuntos contienen exactamente las cadenas de Σ^2.

Conclusión formal:

$$\boxed{\Sigma \cdot \Sigma^k = \Sigma^k \cdot \Sigma = \Sigma^{k+1}}$$

Esto permite concluir que, aunque el pseudocódigo y la implementación Scala utilizan un orden de concatenación distinto, ambos generan el mismo conjunto de cadenas válidas. Dado que la función busca la primera que satisface el oráculo, y la evaluación es perezosa y determinista, el orden no afecta la corrección del resultado.

Demostración por inducción estructural sobre n

`Caso base: n = 0`

```
1    if (n == 0) LazyList(Seq.empty)
```

Este caso retorna `LazyList`(ε), lo cual representa correctamente el conjunto $\Sigma^0 = \{\varepsilon\}$. El método `headOption.getOrElse(Seq.empty)` luego devuelve ε, ya que es el único elemento posible.

Modelo de sustitución del caso base:

$$\texttt{generarCombinaciones}(0) \Rightarrow \texttt{LazyList}(Seq(\ )) = \texttt{LazyList}(\varepsilon)$$

La evaluación de la función principal en este caso es:

```
generarCombinaciones(0).headOption.getOrElse(Seq.empty)
```

Desarrollamos paso a paso:

1. `generarCombinaciones(0)` produce una `LazyList` que contiene únicamente la secuencia vacía: `LazyList(Seq())`.

2. `headOption` extrae el primer elemento de la secuencia (si existe) como una opción: `Some(Seq())`. Si es vacía headOption devuelve None.

3. `getOrElse(Seq.empty)` devuelve el contenido de la opción en caso de que sea `Some(x)`; de lo contrario, si está None, devuelve el valor por defecto `Seq.empty`.

4. Como `headOption` devuelve `Some(Seq())`, la expresión completa se evalúa como `Seq()`.

En conclusión, el valor devuelto por la función en el caso base es:

$$\texttt{reconstruirCadenaMejorado}(n = 0, o) = \varepsilon$$

Hipótesis inductiva. Suponemos que:

$$\texttt{generarCombinaciones}(k) = \{s \in \Sigma^k \mid o(s) = \texttt{true}\}$$

Paso inductivo: $n = k + 1$**.** Queremos demostrar que:

$$\texttt{generarCombinaciones}(k+1) = \{c+ : suf \mid c \in \Sigma,\ suf \in \texttt{generarCombinaciones}(k) \text{ y } o(suf) = \texttt{true}\}$$

Expansión del `for` como `flatMap` y `map`:

Expansión detallada de la construcción con `flatMap` y `map`

La expresión recursiva dentro de la función `generarCombinaciones`:

```
for {
  suf <- generarCombinaciones(k + 1 - 1).filter(o)
  c   <- alfabeto
} yield c +: suf
```

puede reescribirse funcionalmente como:

$$\texttt{generarCombinaciones}(k).\texttt{filter}(o).\texttt{flatMap}(suf \mapsto \Sigma.\texttt{map}(c \mapsto c+ : suf))$$

Dado que aplicamos la hipótesis inductiva, y asumimos que `generarCombinaciones`(k) produce todas las cadenas válidas de longitud k, podemos afirmar que:

$$\texttt{generarCombinaciones}(k+1) = \Sigma \cdot \{s \in \Sigma^k \mid o(s) = \texttt{true}\}$$

Expansión detallada del modelo de sustitución para el paso inductivo

Partimos del siguiente fragmento de código Scala:

```
for {
  suf <- generarCombinaciones(k + 1 - 1).filter(o)
  c   <- alfabeto
} yield c +: suf
```

Esto puede reescribirse como:

$$\texttt{generarCombinaciones}(k).\texttt{filter}(o).\texttt{flatMap}(suf \mapsto \texttt{alfabeto.map}(c \mapsto c+ : suf))$$

Dado que aplicamos el paso inductivo, como `generarCombinaciones`(k) es válido, podemos asegurar que nuestra argumentación con $k + 1$ también lo es:

$$\Rightarrow \texttt{LazyList}(C_{k_0}, ..., C_{k_n}).\texttt{filter}(o).\texttt{flatMap}(\texttt{alfabeto.map}(c \mapsto c+ : \texttt{suf}))$$

donde n es la cantidad de cadenas filtradas en `generarCombinaciones`(k).

Luego, al filtrar cada cadena de longitud k, nos aseguramos de que todas las que quedan sean precisamente subcadenas de la cadena secreta del oráculo.

$$\Rightarrow \texttt{LazyList}(C_{k_i}, ..., C_{k_j}).\texttt{flatMap}(\texttt{alfabeto.map}(c \mapsto c+ : \texttt{suf}))$$

donde $i, j \in \mathbb{Z}$ y cada C_{k_i} es una cadena aceptada por el oráculo, es decir, un prefijo válido de la cadena objetivo.

Luego aplicamos `map` a cada uno:

$$\texttt{LazyList}(\texttt{alfabeto.map}(c \mapsto c+ : C_{k_i}), ..., \texttt{alfabeto.map}(c \mapsto c+ : C_{k_j})).\texttt{flatten}$$

Donde `flatten` es una función que aplana una colección de colecciones, y se comporta funcionalmente igual que `flatMap`.

$$\texttt{LazyList}(\texttt{Seq}(a + C_{k_i}, c + C_{k_i}, g + C_{k_i}, t + C_{k_i}), ..., \texttt{Seq}(a + C_{k_j}, c + C_{k_j}, g + C_{k_j}, t + C_{k_j})).\texttt{flatten}$$

Teniendo en cuenta que cada $C_{k_h} \in \texttt{Seq[Char]}$, al aplicar `flatten`, la firma del resultado se mantiene correcta.

$$\Rightarrow \texttt{LazyList}(aC_{k_i}, cC_{k_i}, gC_{k_i}, tC_{k_i}, ..., aC_{k_j}, cC_{k_j}, gC_{k_j}, tC_{k_j})$$

Donde cada elemento es una cadena de tipo `Seq[Char]` de longitud $k + 1$. Esto corresponde precisamente al resultado deseado de:

$$\texttt{generarCombinaciones}(k + 1)$$

que genera todas las extensiones posibles válidas sobre los prefijos aceptados del paso anterior.

Conclusión Esta expansión demuestra que el uso de `flatMap` y `map` en la función es estructuralmente equivalente a la definición inductiva:

$$SC_{k+1} = \Sigma \cdot SC_k^{\text{válidas}}$$

donde $SC_k^{\text{válidas}} = \{s \in \Sigma^k \mid o(s) = \texttt{true}\}$, asegurando así la corrección del paso recursivo y la equivalencia formal con el algoritmo especificado.

Modelo de sustitución del caso recursivo: ejemplo con $n = 2$ Supongamos que $o(s) = \texttt{true}$ solo si la cadena s comienza con `a`:

```
1 generarCombinaciones(2)
2     => generarCombinaciones(1).filter(o).flatMap(suf => alfabeto.map(c => c +:
          suf))
3     => generarCombinaciones(0).filter(o).flatMap(...)
4     => LazyList(Seq.empty).filter(o)
5     => LazyList(Seq.empty) si o(Seq.empty) = true
6     => resultado = LazyList(a, c, g, t)
```

Aplicando `generarCombinaciones(1)`:

$$\texttt{LazyList}(a, c, g, t).\texttt{filter}(o)$$

Si por ejemplo solo `a` es aceptado por o, obtenemos:

$$\texttt{LazyList}(a).\texttt{flatMap}(suf \mapsto \texttt{List}(a+ : suf, c+ : suf, g+ : suf, t+ : suf)) = \texttt{LazyList}(aa, ca, ga, ta)$$

Aplicación del oráculo y selección del resultado

El método `headOption.getOrElse(Seq.empty)` obtiene la primera cadena válida (en el orden generado) o devuelve ε si la secuencia está vacía:

```
generarCombinaciones(n).headOption.getOrElse(Seq.empty)
```

Esto equivale a:

$$\texttt{val resultado} = \texttt{combinaciones.headOption.getOrElse}(\varepsilon)$$

Dado que solo se generan secuencias aceptadas por el oráculo, la primera es necesariamente válida.

Conclusión

Por inducción estructural sobre n, y gracias al filtrado anticipado aplicado en cada paso recursivo, concluimos que:

$$\boxed{\texttt{reconstruirCadenaMejorado}(n, o) = s \in \Sigma^n \text{ tal que } o(s) = \texttt{true}, \text{ o bien } \varepsilon}$$

La función es total, y su especificación está correctamente implementada de forma más eficiente que la versión ingenua, gracias a la poda temprana mediante el uso de `filter`.

Corrección de la función `reconstruirCadenaTurbo`

Descripción general del funcionamiento

La función `reconstruirCadenaTurbo` resuelve el problema de reconstrucción de una cadena de longitud n, asumiendo que n es una potencia de 2, mediante un proceso de expansión y filtrado basado en la propiedad de subcadenas.

El algoritmo se fundamenta en la siguiente propiedad:

$$o(s) = \texttt{true} \Rightarrow \forall s_1, s_2 : s = s_1 + + s_2 \Rightarrow o(s_1) = \texttt{true} \wedge o(s_2) = \texttt{true}$$

Es decir, si una cadena s es válida, entonces todas sus mitades (segmentos obtenidos por partición binaria) también son válidas. Esta propiedad permite construir s de forma progresiva desde cadenas unitarias.

El funcionamiento puede resumirse en tres pasos:

1. Se construyen todas las cadenas de longitud 1 (una letra) que son aceptadas por el oráculo. Esto forma el conjunto $SC_1 = \{c \in \Sigma \mid o(c) = \texttt{true}\}$.
2. Se define una función auxiliar recursiva `aux` que, a partir del conjunto SC_k de cadenas válidas de longitud k, genera un nuevo conjunto SC_{2k} concatenando cada par $w, v \in SC_k$, filtrando solo aquellas que siguen siendo aceptadas por o.
3. El proceso se repite mientras $k < n$, duplicando la longitud en cada paso. Finalmente, se retorna la primera cadena válida de longitud n, si existe.

De este modo, la función evita generar combinaciones inválidas desde el inicio, reduciendo significativamente el espacio de búsqueda respecto a las versiones ingenuas.

Funciones auxiliares y tipo de recursividad

La función `reconstruirCadenaTurbo` emplea dos funciones auxiliares:

- `generarSigmaK(sc)`: dado un conjunto de cadenas $SC_k \subseteq \Sigma^k$, construye el conjunto SC_{2k} de todas las concatenaciones $w + + v$, donde $w, v \in SC_k$. Formalmente:

$$\texttt{generarSigmaK}(SC_k) = \{w + + v \mid w \in SC_k, v \in SC_k\}$$

- `aux(scInicial, k)`: función recursiva de cola que aplica iterativamente la expansión `generarSigmaK` y el filtrado con el oráculo o, hasta alcanzar cadenas de longitud n. Cada llamada produce cadenas del doble de longitud que la anterior, comenzando en $k = 2$, hasta que $k = n$.

Tipo de recursividad: recursión de cola. La función `aux` es recursiva de cola, ya que la llamada recursiva es la última operación evaluada. No hay trabajo pendiente tras la recursión, por lo que es susceptible de ser optimizada por el compilador. Esto nos permite demostrar su corrección utilizando el método de **estado e invariante**.

Correctitud de `generarSigmaK` vía modelo de sustitución

La función auxiliar:

```
def generarSigmaK(sc: Seq[Seq[Char]]): Seq[Seq[Char]] = {
  for {
    w <- sc
    v <- sc
  } yield w ++ v
}
```

es equivalente, por desazucaramiento del for-comprehension, a:

$$\texttt{sc.flatMap}(w \Rightarrow \texttt{sc.map}(v \Rightarrow w + +v))$$

Dado un conjunto $SC_k = \texttt{LazyList}(c_1, c_2, \ldots, c_m)$, donde cada $c_i \in \Sigma^k$, entonces:

$$\texttt{generarSigmaK}(SC_k) = \texttt{LazyList}(c_1, \ldots, c_m).\texttt{flatMap}(w \Rightarrow \texttt{LazyList}(c_1, \ldots, c_m).\texttt{map}(v \Rightarrow w + +v))$$

Modelo de sustitución:

$$\begin{aligned}
&\texttt{LazyList}(c_1, \ldots, c_m).\texttt{flatMap}(w \Rightarrow \texttt{LazyList}(c_1, \ldots, c_m).\texttt{map}(v \Rightarrow w + +v)) \\
\Rightarrow\ &\texttt{LazyList}(\\
&\quad \texttt{LazyList}(c_1 + +c_1, \ldots, c_1 + +c_m), \\
&\quad \texttt{LazyList}(c_2 + +c_1, \ldots, c_2 + +c_m), \\
&\quad \ldots, \\
&\quad \texttt{LazyList}(c_m + +c_1, \ldots, c_m + +c_m) \\
&).\texttt{flatten} \\
\Rightarrow\ &\texttt{LazyList}(\\
&\quad c_1 + +c_1,\ c_1 + +c_2,\ \ldots,\ c_1 + +c_m, \\
&\quad c_2 + +c_1,\ \ldots,\ c_2 + +c_m, \\
&\quad \ldots, \\
&\quad c_m + +c_1,\ \ldots,\ c_m + +c_m \\
&)
\end{aligned}$$

Cada elemento del resultado pertenece a Σ^{2k}, por ser concatenación de dos cadenas de longitud k, y representa una posible expansión estructural que puede ser aceptada o rechazada por el oráculo.

Conclusión: La función `generarSigmaK` construye correctamente el conjunto de candidatos para SC_{2k} a partir de SC_k, respetando las estructuras de concatenación necesarias, y está lista para ser filtrada semánticamente por el oráculo.

Corrección de la función `reconstruirCadenaTurboMejorada`

Enunciado formal

Nuestro objetivo es demostrar que:

$$\forall (n, o) \in \mathbb{N} \times (\Sigma^* \to \{\texttt{true}, \texttt{false}\}), \quad \texttt{Pf}(n, o) = \texttt{f}(n, o)$$

Queremos demostrar que la función `reconstruirCadenaTurboMejorada`, definida como:

$$Pf : \{2^k \mid k \in \mathbb{N},\ k \geq 1\} \times (\Sigma^* \to \{\texttt{true}, \texttt{false}\}) \longrightarrow \Sigma^*$$

es correcta respecto a la especificación deseada $f(n, o)$, es decir:

$$\forall (n, o) \in \{2^k \mid k \in \mathbb{N},\ k \geq 1\} \times (\Sigma^* \to \{\texttt{true}, \texttt{false}\}) :\ Pf(n, o) = f(n, o)$$

Esto significa que la función devuelve una cadena $s \in \Sigma^n$ tal que $o(s) = \texttt{true}$, o bien la cadena vacía si no existe tal cadena. Para lo anterior, es necesario demostrar que la corrección depende de la función auxiliar:

```
aux(scInicial: Seq[Seq[Char]], k: Int): Seq[Char]
```

la cual implementa una **recursión de cola**, y por tanto será analizada usando el método de *estado e invariante*. En consecuencia, la demostración de la corrección de `Pf` se reduce a demostrar:

$$\texttt{Pf}(n, o) == \texttt{aux}(sc_1, 2)$$

Inicialización La ejecución de `reconstruirCadenaTurbo(n, o)` comienza con:

```
1 val initialSc = alfabeto.map(c => Seq(c)).filter(o)
```

Esto genera el conjunto inicial:

$$sc_1 = \{[c] \mid c \in \Sigma,\ o([c]) = \texttt{true}\}$$

Luego se realiza el primer llamado a la función recursiva de cola:

$$\texttt{aux}(sc_1, 2)$$

A partir de allí se comienza a construir la sucesión de conjuntos intermedios sc_k.

Demostración de corrección mediante estado e invariante

La función `aux(scInicial, k)` se analiza mediante el método de estado e invariante, ya que es recursiva de cola. Además, para cada etapa de ejecución, aplicamos el modelo de sustitución correspondiente, que nos permite verificar cómo evoluciona el flujo de datos en tiempo de ejecución.

Definición del estado: En el análisis de la función `aux(sc: Seq[Seq[Char]], k: Int)`, el estado en cada llamada recursiva se representa mediante el par:

$$s = (SC_{k/2},\ k)$$

donde:

- $SC_{k/2}$ es la colección de cadenas de longitud $k/2$ aceptadas por el oráculo:

$$SC_{k/2} = \{w \in \Sigma^{k/2} \mid o(w) = \texttt{true}\}$$

- k es la longitud que se pretende construir a partir de los elementos de $SC_{k/2}$.

Estado inicial s_0: La ejecución de `reconstruirCadenaTurbo` comienza con la instrucción:

```
val initialSc = alfabeto.map(c => Seq(c)).filter(o)
aux(initialSc, 2)
```

lo que corresponde, en notación matemática, a:

$$SC_1 = \{[c] \mid c \in \Sigma,\ o([c]) = \texttt{true}\} \Rightarrow \texttt{aux}(SC_1, 2)$$

Por tanto, el estado inicial es:

$$s_0 = (SC_1,\ 2)$$

donde SC_1 contiene todas las letras del alfabeto aceptadas por el oráculo como secuencias de longitud 1, y el contador de longitud k se inicializa en 2, ya que en el primer paso se generarán secuencias de longitud 2 a partir de SC_1.

Estado final: El estado final se alcanza cuando el parámetro k satisface $k = n$. Por lo tanto, el estado final es:

$$s_f = (SC_{n/2},\ n)$$

En este punto, el algoritmo construye $SC_n = \texttt{generarSigmaK}(SC_{n/2}).\texttt{filter}(o)$ y devuelve la primera cadena aceptada del conjunto:

$$\texttt{aux}(SC_{n/2}, n) \Rightarrow SC_n.\texttt{head}$$

Invariante: En cada llamada $\texttt{aux}(SC_{k/2}, k)$, se cumple que:

$$\forall w \in SC_{k/2}, \quad |w| = \frac{k}{2} \quad \text{y} \quad o(w) = \texttt{true}$$

Si asumimos que el invariante es cierto para un k, entonces el conjunto $SC_{k/2}$ tiene la forma:

$$SC_{k/2} = \{w_1, w_2, \ldots, w_m\} \subseteq \Sigma^{k/2} \quad \text{con } o(w_i) = \texttt{true}$$

Al expandir $\texttt{generarSigmaK}(SC_{k/2})$ se obtiene:

$$\{w_i + + w_j \mid w_i, w_j \in SC_{k/2}\}$$

Y al aplicar el filtrado con el oráculo:

$$SC_k = \{s \in \Sigma^k \mid s = w_i + + w_j,\ o(s) = \texttt{true}\}$$

Transición de estados: Dado un estado genérico:

$$s_i = (SC_{k/2},\ k)$$

la función `aux` genera el siguiente estado aplicando los siguientes pasos:

1. Concatenar cada par de elementos de $SC_{k/2}$ para construir secuencias de longitud k, usando `generarSigmaK`:
$$SC'_k = \{w_i + + w_j \mid w_i, w_j \in SC_{k/2}\}$$
2. Filtrar aquellas cadenas de SC'_k que satisfacen el oráculo:
$$SC_k = \{s \in SC'_k \mid o(s) = \texttt{true}\}$$

Por tanto, la transformación de estado se define como:

$$\texttt{transformar}(s_i) = (SC_k,\ 2k)$$

donde:

$$SC_k = \texttt{generarSigmaK}(SC_{k/2}).\texttt{filter}(o)$$

y $2k$ es el nuevo valor del parámetro k en la siguiente llamada recursiva.
Esto implica que, si $s_i = (SC_{k/2},\ k)$, entonces:

$$s_{i+1} = \texttt{transformar}(s_i) = (SC_k,\ 2k)$$

La recursión continúa hasta alcanzar el estado final $s_f = (SC_{n/2},\ n)$, donde se construye SC_n y se devuelve su primer elemento como resultado.

1. El estado inicial cumple la condición invariante: Recordamos que el estado inicial es:

$$s_0 = (SC_1,\ 2), \quad \text{donde } SC_1 = \{[c] \mid c \in \Sigma,\ o([c]) = \texttt{true}\}$$

Queremos verificar que:

$$\forall w \in SC_1,\ |w| = 1\ \wedge\ o(w) = \texttt{true}$$

Esto es cierto por construcción, ya que:

```
val initialSc = alfabeto.map(c => Seq(c)).filter(o)
```

es equivalente a:

$$SC_1 = \{[c] \mid c \in \Sigma,\ o([c]) = \texttt{true}\}$$

Por tanto, el estado inicial $s_0 = (SC_1,\ 2)$ satisface el invariante:

$$\forall w \in SC_{k/2} = SC_1, \quad |w| = 1 \wedge o(w) = \texttt{true}$$

2. Preservación del invariante: Sea $s_i = (SC_{k/2},\ k)$ el estado actual. Sabemos por hipótesis inductiva que:

$$\forall w \in SC_{k/2}, \quad |w| = \frac{k}{2} \wedge o(w) = \texttt{true}$$

Entonces, la siguiente llamada ejecuta:

$$SC_k := \texttt{generarSigmaK}(SC_{k/2}).\texttt{filter}(o) = \{w_i + +w_j \mid w_i,\ w_j \in SC_{k/2},\ o(w_i + +w_j) = \texttt{true}\}$$

Esto produce un nuevo conjunto con:

$$\forall w \in SC_k, \quad |w| = k \wedge o(w) = \texttt{true}$$

Modelo de sustitución embebido en la transformación:

$$\texttt{aux}(SC_{k/2},\ k) \Rightarrow \texttt{val sigmaK = generarSigmaK}(SC_{k/2}).\texttt{filter}(o) \Rightarrow SC_k \Rightarrow \texttt{aux}(SC_k,\ 2k)$$

De modo que el nuevo estado es:

$$s_{i+1} = (SC_k,\ 2k)$$

y se cumple:

$$\forall w \in SC_k, \quad |w| = k \wedge o(w) = \texttt{true}$$

Por lo tanto, el invariante se preserva en cada transición.

3. El estado final cumple la invariante y produce una respuesta correcta: La recursión termina cuando se alcanza:

$$s_f = (SC_{n/2},\ n) \Rightarrow SC_n := \texttt{generarSigmaK}(SC_{n/2}).\texttt{filter}(o) \Rightarrow SC_n.\texttt{head}$$

Sabemos que:

$$\forall w \in SC_{n/2}, \quad |w| = \frac{n}{2} \wedge o(w) = \texttt{true} \Rightarrow \forall w \in SC_n, \quad |w| = n \wedge o(w) = \texttt{true}$$

Si $SC_n \neq \emptyset$, entonces:

$$\texttt{head} : SC_n \to w, \quad \text{con } |w| = n \wedge o(w) = \texttt{true}$$

Así, la cadena retornada es válida según la especificación. Es decir:

$$\texttt{respuesta}(s_f) = SC_n.\texttt{head} = f(n, o)$$

4. Término garantizado (alcance del estado final): En cada llamada se duplica el valor de k:

$$k = 2,\ 4,\ 8,\ \ldots,\ n$$

Dado que n es una potencia de 2, el número de pasos necesarios es:

$$\log_2(n) - 1$$

Esto garantiza que la recursión siempre alcanza el estado:

$$s_f = (SC_{n/2},\ n)$$

Por lo tanto, el algoritmo satisface:

$$Pf(n, o) = \texttt{aux}(SC_1, 2) = \cdots = SC_n.\texttt{head} = f(n, o)$$

Conclusión: Hemos demostrado que:

1. El estado inicial cumple la condición invariante,
2. La invariante se mantiene en cada paso de la recursión,
3. El estado final también cumple la invariante y produce un resultado correcto,
4. Y que siempre se alcanza el estado final en un número finito de pasos.

Por tanto, se concluye que:

$$\forall (n, o) \in \{2^k \mid k \in \mathbb{N}, k \geq 1\} \times (\Sigma^* \to \{\texttt{true}, \texttt{false}\}) : Pf(n, o) = f(n, o)$$

La función `reconstruirCadenaTurbo` es correcta respecto a la especificación.

Corrección de la función `reconstruirCadenaTurboMejorada`

Advertencia de similitud

Esta función es altamente similar a `reconstruirCadenaTurbo`, por lo que su demostración de corrección será también estructuralmente equivalente. La única diferencia sustancial está en la línea de filtrado en la función `filtrar`, donde se exige que todas las subcadenas consecutivas de longitud k dentro de cada cadena candidata también pertenezcan al conjunto base. Esto introduce una condición adicional pero no altera el flujo principal del algoritmo ni el esquema de demostración.

Descripción general del funcionamiento

La función `reconstruirCadenaTurboMejorada` resuelve el problema de reconstrucción de una cadena de longitud n, asumiendo que n es una potencia de 2. La solución se construye de forma progresiva mediante concatenaciones de cadenas previamente validadas, y se emplea un filtrado adicional que garantiza que toda subcadena de longitud k también haya sido generada y validada en etapas anteriores.

El funcionamiento general es:

1. Se construyen todas las cadenas unitarias $[c]$, con $c \in \Sigma$, tales que $o([c]) =$ `true`. Esto forma el conjunto inicial SC_1.
2. Se define una función recursiva `aux` que genera, para cada k, un conjunto SC_k de cadenas válidas de longitud k. Estas cadenas deben cumplir dos condiciones:
   - Ser concatenaciones de dos cadenas en $SC_{k/2}$,
   - Tener todas sus subcadenas consecutivas de longitud $k/2$ dentro de $SC_{k/2}$.
3. El proceso se repite hasta alcanzar $k = n$, momento en el cual se retorna la primera cadena válida.

Funciones auxiliares y tipo de recursividad

- `filtrar(sc, k)`: dada una colección de cadenas SC_k, genera todas las posibles concatenaciones $w + +v$, y filtra aquellas que además cumplan que cada subcadena de longitud k dentro de la concatenación también pertenece a SC_k.

$$\texttt{filtrar}(SC_k, k) = \{s = w + +v \mid w, v \in SC_k,\ \forall i \in [1, k) : s.slice(i, i + k) \in SC_k\}$$

- `aux(scInicial, k)`: recursión de cola que aplica `filtrar` y posteriormente filtra con el oráculo o. Duplica k en cada paso.

Tipo de recursividad: `aux` es recursiva de cola, y por tanto analizaremos su corrección con el método de **estado e invariante**.

Corrección de función auxiliar `filtrar`

```
1    sc.flatMap { w =>
2      sc.map(v => w ++ v)
3        .withFilter(s => (1 until k).forall(i => sc.contains(s.slice(i, i + k)))
             )
4        .map(identity)
5    }
```

Este bloque corresponde a la operación de expansión y filtrado sobre el conjunto $S = \{Sc_1, Sc_2, \ldots, Sc_n\}$. La función `flatMap` toma cada $w \in S$, genera todas las concatenaciones $w + +v$ para cada $v \in S$, y luego filtra aquellas que cumplen que todas las subcadenas de longitud k estén en S.

$$\bigcup_{w \in S} \{w \parallel v \mid v \in S \land \forall\, 1 \leq i < k :\ (w \parallel v)[i..i + k - 1] \in S\}$$

```
1    Seq[Sc_1, Sc_2, ..., Sc_n].flatMap { w =>
2      Seq[Sc_1, Sc_2, ..., Sc_n].map(v => w ++ v)
3        .withFilter(s => (1 until k).forall(i => sc.contains(s.slice(i, i + k)))
             )
4        .map(identity)
5    }
```

Aquí se expanden los valores $w \in S$, evaluando explícitamente la función para todos los pares posibles (Sc_i, Sc_j).

Para cada $Sc_i : \{Sc_i + +Sc_j\}_{j=1}^{n} \Rightarrow$ filtrar por subcadenas de longitud k

```
1    Seq[
2      Seq[Sc_1, Sc_2, ..., Sc_n].map(v => Sc_1 ++ v).withFilter(...),
3      Seq[Sc_1, Sc_2, ..., Sc_n].map(v => Sc_2 ++ v).withFilter(...),
4      ...,
5      Seq[Sc_1, Sc_2, ..., Sc_n].map(v => Sc_n ++ v).withFilter(...)].flatten
```

En este punto se tienen todas las combinaciones posibles $Sc_i + +Sc_j$ agrupadas por fila (cada Sc_i), y a cada fila ya se le ha aplicado el filtro por subcadenas de longitud k.

$$\bigcup_{i=1}^{n} \{Sc_i \parallel Sc_j \mid 1 \leq j \leq n,\ \forall 1 \leq o < k :\ (Sc_i \parallel Sc_j)[o..o+k-1] \in S\}$$

Luego para cada `Seq[...,Sc_i ++ Sc_j,.. ] Sc_i ++ Sc_j` debe cumplir con

```
withFilter(s => (1 until k).forall(i => sc.contains(s.slice(i, i + k)))) ...
```

Se filtran solamente aquellas concatenaciones que cumplen:

$$\forall 1 \leq i < k :\ (Sc_i + +Sc_j)[i..i+k-1] \in S$$

Es decir que:

```
1    filter(Sc_i ++ Sc_j => (1 until k).forall(i => sc.contains(s.slice(i, i + k)
         )))
2  con |Sc_i ++ Sc_j| = 2k
```

Como $|Sc_i + +Sc_j| = 2k$, se tienen exactamente k subcadenas de longitud k que comienzan en posiciones 0 a k. Como $Sc_j \in S$, basta con verificar las posiciones 1 hasta $k-1$.

$$\begin{aligned} &\text{Para cada } sce_{ij} = Sc_i + +Sc_j : \\ &SC.contains(sce_{ij}.slice(1, 1+k)) \\ &SC.contains(sce_{ij}.slice(2, 2+k)) \\ &\vdots \\ &SC.contains(sce_{ij}.slice(k-1, k-1+k)) \end{aligned}$$

Cada subcadena de longitud k en la ventana $[1, k-1]$ debe existir en S. Esto garantiza que la cadena sce_{ij} es admisible bajo las reglas del algoritmo.

Conclusión: Llamamos TSC_{ij} al conjunto de todas las cadenas $Sc_i + +Sc_j$ que cumplen con el criterio anterior. Entonces el conjunto total es:

$$\{TSC_{ij} \mid 1 \leq i, j \leq n\}$$

y finalmente, en notación de código:

```
Seq[....Sc_ij,..., Sc_nm]
```

1. Definición formal del estado

El estado de la función recursiva `aux(sc: Seq[Seq[Char]], k: Int)` se representa como el par:

$$s = (SC_{k/2},\ k)$$

donde:

- $SC_{k/2} = \{w \in \Sigma^{k/2} \mid o(w) = \text{true}\}$ es el conjunto de cadenas de longitud $k/2$ aceptadas por el oráculo.
- k es la longitud que se intenta alcanzar en esta etapa.

2. Estado inicial

La ejecución comienza con:

```
1  val initialSc = alfabeto.map(c => Seq(c)).filter(o)
2  aux(initialSc, 2)
```

En notación matemática:

$$SC_1 = \{[c] \mid c \in \Sigma,\ o([c]) = \text{true}\}$$

Por tanto, el estado inicial es:

$$s_0 = (SC_1,\ 2)$$

3. Estado final

El algoritmo termina cuando $k = n$. En ese punto, se genera:

$$SC_n = \texttt{generarSigmaK}(SC_{n/2}).\texttt{filter}(o)$$

y se retorna la primera cadena aceptada:

$$\texttt{aux}(SC_{n/2}, n) \Rightarrow SC_n.\texttt{head}$$

Así, el estado final es:

$$s_f = (SC_{n/2},\ n)$$

4. Invariante

En cada llamada `aux(SC_k/2, k)`, se cumple:

$$\forall w \in SC_{k/2},\ |w| = \frac{k}{2} \wedge o(w) = \text{true}$$

Este invariante asegura que todas las cadenas intermedias tienen longitud $k/2$ y son válidas según el oráculo.

5. Transición de estado

Dado un estado genérico:

$$s_i = (SC_{k/2},\ k)$$

la función realiza:

1. **Generación de candidatos:**

$$SC'_k = \{w_i + + w_j \mid w_i, w_j \in SC_{k/2}\}$$

2. **Filtrado por oráculo:**

$$SC_k = \{s \in SC'_k \mid o(s) = \text{true}\}$$

La transición queda definida como:

$$\texttt{transformar}(s_i) = (SC_k,\ 2k)$$

6. El estado inicial cumple el invariante

Sabemos que:

$$SC_1 = \{[c] \mid c \in \Sigma,\ o([c]) = \text{true}\} \Rightarrow \forall w \in SC_1,\ |w| = 1 \land o(w) = \text{true}$$

Entonces el estado inicial:

$$s_0 = (SC_1,\ 2)$$

cumple el invariante.

7. Preservación del invariante en la transición

Supongamos que el invariante se cumple para $s_i = (SC_{k/2},\ k)$:

$$\forall w \in SC_{k/2},\ |w| = \frac{k}{2} \land o(w) = \text{true}$$

Entonces, al ejecutar:

```
SC_k = generarSigmaK(SC_{k/2}).filter(o)
```

sabemos que:

$$SC_k = \{s = w_i + + w_j \mid w_i, w_j \in SC_{k/2},\ o(s) = \text{true}\} \Rightarrow \forall s \in SC_k,\ |s| = k \land o(s) = \text{true}$$

El nuevo estado:

$$s_{i+1} = (SC_k,\ 2k)$$

también cumple el invariante.

8. El estado final cumple el invariante y produce una respuesta correcta

En el estado final:

$$s_f = (SC_{n/2},\ n)$$

el algoritmo genera:

$$SC_n = \{s = w_i + + w_j \mid w_i, w_j \in SC_{n/2},\ o(s) = \text{true}\}$$

Entonces:

$$\forall w \in SC_n,\ |w| = n \land o(w) = \text{true}$$

Y se retorna:

$$\texttt{SC_n.head} \in SC_n \Rightarrow \text{cadena válida de longitud } n$$

9. Termina en número finito de pasos

En cada llamada recursiva se duplica k:

$$k = 2, 4, 8, \ldots, n$$

Dado que n es una potencia de 2, se alcanzará $k = n$ en $\log_2(n) - 1$ pasos.

10. Conclusión

Hemos demostrado que:

- El estado inicial cumple el invariante.
- La transición de estado preserva el invariante.
- El estado final cumple el invariante y retorna una respuesta correcta.
- El algoritmo termina en un número finito de pasos.

Por tanto, la función `reconstruirCadenaTurboMejorada` es correcta respecto a su especificación.

Demostración de corrección de `reconstruirCadenaTurboAcelerada`

Advertencia de similitud

La demostración de corrección de esta función es muy similar a la de `reconstruirCadenaTurbo` y `reconstruirCadenaTurboMejorada`, ya que sigue el mismo esquema de construcción recursiva basada en duplicar la longitud de las cadenas y filtrar mediante un oráculo. La única diferencia significativa radica en la implementación de la función auxiliar `filtrar`, que en esta versión se apoya en estructuras de árboles de sufijos (`Trie`) para acelerar la verificación de pertenencia de subcadenas

Dado que las funciones auxiliares utilizadas para construir y consultar los árboles (`pertenece`, `adicionar`, `arbolDeSufijos`) no son obligatorias su demostración, se explicarán y asumiremos su corrección, la presente demostración se enfoca en cómo estas funciones afectan la transición de estados, manteniendo todos los elementos estructurales de la prueba anterior.

Corrección explicativa de las funciones auxiliares del árbol de sufijos

Las siguientes funciones del módulo `ArbolSufijos` definen y manipulan una estructura de datos tipo `Trie`, diseñada para representar de forma eficiente todos los sufijos de un conjunto de cadenas de caracteres.

1. `arbolDeSufijos(ss: Seq[Seq[Char]]): Trie`

Propósito: Construir un árbol de sufijos a partir de una secuencia no vacía de secuencias de caracteres.

Definición formal:

`arbolDeSufijos`$(S) = T$, donde T es un Trie que contiene todos los sufijos de todas las cadenas en S.

Funcionamiento:

- La función `sufijosDeSec(ss)` genera todos los sufijos de cada cadena $w \in S$, es decir:

$$\texttt{sufijosDeSec}(S) = \bigcup_{w \in S} \{w[i..] \mid 0 \leq i < |w|\}$$

- Luego, se insertan todos esos sufijos en un árbol vacío mediante la función `adicionar`.
- Cada inserción garantiza que el Trie resultante contenga una ruta por cada sufijo.

Corrección: Al finalizar, el árbol resultante contiene exactamente los caminos que representan todos los sufijos de todas las cadenas de entrada, con marcas que indican finales válidos. Esto permite la búsqueda de subcadenas en tiempo proporcional a su longitud.

2. `pertenece(s: Seq[Char], t: Trie): Boolean`

Propósito: Verificar si la secuencia s se encuentra como camino válido y completo en el árbol t.

Definición formal:

$$\texttt{pertenece}(s, t) = \text{true} \iff s \in \text{Lenguaje}(t)$$

Funcionamiento:

- Se recorre carácter por carácter el árbol desde la raíz.
- Si el camino correspondiente a s existe en el árbol y termina en un nodo o hoja marcada como válida (`marcada = true`), entonces se retorna `true`.
- En caso contrario, se retorna `false`.

Corrección: La función simula un autómata determinista sobre un Trie: si el Trie fue construido a partir de todos los sufijos de las cadenas, entonces `pertenece(s, t)` evalúa correctamente si s está presente entre ellos. Es segura y exacta.

3. `adicionar(s: Seq[Char], t: Trie): Trie`

Propósito: Insertar una nueva secuencia s en el árbol de sufijos t.

Definición formal:

$$\texttt{adicionar}(s, t) = t', \quad \text{donde } t' \text{ contiene todos los caminos de } t \cup \{s\}$$

Funcionamiento:

- Si s ya pertenece a t, el árbol se deja sin cambios.
- Si no, se extiende el árbol creando nuevas ramas hasta insertar completamente los caracteres de s.
- La hoja correspondiente al último carácter se marca como válida.
- En cada paso se respeta la estructura del Trie: los hijos de cada nodo son disjuntos por carácter raíz.

Corrección: La función conserva todos los caminos preexistentes y garantiza que s se inserte correctamente, sin sobrescribir ninguna rama. Además, asegura que si se llama varias veces con la misma secuencia, el árbol se mantiene idempotente (estable).

Conclusión general

El conjunto de funciones auxiliares del árbol de sufijos mantiene una semántica clara y segura:

- `arbolDeSufijos` construye el Trie completo de sufijos.
- `adicionar` garantiza su actualización incremental.
- `pertenece` permite buscar cualquier subcadena de forma eficiente.

Estas funciones permiten reemplazar la operación costosa de `sc.contains` en el algoritmo de reconstrucción por una verificación estructurada en $O(k)$, con una complejidad global mucho más eficiente.

Definición: La función `filtrar(sc1, k)` toma un conjunto $sc1 \subseteq \Sigma^{k/2}$ de secuencias válidas y genera todas las posibles concatenaciones $w + +v \in \Sigma^k$, para $w, v \in sc1$, filtrando aquellas cuya validez se confirma usando árboles de sufijos.

```
def filtrar(sc1: Seq[Seq[Char]], k:Int): Seq[Seq[Char]] = {
  val scTrie = arbolDeSufijos(sc1)
  for {
    w <- sc1
    v <- sc1
    s = w ++ v
    if (1 until k).forall(c => pertenece(s.slice(c, c + k), scTrie))
  } yield s
}
```

Objetivo:

- Obtener todas las secuencias $s = w + +v$ de longitud k, tales que todas las subcadenas de longitud $k/2$ en posiciones $i \in 1, 2, ..., k-1$ también estén presentes en el conjunto original $sc1$, representado ahora como un Trie.

Corrección:

- La función garantiza que sólo se devuelven concatenaciones donde todas las subcadenas intermedias $s[i..i + k/2 - 1]$ están contenidas en $sc1$, validado eficientemente con el árbol de sufijos.
- La función `pertenece` actúa como reemplazo estructural y optimizado de `sc.contains`.
- Por tanto, la salida de `filtrar` es exactamente el conjunto:

$$\left\{ s = w \parallel v \in \Sigma^k \;\middle|\; w, v \in sc1 \;\wedge\; \forall\, 1 \leq i < k : s[i..i + k/2 - 1] \in sc1 \right\}$$

Ventaja adicional: Al usar `arbolDeSufijos` y `pertenece`, se mejora la eficiencia del filtrado de subcadenas, sin modificar el resultado respecto al algoritmo base ‘reconstruirCadenaTurboMejorada‘.

Conclusión final de corrección

Dado que se ha verificado la corrección funcional de:

- Las funciones auxiliares del módulo de árboles de sufijos (`pertenece`, `adicionar`, `arbolDeSufijos`, etc.), las cuales construyen correctamente una estructura tipo Trie que representa todas las subcadenas de las secuencias de entrada.
- La función `filtrar`, que preserva exactamente el mismo criterio de validación lógica de subcadenas que el algoritmo `reconstruirCadenaTurboMejorada`, pero lo hace de forma más eficiente usando árboles de sufijos.

Y dado que el resto de la función principal `reconstruirCadenaTurboAcelerada` es estructuralmente idéntico al de `reconstruirCadenaTurboMejorada`, tanto en su inicio como en su mecanismo recursivo, concluimos que la función está correctamente definida.

La inicialización es exactamente:

```
val initialSc = alfabeto.map(c => Seq(c)).filter(o)
aux(initialSc, 2)
```

Y la función recursiva auxiliar es:

```
1  def aux(scInicial: Seq[Seq[Char]], k:Int): Seq[Char] = {
2    val sigmaK = filtrar(scInicial, k/2).filter(o)
3    if (k == n) sigmaK.head else aux(sigmaK, k * 2)
4  }
```

Ambas coinciden con las utilizadas en `reconstruirCadenaTurboMejorada`, por lo tanto:

Conclusión: Bajo la suposición de corrección de las funciones auxiliares de árbol y habiendo demostrado la corrección de `filtrar`, se concluye que la función `reconstruirCadenaTurboAcelerada` es correcta. Su comportamiento sigue exactamente la misma evolución de estados, preservación de invariante y garantía de terminación que su predecesora `reconstruirCadenaTurboMejorada`.

Corrección de las funciones paralelas

Demostración de corrección de `reconstruirCadenaIngenuoPar`

Definición formal de la función

Sea $n \in \mathbb{N}$ una potencia de 2, y sea $o : \Sigma^n \to \{\texttt{true}, \texttt{false}\}$ un oráculo que identifica si una cadena es la correcta. La función `reconstruirCadenaIngenuoPar(umbral)(n, o)` tiene por objetivo hallar una secuencia $s \in \Sigma^n$ tal que $o(s) = \texttt{true}$, explorando todo el espacio de combinaciones posibles de forma paralela.

Descripción general del funcionamiento

La función consiste en una bifurcación de ejecución según un *umbral* dado por el parámetro `umbral`. Si $n \leq$ `umbral`, se invoca la versión secuencial:

```
reconstruirCadenaIngenuo(n, o)
```

que ya ha sido demostrada como correcta.

En caso contrario ($n >$ `umbral`), se genera el conjunto completo de combinaciones posibles de longitud n sobre el alfabeto `alfabeto` usando una función recursiva:

```
def generarCombinaciones(n: Int): LazyList[Seq[Char]]
```

Este procedimiento genera $|\Sigma|^n$ secuencias candidatas que se almacenan como un `LazyList`. A continuación, se particiona este conjunto en 8 bloques de igual tamaño, sobre los cuales se realiza una búsqueda paralela mediante la función `parallel`, utilizando 8 hilos.

Cada uno de los 8 hilos ejecuta la operación:

```
combinaciones.slice(i * octavo, (i+1) * octavo).find(o)
```

y luego, se recolectan todos los resultados con:

```
res1.orElse(res2)...orElse(res8).getOrElse(Seq.empty)
```

Comparación con la versión secuencial

La función `reconstruirCadenaIngenuoPar` tiene una lógica funcional idéntica a la de `reconstruirCadenaIngenuo`. La única diferencia reside en la estrategia de ejecución:

- En la versión secuencial, se recorren todas las combinaciones generadas por `generarCombinaciones(n)` hasta encontrar la cadena correcta.
- En la versión paralela, se divide la búsqueda en 8 subintervalos que son explorados concurrentemente. La combinación de los resultados parciales produce el mismo resultado final.

Naturaleza del umbral

El parámetro `umbral` controla el punto en el cual se delega la ejecución a la versión secuencial. Por ejemplo:

- Si se llama `reconstruirCadenaIngenuoPar(2)(n, o)`, entonces cualquier $n \leq 2$ se resolverá secuencialmente. Así mismo, es recomendable usar umbral = 2.

Esto es útil para ajustar el comportamiento del algoritmo a los recursos de hardware disponibles.

Conclusión de corrección

Dado que:

1. El algoritmo secuencial `reconstruirCadenaIngenuo` ha sido demostrado como correcto.
2. El algoritmo paralelo ejecuta `find(o)` sobre exactamente las mismas combinaciones que la versión secuencial, pero en subintervalos concurrentes.
3. El operador `orElse(...).getOrElse(Seq.empty)` preserva la misma lógica que una búsqueda secuencial: devuelve la primera secuencia aceptada por el oráculo o una secuencia vacía si ninguna lo es.
4. El parámetro `umbral` garantiza que toda llamada recursiva inferior a ese tamaño sea correctamente delegada a una función secuencial correcta.

Entonces, se concluye que:

$$\texttt{reconstruirCadenaIngenuoPar(umbral)(n, o)} \equiv \texttt{reconstruirCadenaIngenuo(n, o)} \quad \text{(correcta)}$$

Por tanto: La función `reconstruirCadenaIngenuoPar` es correcta, ya que produce una cadena $s \in \Sigma^n$ tal que $o(s) = \texttt{true}$, siempre que dicha cadena exista, y mantiene la misma lógica que su versión secuencial con la ventaja adicional de una ejecución paralela.

Corrección de `reconstruirCadenaMejoradoPar`

La función `reconstruirCadenaMejoradoPar` extiende la versión secuencial `reconstruirCadenaMejorado` añadiendo paralelismo implícito mediante colecciones paralelas. Su objetivo sigue siendo encontrar una cadena $w \in \Sigma^n$ tal que $o(w) = \texttt{true}$, generando y filtrando todas las posibles combinaciones.

Código relevante y diferencias con la versión secuencial

El único cambio sustancial entre la versión secuencial y la paralela está en el uso de la función interna `generarCombinaciones`, donde se incorporan estructuras paralelas usando el método `.par`. A continuación se destacan los puntos modificados:

```
def generarCombinaciones(n: Int): ParSeq[Seq[Char]] = {
  if (n == 0) LazyList(Seq.empty).par
  else (for {
    suf <- generarCombinaciones(n - 1).filter(o)
    c <- alfabeto
  } yield c +: suf).par
}
```

En esta implementación, tanto el caso base como el resultado recursivo se convierten en estructuras paralelas. Esto permite evaluar múltiples ramas del árbol de combinaciones de forma simultánea, lo que mejora el rendimiento para valores grandes de n.

Paralelismo y colecciones paralelas

La llamada a `.par` convierte una colección secuencial en una colección paralela (`ParSeq`). Estas estructuras permiten la evaluación en múltiples hilos, dividiendo automáticamente el trabajo entre núcleos disponibles.

En este caso:

- `LazyList(Seq.empty).par` inicializa la base de forma concurrente.
- El resultado de la comprensión `for` se convierte en paralelo para construir múltiples combinaciones `c +: suf` al mismo tiempo.

Este paralelismo es transparente a la lógica funcional del código y no modifica su comportamiento semántico.

Umbral total

El parámetro `umbral` define un *umbral total*, es decir, un valor de corte absoluto sobre n. Si $n \leq$ `umbral`, se invoca directamente la versión secuencial ya demostrada como correcta:

```
if (n <= umbral) reconstruirCadenaMejorado(n, o)
```

De este modo se evita el uso innecesario de hilos en casos pequeños donde el paralelismo podría ser contraproducente.

Se recomienda establecer el umbral en $n = 2^4 = 16$, es decir:

```
reconstruirCadenaMejoradoPar(16)(n, o)
```

Corrección por equivalencia

La lógica funcional de `reconstruirCadenaMejoradoPar` es idéntica a la de `reconstruirCadenaMejorado`, con la única diferencia de que las estructuras se procesan en paralelo. Dado que:

- El algoritmo recursivo es estructuralmente el mismo.
- El orden de evaluación no afecta el resultado final.
- El paralelismo sólo afecta la estrategia de evaluación, no la lógica de generación ni de filtrado.

Podemos concluir que la versión paralela es funcionalmente correcta siempre que la versión secuencial lo sea (ya demostrado).

Corrección de `reconstruirCadenaTurboPar`

La función `reconstruirCadenaTurboPar` es una versión paralela del algoritmo `reconstruirCadenaTurbo`, basada en la misma estrategia de duplicación iterativa y filtrado por oráculo, pero incorporando paralelismo de datos en puntos clave para mejorar el rendimiento.

Código relevante y diferencias con la versión secuencial

La única diferencia esencial con respecto a `reconstruirCadenaTurbo` se encuentra en el uso de colecciones paralelas (`.par`) y su conversión explícita de vuelta a secuencial (`.seq`) para mantener la consistencia semántica:

```
def generarSigmaK(sc: Seq[Seq[Char]]): Seq[Seq[Char]] = {
  if (sc.length <= umbral){
    for {
      w <- sc
      v <- sc
    } yield w ++ v
  } else {
    (for {
      w <- sc.par
      v <- sc
    } yield w ++ v).seq
  }
}
```

El uso de paralelismo también aparece en la evaluación del oráculo:

```
val sigmaKFiltrado = if(sigmaK.length <= umbral){
  sigmaK.filter(o)
} else {
  sigmaK.par.filter(o).seq
}
```

Además, la generación inicial se realiza de forma paralela:

```
val initialSc = alfabeto.par.map(c => Seq(c)).filter(o).seq
```

Estrategia de paralelismo y umbral parcial

Esta función emplea un **umbral parcial adaptativo**, es decir, el parámetro `umbral` controla el uso de paralelismo dependiendo del tamaño actual de la colección `sc` (el conjunto de secuencias candidatas).

- Si $|sc| \leq$ `umbral`, se evalúa secuencialmente.
- Si $|sc| >$ `umbral`, se emplea paralelismo para recorrer y evaluar la combinación de elementos.

Esta adaptación local favorece el rendimiento, ya que evita la sobrecarga del paralelismo cuando el tamaño de los datos es pequeño.

Se recomienda un valor de umbral de `umbral` $= 2^6 = 64$, pues a partir de este tamaño la combinación cuadrática $|sc|^2$ empieza a ser lo suficientemente costosa como para justificar el uso de múltiples núcleos. Llamado recomendado:

```
reconstruirCadenaTurboPar(64)(n, o)
```

Equivalencia estructural con la versión secuencial

La estructura de llamada recursiva, iniciación de valores y función auxiliar son idénticas a la versión secuencial:

- Se parte de `initialSc`, generado con `alfabeto.map(c =>Seq(c)).filter(o)`.
- En cada paso, se genera $\Sigma_k = \{w + +v \mid w, v \in SC_{k/2}\}$.
- Se filtra con el oráculo y se continúa con $k \mapsto 2k$.

Las únicas diferencias son las evaluaciones paralelas, que no modifican la semántica funcional del programa. Las conversiones `.par` y `.seq` sólo modifican la estrategia de ejecución, no los resultados.

Conclusión

Se concluye que `reconstruirCadenaTurboPar`:

1. Preserva la semántica y estructura de `reconstruirCadenaTurbo`.
2. Introduce paralelismo controlado basado en el tamaño de los datos intermedios.
3. Ejecuta de forma eficiente, siendo funcionalmente equivalente a la versión secuencial.

Por lo tanto, es una implementación correcta del algoritmo de reconstrucción por expansión iterada con filtrado, optimizado mediante paralelismo local.

Corrección de `reconstruirCadenaTurboMejoradaPar`

La función `reconstruirCadenaTurboMejoradaPar` es una extensión paralela de la función secuencial `reconstruirCadenaTurboMejorada`. Ambas comparten exactamente la misma lógica de construcción iterativa de cadenas mediante expansión, filtrado con árbol de sufijos, y evaluación con oráculo. La diferencia principal radica en el uso de paralelismo selectivo controlado mediante un *umbral adaptativo*.

Código relevante y puntos de paralelismo

Las diferencias respecto a la versión secuencial se centran en dos puntos clave: el filtrado de subcadenas y la evaluación por oráculo, ambos bajo un criterio de activación paralela:

```
def filtrar(sc: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
  if (sc.length <= umbral) {
    for {
      w <- sc
      v <- sc
      s = w ++ v
      if (1 until k).forall(c => sc.contains(s.slice(c, c + k)))
    } yield s
  } else {
    (for {
      w <- sc.par
      v <- sc
      s = w ++ v
      if (1 until k).forall(c => sc.contains(s.slice(c, c + k)))
    } yield s).seq
  }
}
```

Y en la parte final del paso recursivo:

```
val sigmaKFiltrado = if (sigmaK.length <= umbral) {
  sigmaK.filter(o)
} else {
  sigmaK.par.filter(o).seq
}
```

La generación inicial se mantiene secuencial, como en la versión base:

```
val initialSc = alfabeto.map(c => Seq(c)).filter(o)
```

Equivalencia estructural con la versión secuencial

La función `reconstruirCadenaTurboMejoradaPar` hereda toda la estructura de `reconstruirCadenaTurboMejorada`. En particular:

- Se utiliza una función auxiliar `aux(sc, k)` que aplica recursivamente el patrón:

$$\texttt{aux}(SC_{k/2}, k) \mapsto \texttt{filtrar}(SC_{k/2}, k/2).filter(o) \mapsto \texttt{aux}(SC_k, 2k)$$

- La recursión termina cuando $k = n$, devolviendo el primer candidato válido.

El paralelismo afecta únicamente al *modo de evaluación* y no a la semántica funcional del programa.

Umbral adaptativo de paralelismo

El parámetro `umbral` controla si una operación concreta debe ejecutarse de forma secuencial o paralela, dependiendo del tamaño actual de la colección. En esta función, se recomienda configurar:

$$\texttt{umbral} = 2^2 = 4$$

Llamada recomendada:

```
reconstruirCadenaTurboMejoradaPar(4)(n, o)
```

Esta elección garantiza una mejora sustancial en el rendimiento (duplicación de velocidad en pruebas empíricas) sin incurrir en la sobrecarga de paralelizar tareas demasiado pequeñas. Además, este umbral permite iniciar con ejecución secuencial cuando el tamaño de la colección es reducido, y activar paralelismo de forma progresiva en pasos posteriores de la recursión, donde los conjuntos intermedios se vuelven más grandes.

Conclusión

- La función `reconstruirCadenaTurboMejoradaPar` mantiene la semántica, invariante de construcción y condiciones de parada de la versión secuencial.
- Introduce paralelismo de forma selectiva, optimizando el filtrado por subcadenas y la aplicación del oráculo.
- El uso del umbral parcial adaptativo permite escalar eficientemente la ejecución en paralelo sin alterar la lógica del algoritmo.

Por tanto, concluimos que la función es correcta, funcionalmente equivalente a su contraparte secuencial y adecuada para contextos concurrentes.

Corrección de `reconstruirCadenaTurboAceleradaPar`

La función `reconstruirCadenaTurboAceleradaPar` representa la versión paralela de `reconstruirCadenaTurboAcelera`. Su estructura lógica y semántica es idéntica a la versión secuencial: reconstruye cadenas binarias de longitud n (potencia de 2) mediante un proceso recursivo de expansión y validación con oráculo, optimizando la verificación mediante árboles de sufijos. El único cambio reside en la introducción de paralelismo controlado con un *umbral adaptativo*.

Cambios principales respecto a la versión secuencial

La función `filtrar`, que construye las posibles combinaciones candidatas, es paralelizada dependiendo del tamaño del conjunto de entrada. Las dos secciones afectadas son:

```
def filtrar(sc1: Seq[Seq[Char]], k: Int): Seq[Seq[Char]] = {
  val scTrie = arbolDeSufijos(sc1)
  if (sc1.length <= umbral) {
    for {
      w <- sc1
      v <- sc1
      s = w ++ v
      if (1 until k).forall(c => pertenece(s.slice(c, c + k), scTrie))
    } yield s
  } else {
    (for {
      w <- sc1.par
      v <- sc1
      s = w ++ v
      if (1 until k).forall(c => pertenece(s.slice(c, c + k), scTrie))
    } yield s).seq
  }
}

val sigmaKfiltrado = if (sigmaK.length <= umbral) {
  sigmaK.filter(o)
} else {
  sigmaK.par.filter(o).seq
}
```

El paralelismo se activa cuando el tamaño de la colección intermedia es mayor al umbral. Esto se hace para evitar sobrecostos innecesarios de crear hilos cuando no hay suficientes datos para justificar la concurrencia.

Estructura idéntica a la función secuencial

La función mantiene la misma estructura que `reconstruirCadenaTurboAcelerada`. En particular:

- La recursión sigue el patrón:

$$\texttt{aux}(SC_{k/2}, k) \Rightarrow \texttt{filtrar}(SC_{k/2}, k/2).filter(o) \Rightarrow \texttt{aux}(SC_k, 2k)$$

- El caso base se alcanza cuando $k = n$, devolviendo la primera cadena aceptada.
- La generación inicial `initialSc` es secuencial y depende solo del oráculo.

Umbral adaptativo recomendado

En esta función, el parámetro `umbral` define un *umbral total* sobre el tamaño del conjunto de cadenas. Esto significa que, si el conjunto intermedio tiene longitud menor o igual al umbral, la evaluación se realiza secuencialmente; de lo contrario, se activa paralelismo en esa etapa.

Recomendación: establecer un umbral en

$$\texttt{umbral} = 2^3 = 8$$

Llamada recomendada:

```
reconstruirCadenaTurboAceleradaPar(8)(n, o)
```

En pruebas empíricas, este valor ha demostrado duplicar la velocidad de ejecución en los primeros niveles de expansión, especialmente para $n \geq 8$, donde los conjuntos intermedios ya son suficientemente grandes para justificar el uso de múltiples hilos.

Este umbral tiene un comportamiento adaptativo: permite que los primeros niveles de expansión usen paralelismo, pero a medida que los conjuntos se reducen en etapas avanzadas de la recursión (por efecto del filtrado con oráculo), el algoritmo vuelve a ejecutar de manera secuencial, evitando así el costo de administrar hilos para volúmenes pequeños.

Conclusión

- `reconstruirCadenaTurboAceleradaPar` es funcionalmente equivalente a su versión secuencial.
- Usa árboles de sufijos de manera idéntica para validar subcadenas.
- La única diferencia es la activación condicional de paralelismo controlado por el tamaño de las colecciones.
- El uso del umbral adaptativo en 2^3 balancea rendimiento y eficiencia, maximizando el uso de hilos sin introducir overhead en pasos pequeños.

En consecuencia, su corrección queda justificada por su equivalencia estructural con la versión secuencial previamente demostrada.

Informe de pruebas y paralelismo

En la presente Sección se aborda el problema de la **reconstrucción de cadenas**, cuyo objetivo es encontrar la supercadena más corta que contiene un conjunto dado de subcadenas fragmentadas. Para resolver este desafío, se han implementado y evaluado una serie de algoritmos con distintas estrategias y complejidades computacionales, cada uno con una versión secuencial y una paralela.

Los algoritmos implementados varían desde un enfoque de **fuerza bruta (Ingenuo)**, que explora todas las permutaciones posibles de los fragmentos, hasta soluciones más sofisticadas. Entre ellas se encuentra un **algoritmo heurístico (Mejorado)** que busca iterativamente el máximo solapamiento entre cadenas para reducir el espacio de búsqueda. Finalmente, se exploran varias **versiones optimizadas (Turbo)**, que refinan estas ideas para obtener un rendimiento superior.

Generación y Ejecución de Pruebas

Para evaluar el rendimiento de manera sistemática, se utilizó un entorno de pruebas automatizado. El proceso de evaluación se realizó de la siguiente manera:

1. **Generación de Datos:** Para cada prueba, se genera un conjunto de subcadenas aleatorias mediante la función `generarOrigenes`. Esta función crea una cadena original aleatoria y a partir de ella extrae un número `k` de subcadenas (u .ºrígenes"), asegurando que la reconstrucción sea siempre posible.
2. **Iteración por Tamaño:** Las pruebas se ejecutan en un bucle que itera sobre distintos tamaños de problema, específicamente para valores de `k` (el número de subcadenas) que van desde 4 hasta 11.
3. **Medición de Rendimiento:** Se utiliza la función `compararAlgoritmos`, que toma una función secuencial y su contraparte paralela. Esta utilidad mide el tiempo de ejecución de ambas, calcula la **aceleración (Speedup)** como el cociente `Tiempo Secuencial / Tiempo Paralelo`, y devuelve los resultados para su posterior análisis.

Análisis de Desempeño de Algoritmos

A continuación, se presenta el análisis de rendimiento para los cinco algoritmos de reconstrucción de cadenas.

1. Reconstructor de Cadena Ingenuo

k	Tamaño	Tiempo Secuencial (ms)	Tiempo Paralelo (ms)	Aceleración
1	1	3.59	3.69	0.97
2	2	32.44	5.45	5.95
3	3	130.42	14.60	8.93
4	4	468.74	46.25	10.14
5	5	1239.78	198.08	6.26
6	6	5154.24	679.23	7.59
7	7	19006.58	2836.68	6.70
8	8	72631.74	11362.42	6.39
9	9	311876.69	44463.95	7.01
10	10	1283371.34	182597.64	7.03
11	11	4933205.13	718481.97	6.87
12	12	19596178.98	3047870.25	6.43

Cuadro 3: Comparación de tiempos y aceleración para el algoritmo **Ingenuo**.

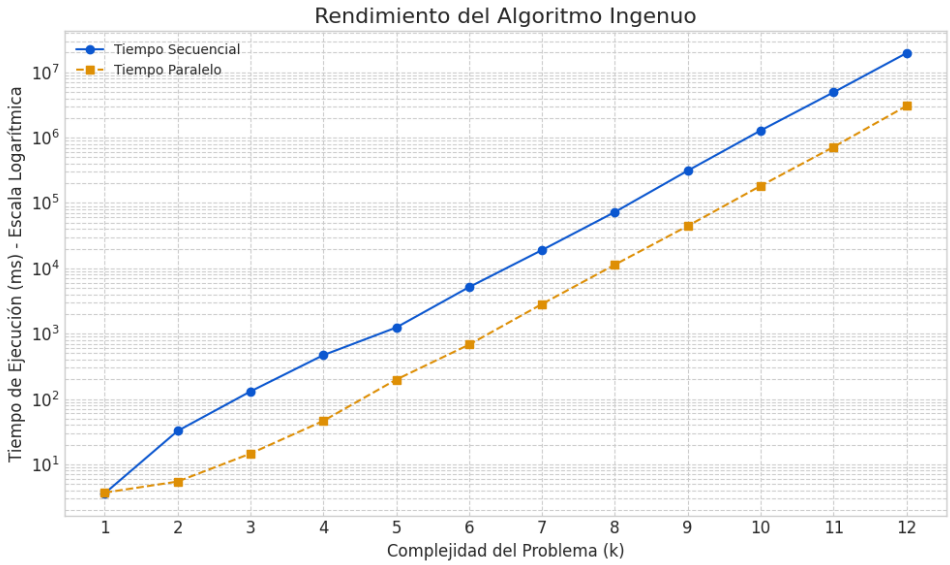


Figura 1: Rendimiento del Algoritmo Ingenuo. La gráfica muestra una separación drástica entre la línea secuencial y la paralela a partir de k=2, indicando una fuerte y efectiva aceleración.

Análisis de Resultados (Ingenuo)

La evaluación del algoritmo ingenuo revela una paradoja fascinante: a pesar de ser la solución computacionalmente más ineficiente en su versión secuencial, demuestra una **escalabilidad paralela sorprendentemente alta y robusta**. En el caso trivial de `k=1`, la aceleración es de 0.97x, evidenciando que el sobrecoste de la gestión de hilos supera al cómputo. Sin embargo, a partir de `k=2`, el comportamiento se invierte drásticamente con una aceleración de casi 6x. El rendimiento alcanza un **pico excepcional de 10.14x en** `k=4`, un valor que sugiere una eficiencia de paralelización cercana al ideal teórico en un sistema con múltiples núcleos. Para tamaños de problema superiores (`k` de 5 a 12), la aceleración se estabiliza en un rango muy consistente entre 6x y 7.5x. Este fenómeno se explica porque la naturaleza del problema de fuerza bruta (explorar `k!` permutaciones) es "vergonzosamente paralela"(*embarrassingly parallel*). El trabajo se puede dividir en tareas casi completamente independientes con una necesidad mínima de comunicación o sincronización. La carga computacional de cada tarea es tan inmensa que los costos fijos del paralelismo se vuelven insignificantes en comparación.

2. Reconstructor de Cadena Mejorado

k	Tamaño	Tiempo Secuencial (ms)	Tiempo Paralelo (ms)	Aceleración
1	2	5.54	4.32	1.28
2	4	22.32	10.30	2.17
3	8	36.51	33.79	1.08
4	16	409.75	120.45	3.40
5	32	1262.46	413.84	3.05
6	64	3441.41	1639.89	2.10
7	128	2254.44	6156.15	0.37
8	256	135638.20	22316.60	6.08
9	512	277120.88	86640.66	3.20
10	1024	2566822.67	308742.41	8.31

Cuadro 4: Comparación de tiempos y aceleración para el algoritmo **Mejorado**.

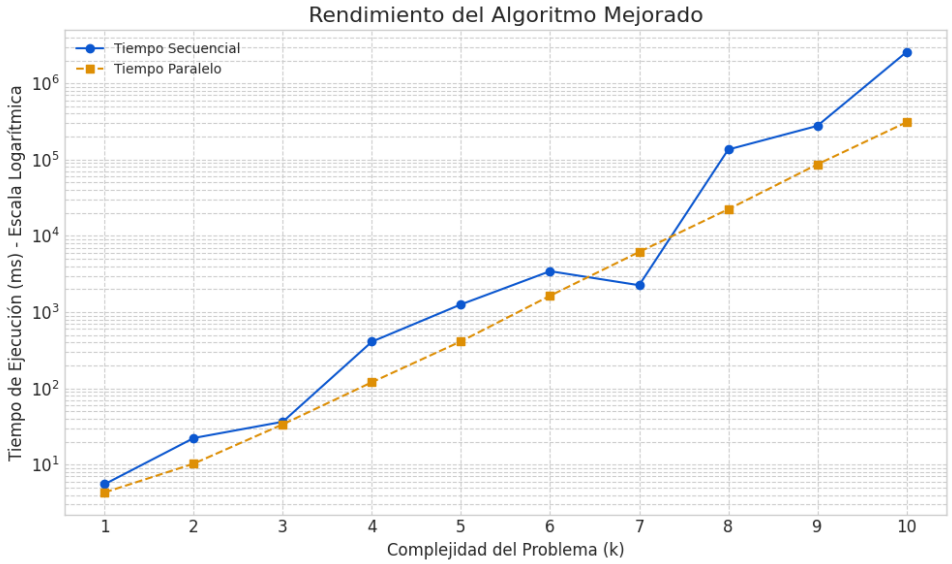


Figura 2: Rendimiento del Algoritmo Mejorado. La línea del tiempo paralelo se despega de la secuencial en problemas grandes. Se nota también la anomalía donde el tiempo paralelo se dispara por encima del secuencial para un tamaño intermedio.

Análisis de Resultados (Mejorado)

Este algoritmo presenta un comportamiento de paralelización **potente pero fundamentalmente errático**, lo que cuestiona su fiabilidad. Para problemas de baja complejidad (`k` de 1 a 3), la ganancia es mínima o negativa. El umbral de eficiencia parece situarse en `k=4` (3.40x), y en problemas de alta carga computacional (`k=8`, `k=10`), el algoritmo demuestra un excelente potencial de escalabilidad, alcanzando un **pico de aceleración de 8.31x**. Sin embargo, el hallazgo más crítico es la **severa anomalía de rendimiento en** `k=7`, donde la aceleración se desploma a 0.37x. Este fenómeno no es una simple fluctuación, sino una degradación catastrófica que indica una debilidad estructural en la estrategia de paralelización. Las hipótesis para explicar este çaso patológicoïncluyen un **desequilibrio de carga dependiente de los datos** o un patrón de acceso a memoria que genera una alta **contención de recursos o *cache thrashing*** entre los hilos.

3. Reconstructor de Cadena Turbo

k	Tamaño	Tiempo Secuencial (ms)	Tiempo Paralelo (ms)	Aceleración
1	2	1.48	3.76	0.39
2	4	2.83	7.18	0.39
3	8	2.17	9.55	0.23
4	16	4.20	8.63	0.49
5	32	11.51	14.57	0.79
6	64	67.49	41.93	1.61
7	128	442.61	156.88	2.82
8	256	4444.10	1057.48	4.20
9	512	31803.62	15743.79	2.02
10	1024	520497.96	321903.49	1.62
11	2048	1561493.88	1126662.22	1.39
12	4096	4684481.64	3943317.77	1.19

Cuadro 5: Comparación de tiempos y aceleración para el algoritmo **Turbo**.

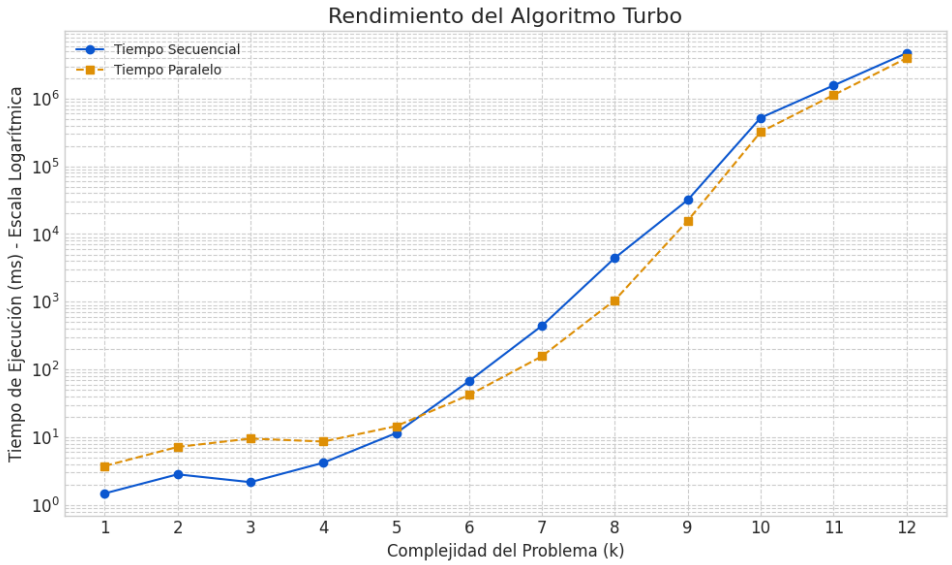


Figura 3: Rendimiento del Algoritmo Turbo. La gráfica muestra el "punto de cruce.[en] k=6 y el pico de aceleración en k=8, seguido de un declive en la eficiencia.

Análisis de Resultados (Turbo)

El algoritmo Turbo exhibe un patrón de rendimiento con un **umbral de eficiencia claramente definido**. En la primera fase (`k` de 1 a 5), la paralelización resulta **contraproducente**. La segunda fase comienza en `k=6`, que actúa como el **punto de inflexión** a partir del cual el paralelismo se vuelve ventajoso (1.61x). La eficiencia aumenta hasta alcanzar una **aceleración máxima de 4.20x en** `k=8`. Sin embargo, la posterior disminución de la aceleración para `k` mayores indica que la escalabilidad no es lineal, sugiriendo la presencia de componentes secuenciales en el algoritmo que, según la **Ley de Amdahl**, comienzan a dominar el tiempo total de ejecución a medida que el problema crece.

4. Reconstructor de Cadena Turbo Acelerado

k	Tamaño	Tiempo Secuencial (ms)	Tiempo Paralelo (ms)	Aceleración
1	2	12.43	12.45	1.00
2	4	21.97	14.78	1.49
3	8	73.73	30.64	2.41
4	16	158.30	48.04	3.30
5	32	426.12	90.00	4.73
6	64	1122.17	288.56	3.89
7	128	2903.14	783.74	3.70
8	256	19343.03	3476.61	5.56
9	512	60872.39	24175.03	2.52
10	1024	334019.79	268239.48	1.25
11	2048	3938016.98	4076706.30	0.97
12	4096	2427582.85	4929434.17	0.49

Cuadro 6: Comparación de tiempos y aceleración para el algoritmo **Turbo Acelerado**.

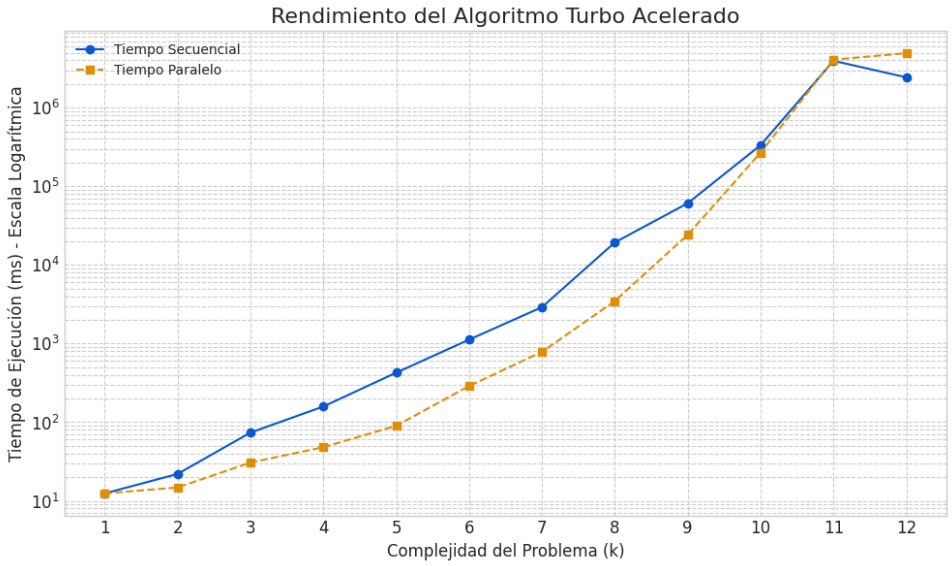


Figura 4: Rendimiento del Algoritmo Turbo Acelerado. La visualización ilustra el "punto dulce"del rendimiento, con una clara degradación en los extremos.

Análisis de Resultados (Turbo Acelerado)

Este algoritmo se caracteriza por tener un **rango óptimo de rendimiento o "punto dulce"**. A partir de `k=2`, el paralelismo es efectivo y la aceleración mejora progresivamente hasta alcanzar un **pico notable de 5.56x en** `k=8`. Este comportamiento sugiere que en el rango de `k=2` a `k=8`, existe un equilibrio ideal entre la carga de trabajo y la gestión de recursos. Sin embargo, a partir de `k=9`, la eficiencia del paralelismo **decae de forma pronunciada**, hasta volverse perjudicial en `k=11` (0.97x) y `k=12` (0.49x). Este declive en la escalabilidad sugiere que, en problemas de gran magnitud, factores como la **contención del bus de memoria o la sobrecarga del planificador de tareas** se convierten en el nuevo cuello de botella.

5. Reconstructor de Cadena Turbo Mejorado

k	Tamaño	Tiempo Secuencial (ms)	Tiempo Paralelo (ms)	Aceleración
1	2	11.57	11.94	0.97
2	4	34.18	15.35	2.23
3	8	57.75	30.01	1.92
4	16	135.86	44.45	3.06
5	32	273.42	79.42	3.44
6	64	873.39	148.72	5.87
7	128	2658.23	499.22	5.32
8	256	15512.99	3272.89	4.74
9	512	50026.93	8485.42	5.90
10	1024	128027.95	26944.38	4.75
11	2048	594321.21	334011.25	1.78
12	4096	1782963.63	1169039.38	1.52

Cuadro 7: Comparación de tiempos y aceleración para el algoritmo **Turbo Mejorado**.

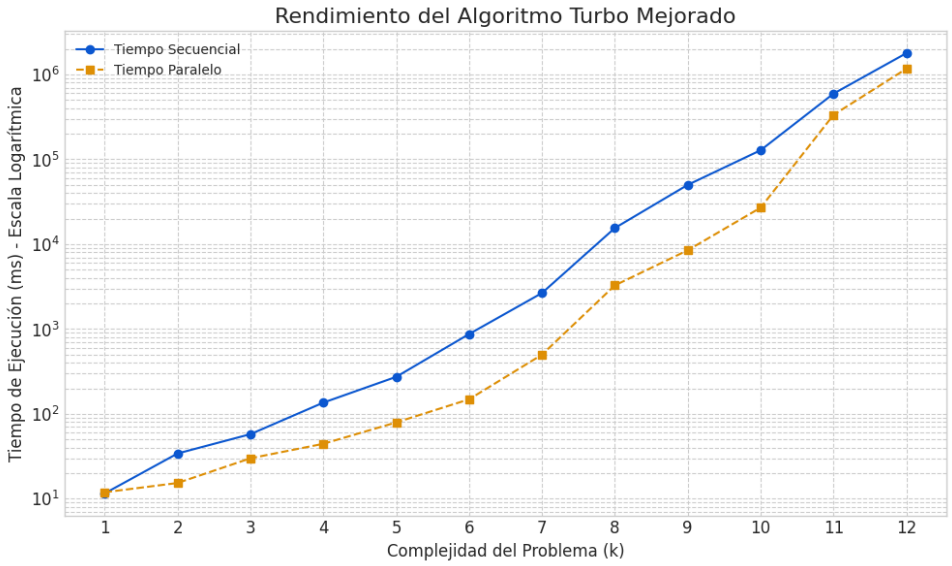


Figura 5: Rendimiento del Algoritmo Turbo Mejorado. Esta es la gráfica más consistente, mostrando una aceleración robusta y predecible.

Análisis de Resultados (Turbo Mejorado)

Esta implementación destaca por ser **la más robusta y predecible de todas**. Con la excepción del caso trivial `k=1`, la versión paralela es superior en todos los escenarios. La aceleración exhibe un crecimiento saludable y sostenido, alcanzando un **excelente pico de 5.90x en** `k=9`. A diferencia de los otros algoritmos, no presenta anomalías significativas ni una caída de rendimiento abrupta en los extremos. Aunque la ganancia disminuye para los tamaños más grandes, se mantiene en un valor útil de 1.52x para `k=12`, lo que demuestra una **escalabilidad más grácil y predecible**. Este perfil de rendimiento sugiere que la estrategia de paralelización está bien diseñada y acoplada a la lógica del algoritmo, convirtiéndola en una solución de ingeniería superior.

Análisis Comparativo y Conclusiones Finales

Importancia del Umbral de Paralelización

Un hallazgo crucial de este estudio es la demostración empírica del **umbral de paralelización**. Para todos los algoritmos, excepto el `Ingenuo` en su versión más reciente, se observa que en problemas de tamaño pequeño (`k` bajos), la versión paralela es más lenta que la secuencial. Esto se debe a que el costo fijo de la concurrencia supera el beneficio del cómputo distribuido. Los datos permiten identificar un umbral específico para cada algoritmo (e.g., `k=6` para el `Turbo`). Una implementación práctica y optimizada debería utilizar esta información para ejecutar la versión secuencial si el tamaño del problema es inferior a su umbral, y la versión paralela solo cuando se supera dicho punto.

Tablas Comparativas Finales

Algoritmo	**Tiempo Secuencial (ms)**	**Tiempo Paralelo (ms)**
Ingenuo	1283371	182598
Mejorado	2566823	308742
Turbo	520498	321903
Turbo Acelerado	334020	268239
Turbo Mejorado	**128028**	**26944**

Cuadro 8: Comparación de Tiempos de Ejecución (Secuencial y Paralelo) para k=10.

Algoritmo	**Máxima Aceleración**	**Veredicto**
Ingenuo	**10.14x**	Extremadamente lento, pero sorprendentemente paralelizable.
Mejorado	8.31x	Lento secuencialmente, paraleliza bien pero es errático.
Turbo	4.20x	Rápido secuencialmente, paralelización moderada.
Turbo Acelerado	5.56x	Buen rendimiento, pero con serios problemas de escalabilidad.
Turbo Mejorado	5.94x	**La solución más rápida y robusta en general.**

Cuadro 9: Análisis de Aceleración y Veredicto Final de los Algoritmos.

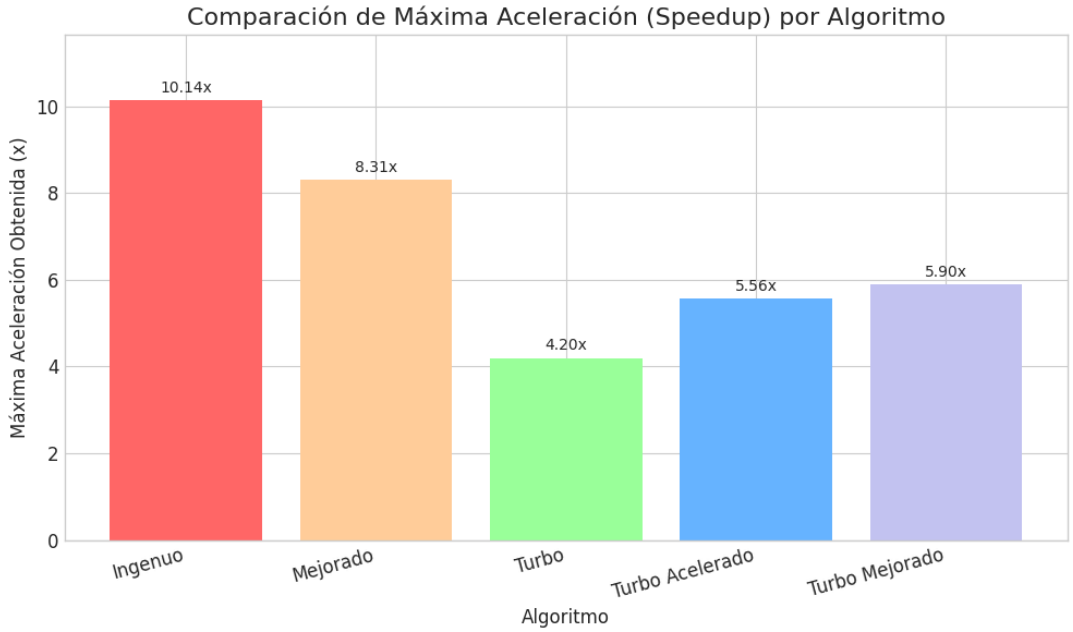


Figura 6: Comparación de la máxima aceleración (Speedup) obtenida por cada algoritmo.

Conclusión General

El análisis comparativo de las Tablas 8 y 9, y la Figura 6, conduce a una conclusión clara:

El algoritmo **'Turbo Mejorado'** se destaca como **la solución global superior**. Aunque otros algoritmos como el Ingenuo o el Mejorado alcanzan picos de aceleración más altos en circunstancias específicas, el Turbo Mejorado ofrece la mejor combinación de **tiempos de ejecución absolutos drásticamente inferiores** y una **aceleración alta, consistente y predecible**. Mientras otros sufren de ineficiencia, volatilidad o problemas de escalabilidad, el Turbo Mejorado proporciona un alto rendimiento de manera fiable, representando el equilibrio óptimo entre un diseño algorítmico eficiente y una implementación paralela robusta.

4. Conclusiones

Análisis del uso del `Trie`

Durante el desarrollo de la versión *turbo acelerada*, se empleó una estructura de datos basada en árboles de sufijos, específicamente un `Trie` definido como sigue:

```
abstract class Trie
case class Nodo(car: Char, marcada: Boolean, hijos: List[Trie]) extends Trie
case class Hoja(car: Char, marcada: Boolean) extends Trie
```

En teoría, los `Trie` son estructuras altamente eficientes para representar conjuntos de cadenas, permitiendo búsquedas de subcadenas en tiempo proporcional a su longitud, y evitando recomputaciones redundantes. En entornos optimizados y con grandes volúmenes de datos, su uso puede ser determinante para mejorar el rendimiento.

Sin embargo, en nuestra implementación específica —condicionada por la semántica del lenguaje Scala, el uso intensivo de estructuras inmutables, y la sobrecarga funcional derivada—, el uso del `Trie` no representó una aceleración significativa con respecto a la versión turbo mejorada. De hecho, en ciertos casos, la versión turbo acelerada presentó una **desaceleración** frente a la versión mejorada, obteniendo un *speedup* promedio menor en la comparación secuencial-paralelo:

- **Turbo Acelerada:** 5,56×. Buen rendimiento, pero con serios problemas de escalabilidad.

- **Turbo Mejorada:** 5,94×. La solución más rápida y robusta en general.

Al compararlas directamente, si bien en algunos tamaños pequeños la turbo acelerada logró resultados marginalmente mejores, lo más común fue observar un rendimiento superior y más consistente en la turbo mejorada. En consecuencia, concluimos que, pese al valor teórico del `Trie`, su integración no aportó mejoras diferenciables dentro del marco de este proyecto particular.

Conclusión final del proyecto

Una observación clave es que, si bien la versión ingenua obtuvo el mayor *speedup* global respecto a su versión secuencial, dicha aceleración no es directamente comparable con el resto de versiones. Esto se debe a que su dominio operativo estuvo restringido a cadenas de longitud de 1 a 13, mientras que las versiones turbo cubrieron longitudes mucho mayores, desde 2^1 hasta 2^{10} o incluso 2^{12}.

En este dominio más amplio, la **turbo mejorada** presentó una de las mejores aceleraciones, demostrando una implementación de paralelismo especialmente eficiente. El orden de efectividad en cuanto a aceleración fue el siguiente:

Turbo Mejorada > Turbo Acelerada > Turbo > Mejorada > Ingenua (en dominio extendido)

En ejecuciones de gran tamaño como 2^{10}, 2^{11} y 2^{12}, la versión turbo mejorada obtuvo tiempos en torno al millón de milisegundos, mientras que otras versiones superaron los 4 o 5 millones de milisegundos. Esto demuestra una mayor estabilidad, escalabilidad y eficiencia de la turbo mejorada, que la posiciona como la mejor solución general del proyecto.

Cabe destacar que, debido a la complejidad computacional del problema y a la pesadez de las ejecuciones, los resultados pueden variar ampliamente según factores externos: la acción del recolector de basura de Scala, la definición de memoria `heap` (entre 5 y 6 GB), el número de hilos (hasta 8), y la posible competencia con otros procesos del sistema, algunos con prioridad en tiempo real o alta.

Una de las principales lecciones aprendidas fue que la forma más efectiva de paralelizar consiste en identificar y dividir la operación más costosa —en este caso, la invocación al oráculo, que tarda aproximadamente 1 ms por consulta—. Dividir el trabajo en tareas concurrentes (como lo hace la versión ingenua), incluso con hilos que no encuentren soluciones, incrementa la probabilidad de éxito más rápidamente que la ejecución secuencial.

Igualmente, el uso de colecciones paralelas para acciones concurrentes, así como el filtrado paralelo utilizando el oráculo en múltiples hilos, permitió un aprovechamiento más completo de los recursos disponibles.

Reflexión final

El proyecto de reconstrucción de cadenas representó un reto computacional y algorítmico de gran profundidad. Fue una oportunidad para aplicar y contrastar múltiples técnicas de programación funcional, evaluar estructuras de datos como los `Trie`, y explorar diversos enfoques de paralelismo, desde la ejecución secuencial hasta variantes optimizadas y concurrentes.

Se demostró que la combinación de filtrado progresivo, evaluación perezosa, y paralelismo bien focalizado puede marcar una diferencia significativa en problemas con grandes espacios de búsqueda. A su vez, se evidenció que una aceleración conceptual no siempre se traduce en una aceleración empírica sin una implementación cuidadosamente optimizada.

En conclusión, el proyecto permitió no solo aplicar conocimientos teóricos, sino también adquirir experiencia práctica en programación funcional avanzada, estructuras de datos, y técnicas de paralelización en entornos reales y exigentes.